EMBEDDED LINUX НА ПРАКТИКЕ

Свой SDK для ARM-плат на mainline-компонентах

Как с нуля собрать TF-A, U-Boot, Linux и Buildroot по отдельности, склеить их в образ SD-карты и заставить его загрузиться на ODROID-GO Advance, BeagleBone Black и Raspberry Pi

По материалам сборки SDK `github.com/sobs1992/linux-sdk`
TF-A v2.15.0 · U-Boot v2026.07 · Linux v7.2.9 · Buildroot 2026.08
Октябрь 2026

Содержание

Предисловие **5**
Что вы узнаете 5
Как читать 5

1. Как загружается ARM-плата **7**
Проблема курицы и яйца 7
Стадии загрузки 7
Цепочка Rockchip целиком 9
Где что лежит на SD-карте 10

2. Kconfig, device tree и тулчейны **11**
Kconfig 11
Device tree подробнее 13
Тулчейны 13

3. Отправная точка: разбор прошлых попыток **15**
Что было в каталоге 15
Как искать причину по следам 15
Что взять с собой 16
Требования к SDK 17

4. Исходники и версии **18**
Какие версии брать 18
Откуда клонировать 18
Сабмодули 19
Кэш загрузок 20
Место на диске 20

5. Архитектура SDK **21**
Идея в одном абзаце 21
Дерево каталогов 21
Поток данных 22
Принципы 22
Конфиг прошивки 22
Выбор прошивки 24

6. Верхний Makefile **25**
Шапка и глобальные переменные 25
Выбор прошивки 25
Пути 26
Kconfig-помощники 27
Компоненты 28
Образ и прошивка карты 32

7. TF-A и U-Boot **33**
TF-A: BL31 для PX30 33
U-Boot для ODROID-GO Advance 33
Адреса загрузки ядра 36

8. Ядро Linux **37**
Какой defconfig взять за основу 37

Что добавить для OGA 37
Сборка 38
Командная строка ядра 39
Устройства ввода 40

9. Корневая файловая система: Buildroot 41
Почему Buildroot 41
BR2_EXTERNAL: свои файлы вне дерева Buildroot 41
Defconfig платы 41
post-build.sh 42
Фрагмент busybox 43
Время и место 43
SDK для приложений 44

10. Образ SD-карты 45
Разметка 45
mkimage.sh по шагам 45
PARTUUID и LABEL 49
Проверка образа без железа 49
Запись на карту 50

11. Отладка: «прошил, вставил — ничего не происходит» 52
Шаг 1. Получить UART 52
Шаг 2. Порт занят 52
Шаг 3. Записать загрузку в файл 52
Шаг 4. Читать мусор 53
Шаг 5. Что должно было печататься 53
Шаг 6. Исключить очевидное 54
Шаг 7. Индикатор без UART 54
Шаг 8. Кто печатает на 1,5 Мбод? 54
Варианты решения 55
Методика в сжатом виде 56

12. Гибридный образ для OGA 57
Разметка 57
Форматы Rockchip 58
Скрипт упаковки 59
Проверка 60
Результат 60

13. Несколько плат: BeagleBone Black и Raspberry Pi 62
Что было зашито под OGA 62
BeagleBone Black 63
Raspberry Pi: другая модель загрузки 64
Сборка нескольких плат 67
Проверка образов 67

14. Ошибки и уроки 68
Нехватка памяти: OOM убил графическую сессию 68
pkill -f убил собственную команду 68
cd в несуществующий каталог 68
unexport FW создал пустую переменную 69
git mv -k молча ничего не сделал 69

[...] && x в конце цикла под set -e 69
Забытый chmod +x 69
Опции Kconfig, которые не включились 69
Параметр ядра, который ни на что не влиял 70
Файл, удалённый из overlay, остался в rootfs 70
Захват UART мешал работать с консолью 70
Неполная первая гипотеза 70
Общие выводы 71

15. Git: сабмодули, форки и обновления 72
Как устроен репозиторий SDK 72
Вносить изменения в компонент 72
Обновить версию компонента 73
Что хранить в SDK, а что — в компоненте 73
.gitignore 73

16. Своя плата: пошаговый план и упражнения 75
Шаг 1. Собрать факты 75
Шаг 2. Создать файлы платы 75
Шаг 3. Собирать по частям и проверять каждую 76
Шаг 4. Первый запуск 76
Упражнения 77

A. Шпаргалка команд 79
SDK 79
Исследование исходников 79
Проверка результатов 79
UART 79
Система 79
U-Boot (консоль платы) 80

B. Справочник переменных конфига прошивки 81
Общие 81
TF-A 81
U-Boot 81
Rockchip legacy (OGA) 81
Блобы прошивки 82
Linux 82
Buildroot 82
Образ 82
Переменные командной строки 83

C. Глоссарий 84

D. Полные листинги 85
Makefile 85
scripts/mkimage.sh 90
configs/rpi4_defconfig 92
board/rpi4/buildroot/defconfig 93

Предисловие

Эта книга — подробный разбор того, как был построен SDK `linux-sdk` для ODROID-GO Advance, а потом расширен на BeagleBone Black и Raspberry Pi. Она написана не как справочник «нажми сюда», а как учебник: на каждом шаге объясняется, **почему** сделано именно так, какие были варианты, где спрятаны ловушки и как их находить самостоятельно.

Исходная задача звучала так:

- собрать SDK для разработки под ODROID-GO Advance (OGA);
- основные компоненты — загрузчик, ядро и корневая файловая система — должны собираться **отдельно**, чтобы каждый можно было держать в своём git-репозитории и вносить туда изменения;
- у каждой «прошивки» должен быть свой конфиг, а после сборки компоненты должны **склеиваться в один `.img`**, который пишется на SD-карту;
- по возможности использовать **mainline** — официальные версии проектов, а не форки производителя.

До этого уже была попытка: ядро и rootfs собрать удалось, а загрузчик «победить» — нет. Поэтому значительная часть книги посвящена загрузчику: как он устроен, как его собрать, как понять, что происходит, когда на экране пусто, и как в итоге выяснилось, что дело было вовсе не в нашем загрузчике.

Что вы узнаете

- Как устроена цепочка загрузки ARM-платы: BootROM, TPL, SPL, ARM Trusted Firmware, U-Boot, ядро, device tree, rootfs.
- Как собирать TF-A, U-Boot, Linux и Buildroot **out-of-tree**, не загрязняя исходники.
- Как устроена система конфигурации Kconfig, что такое defconfig и фрагменты, и почему опция, которую вы попросили включить, может «тихо» не включиться.
- Как без прав root собрать образ SD-карты: таблица разделов, FAT, ext4, raw-области.
- Как отлаживать загрузку через UART, даже когда адаптер не умеет нужную скорость.
- Как один и тот же SDK обслуживает разные платы: Rockchip, TI AM335x, Broadcom.
- Какие ошибки были сделаны по дороге и как их было видно.

Как читать

Часть I (главы 1–2) — теория: загрузка ARM-платы, Kconfig, device tree, тулчейны. Если это знакомо, можно пролистать, но в главе 1 есть детали про Rockchip, без которых дальше будет непонятно.

Часть II (главы 3–10) — построение SDK по шагам: разбор прошлых попыток, исходники, архитектура, Makefile, каждый компонент и сборка образа.

Часть III (главы 11–12) — отладка на железе: как выяснилось, почему образ не грузится, и как появился «гибридный» образ.

Часть IV (глава 13) — обобщение SDK на BeagleBone Black и Raspberry Pi.

Часть V (главы 14–16) — ошибки и уроки, работа с git, пошаговый план для своей платы и упражнения.

В приложениях — шпаргалка команд, справочник переменных конфига, глоссарий и полные листинги ключевых файлов.

> **СОГЛАШЕНИЯ**
> Команды, которые выполняются на ПК, начинаются без приглашения или с `$`. Команды в консоли U-Boot начинаются с `=>`, в консоли платы — с `#`. Пути вида `output/oga/...` даны относительно корня SDK. Листинги с подписью «файл» взяты прямо из репозитория в момент сборки книги.

Врезки в тексте бывают пяти видов: **заметка** — полезная деталь; **ловушка** — место, где легко ошибиться; **опасно** — действие, которое может что-то сломать; **совет** — приём, который экономит время; **упражнение** — задание, чтобы закрепить материал руками.

ГЛАВА 1

Как загружается ARM-плата

Прежде чем что-то собирать, нужно понимать, **что именно** мы собираем и в каком порядке это работает при включении питания. У ПК есть BIOS/UEFI, который сам находит диск и загрузчик. У большинства ARM-плат ничего похожего нет: есть крошечная программа в ПЗУ процессора, и дальше всё зависит от того, что лежит на носителе.

Проблема курицы и яйца

Когда процессор только включился, у него нет работающей оперативной памяти. Чипы DDR нужно сначала **инициализировать**: настроить контроллер памяти, частоты, задержки, провести «тренировку» линий. Код, который это делает, сам должен где-то выполняться — и выполняется он во **внутренней SRAM** процессора, которой обычно от нескольких десятков до пары сотен килобайт.

Отсюда многоступенчатая загрузка: каждая следующая стадия больше и умнее предыдущей, и каждая подготавливает условия для следующей.

Стадии загрузки

BootROM

Код, зашитый в процессор на заводе. Его нельзя изменить. Он умеет совсем немного: перебрать носители в фиксированном порядке (SPI-флеш, eMMC, SD, USB…), найти на носителе загрузчик в **известном ему формате и по известному смещению**, загрузить его в SRAM, проверить заголовок и передать управление.

Порядок перебора носителей и формат заголовка — свои у каждого производителя SoC. Это главная причина, по которой «образ для одной платы» не грузится на другой.

> **ЛОВУШКА**
> BootROM грузит первый **найденный** валидный загрузчик. Если на носителе, который он проверяет раньше SD-карты (SPI-флеш, eMMC), уже есть загрузчик, ваша SD-карта будет проигнорирована — даже если на ней всё идеально. Ровно это и случилось с OGA (глава 11), и это же типичная проблема BeagleBone Black (глава 13).

TPL и SPL

SPL (Secondary Program Loader) — урезанная версия U-Boot, которая помещается в SRAM. Её задача — инициализировать DRAM и загрузить следующую стадию.

На некоторых SoC SRAM настолько мала, что даже SPL туда не влезает целиком. Тогда появляется **TPL** (Tertiary Program Loader) — ещё более крошечная стадия, которая делает только инициализацию DDR. На Rockchip PX30/RK3326 так и устроено:

1. BootROM загружает TPL в SRAM и запускает его.

2. TPL инициализирует DDR и **возвращает управление обратно в BootROM** (опция `CONFIG_TPL_ROCKCHIP_BACK_TO_BROM=y`).
3. BootROM загружает следующую часть — SPL — и запускает её уже с работающей DRAM.

TPL и SPL вместе Rockchip называет **idbloader**: это один файл с заголовком, который BootROM ищет по смещению **32 КиБ (сектор 64)** от начала носителя.

ARM Trusted Firmware (BL31)

64-битные ARM-процессоры (ARMv8) имеют уровни привилегий — **Exception Levels**:

Уровень	Кто там работает
EL3	Secure monitor — самый привилегированный код (TF-A BL31)
EL2	Гипервизор (или U-Boot/ядро, если гипервизора нет)
EL1	Ядро ОС
EL0	Пользовательские программы

После включения процессор находится в EL3. Кто-то должен остаться жить в EL3 и обслуживать запросы ядра: включение и выключение ядер CPU, сон, перезагрузку (интерфейс **PSCI**), а также безопасные сервисы. Этим «кем-то» является **BL31** из проекта **Trusted Firmware-A (TF-A)**. Названия BL1, BL2, BL31, BL32, BL33 пришли из терминологии TF-A:

- **BL31** — runtime-монитор EL3, остаётся в памяти навсегда;
- **BL32** — опциональная безопасная ОС (например, OP-TEE);
- **BL33** — «нормальный мир», то есть U-Boot, который потом запустит ядро.

SPL загружает BL31 и U-Boot, передаёт управление BL31, а тот переключается в EL2 и прыгает в U-Boot. Без BL31 64-битное ядро на Rockchip не сможет, например, поднять второе ядро процессора.

> **32-БИТНЫЕ ПЛАТЫ**
> У BeagleBone Black (Cortex-A8) и Raspberry Pi 1 (ARM1176) нет EL3 в смысле ARMv8, и TF-A им не нужна. На Raspberry Pi 4 роль монитора исполняет «armstub», который прошивка Raspberry Pi подкладывает сама.

U-Boot

Полноценный загрузчик: драйверы MMC, USB, сети, файловых систем, командная строка, скрипты, переменные окружения. Его задача в нашем SDK:

1. Определить модель платы и выбрать подходящий device tree.
2. Найти на SD-карте раздел с ядром.
3. Прочитать конфигурацию загрузки (у нас — `extlinux/extlinux.conf`).
4. Загрузить ядро и device tree в память и передать управление ядру.

Современный U-Boot делает это через **bootstd** («standard boot»): сам сканирует носители и разделы, находит `extlinux.conf` и грузит то, что там написано. Никаких скриптов `boot.ini` писать не нужно.

Ядро и device tree

Ядро ARM, в отличие от x86, не умеет само узнать, какие устройства есть на плате: шины, контроллеры, адреса регистров, прерывания, GPIO. Всё это описывается в **device tree** — текстовом файле `.dts`, который компилируется в бинарный `.dtb`. Загрузчик передаёт ядру адрес `.dtb` в памяти, и ядро по нему создаёт устройства.

Важный вывод: **для каждой ревизии платы нужен свой `.dtb`**. У OGA их три (rev 1.0, Black Edition rev 1.1, ODROID-GO Super), и U-Boot должен выбрать нужный.

Ядро бывает в двух форматах:

- `Image` — несжатый образ ядра arm64, загрузчик кладёт его в память и прыгает;
- `zImage` — самораспаковывающееся ядро arm (32 бита).

Корневая файловая система и init

Ядро монтирует корневую файловую систему (rootfs), указанную в командной строке (`root=...`), и запускает первый процесс — `/sbin/init`. Дальше init поднимает сервисы и запускает `getty` — программу, которая показывает приглашение `login:`.

Цепочка Rockchip целиком

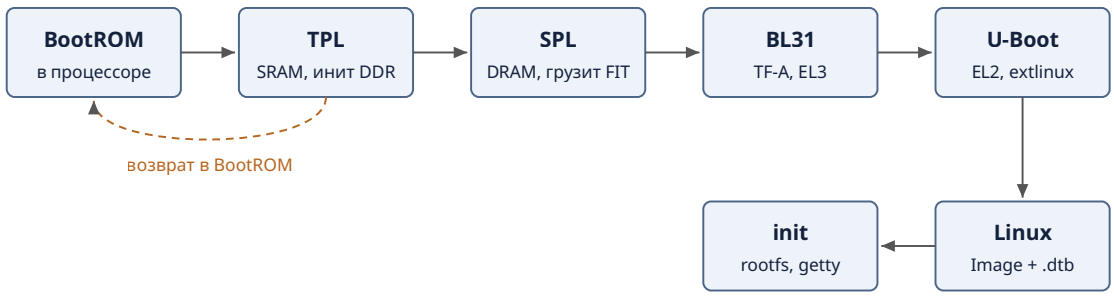


Рис. 1.1. Mainline-цепочка загрузки Rockchip RK3326

FIT-образ u-boot.itb

SPL загружает не один файл, а **FIT** (Flattened Image Tree) — контейнер, который сам устроен как device tree и содержит несколько образов с адресами загрузки: U-Boot, сегменты BL31, несколько `.dtb`. Для OGA FIT выглядит так (вывод `dumpimage -l u-boot.itb`, сокращённо):

```
Image 0 (u-boot)   U-Boot                 Load Address: 0x00800000
Image 1 (atf-1)    ARM Trusted Firmware   Load Address: 0x00040000
Image 2 (atf-2)    ARM Trusted Firmware   Load Address: 0x00058000
Image 3 (atf-3)    ARM Trusted Firmware   Load Address: 0xff020000
Image 4 (fdt-1)    fdt-rockchip/rk3326-odroid-go2
```

```
Image 5 (fdt-2)    fdt-rockchip/rk3326-odroid-go2-v11
Image 6 (fdt-3)    fdt-rockchip/rk3326-odroid-go3
```

BL31 разбит на три сегмента, потому что разные части TF-A должны лежать в разных местах: в DRAM и во внутренней SRAM (`0xff020000`). Три `.dtb` нужны, чтобы SPL мог выбрать правильный для U-Boot по ревизии платы.

U-Boot склеивает idbloader и FIT в один файл `u-boot-rockchip.bin`, который пишется на карту с сектора 64. Внутри него FIT лежит по фиксированному смещению, и SPL ищет его по номеру сектора `CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR`.

> **КАК ЭТО ПРОВЕРИТЬ САМОМУ**
>
> `dumpimage -l output/oga/u-boot/u-boot.itb` показывает содержимое FIT. Если в списке нет `atf-*`, U-Boot собран без BL31 — такой образ не загрузится.

Где что лежит на SD-карте

У каждого SoC свои правила, но общий принцип одинаков: **начало носителя** занимают служебные области (таблица разделов, сырые загрузчики), а **дальше идут разделы** с файловыми системами. Сырые области не принадлежат никакой файловой системе — они пишутся `dd` по номерам секторов. Если раздел случайно перекроет такую область, форматирование раздела сотрёт загрузчик.

Подробная карта нашей SD-карты — в главе 10. Пока достаточно запомнить:

- сектор = 512 байт; сектор 64 = 32 КиБ; сектор 2048 = 1 МиБ; сектор 32768 = 16 МиБ;
- BootROM Rockchip ищет idbloader в секторе 64;
- U-Boot SPL ищет FIT по номеру сектора, заданному в конфиге;
- первый раздел у нас начинается с 16 МиБ, чтобы всё служебное поместилось перед ним.

> **УПРАЖНЕНИЕ**
>
> Посчитайте, в каком секторе начинается байт со смещением 8 МиБ и сколько секторов занимает файл размером 9 469 952 байт. Проверьте, не залезет ли он за 16 МиБ, если писать его с сектора 64. (Ответы: 16384; 18496; конец на ≈ 9,06 МиБ — не залезет.)

ГЛАВА 2

Kconfig, device tree и тулчейны

Три вещи, без которых нельзя ни собрать компоненты, ни понять, почему что-то собралось не так: система конфигурации Kconfig, device tree и кросс-компилятор.

Kconfig

Linux, U-Boot и Buildroot используют одну и ту же систему конфигурации — **Kconfig**. Тысячи опций описаны в файлах `Kconfig` с зависимостями между ними, а выбранная конфигурация хранится в файле `.config`.

Файлы и команды

Что	Зачем
`Kconfig`	описание опций: тип, зависимости, значения по умолчанию
`.config`	**полная** текущая конфигурация (тысячи строк)
`defconfig`	**минимальная** конфигурация: только отличия от значений по умолчанию
`make <name>_defconfig`	развернуть defconfig из дерева исходников в `.config`
`make olddefconfig`	дополнить существующий `.config` значениями по умолчанию для новых опций
`make menuconfig`	интерактивно поменять опции
`make savedefconfig`	свернуть `.config` обратно в минимальный `defconfig`

Defconfig — это то, что хранят в git. `.config` — то, что реально используется при сборке. Defconfig короткий и читаемый, но из-за значений по умолчанию одна и та же строка в defconfig может дать разный `.config` в разных версиях исходников.

Типы опций

- `bool` — `y` или не задана;
- `tristate` — `y` (встроено в ядро), `m` (модуль) или не задана;
- `string`, `int`, `hex` — значения.

Неустановленная опция записывается в `.config` как комментарий: `# CONFIG_FOO is not set`.

Зависимости и «тихое» понижение

Самое коварное в Kconfig — **зависимости ограничивают значение**. Если опция `A` объявлена как `depends on B`, а `B=m`, то `A` не может быть `y` — максимум `m`. Если вы попросите `A=y`, Kconfig **молча** поставит `A=m` и ничего не скажет.

Это случилось в проекте дважды:

```
WARNING: oga.config: requested CONFIG_DRM_PANEL_ELIDA_KD35T133=y, got: ...=m
```

Драйвер панели зависит от `BACKLIGHT_CLASS_DEVICE`, который в arm64 `defconfig` стоит `=m`. Пока не попросили `BACKLIGHT_CLASS_DEVICE=y`, панель оставалась модулем.

```
WARNING: rpi4.config: requested CONFIG_DRM_VC4=y, got: CONFIG_DRM_VC4=m
WARNING: rpi4.config: requested CONFIG_SND=y, got: CONFIG_SND=m
```

`DRM_VC4` объявлен как `depends on SND && SND_SOC`, а `SND` зависит от `SOUND`, который был `=m`. Цепочка: `SOUND=m` → `SND≤m` → `DRM_VC4≤m`. Решение — поднять всю цепочку: `CONFIG_SOUND=y`, `CONFIG_SND=y`, `CONFIG_SND_SOC=y`.

> **КАК НАЙТИ, КТО ОГРАНИЧИВАЕТ ОПЦИЮ**
> Откройте `make menuconfig`, нажмите `/` и введите имя опции. Menuconfig покажет строку `Depends on:` с текущими значениями всех зависимостей, например `Depends on: ... && SND [=m] && SND_SOC [=m]`. Ищите зависимость со значением `m` или `n`. Тот же текст можно найти и прямо в файле Kconfig: `grep -n -A8 '^config DRM_VC4$' drivers/gpu/drm/vc4/Kconfig`.

Фрагменты конфигурации

Хранить полный defconfig ядра для своей платы неудобно: в нём сотни строк, и при обновлении ядра его придётся сливать с изменениями upstream. Удобнее взять **готовый defconfig из дерева исходников** и наложить поверх **фрагмент** — короткий файл только с вашими изменениями:

```
# board/oga/linux/oga.config (фрагмент)
CONFIG_LOCALVERSION="-oga"
CONFIG_DRM=y
CONFIG_DRM_ROCKCHIP=y
CONFIG_JOYSTICK_ADC=y
```

Слияние делает скрипт `scripts/kconfig/merge_config.sh`, который есть и в ядре, и в U-Boot:

```
make O=out defconfig                                          # база
scripts/kconfig/merge_config.sh -m -O out out/.config frag.config
make O=out olddefconfig                                       # дорешать зависимости
```

Ключ `-m` значит «только слить файлы, не запускать make». После слияния обязательно нужен `olddefconfig`, который пересчитает зависимости, и вот тут-то опции могут «понизиться». Поэтому в SDK после каждого слияния запускается `scripts/kconfig-check.sh`, который сравнивает фрагмент с итоговым `.config` и печатает предупреждение для каждой опции, которая не получила запрошенного значения (глава 6).

Out-of-tree сборка

И ядро, и U-Boot умеют собираться **вне дерева исходников**: `make O=<каталог>`. Все объектные файлы, `.config` и результаты складываются в `<каталог>`, а исходники остаются чистыми. Это ключевое требование SDK: исходники — это git-репозитории, куда вы вносите изменения, и мусор от сборки в них не нужен. Кроме того, из одного дерева исходников можно параллельно собирать несколько конфигураций (для разных плат).

> **ЛОВУШКА**
> Если в дереве исходников уже был собран `.config` (сборка «на месте»), ядро откажется собираться с `O=` и попросит `make mrproper`. Клонируйте исходники заново или очистите дерево, прежде чем переходить на out-of-tree.

Device tree подробнее

Исходник `.dts` описывает плату, `.dtsi` — общие части (SoC, семейство плат), которые включаются через `#include`. Например, для OGA:

```
rk3326-odroid-go2-v11.dts      Black Edition (rev 1.1)
  └─ #include "rk3326-odroid-go.dtsi"   общее для OGA/OGS
       └─ #include "rk3326.dtsi" → "px30.dtsi"   SoC
```

Несколько полезных узлов, на которые мы будем смотреть:

```
aliases {
    mmc0 = &sdmmc;          /* SD-карта станет mmcblk0 */
};
chosen {
    stdout-path = "serial2:115200n8";   /* консоль: UART2, 115200 */
};
```

- `aliases` задаёт номера устройств. Если для SD-карты нет алиаса, её номер `mmcblkN` зависит от порядка инициализации — поэтому в главе 13 пришлось монтировать `/boot` по метке, а не по имени устройства.
- `chosen/stdout-path` говорит, куда печатать консоль и на какой скорости. Его читает не только ядро, но и U-Boot, и даже TF-A на Rockchip.

В mainline-ядре 7.x `.dts` лежат по каталогам производителей: `arch/arm64/boot/dts/rockchip/`, `arch/arm/boot/dts/ti/omap/`, `arch/arm/boot/dts/broadcom/`. Отдельный `.dtb` собирается так:

```
make ARCH=arm64 O=out rockchip/rk3326-odroid-go2-v11.dtb
```

Тулчейны

Кросс-компилятор работает на ПК (x86_64), а код генерирует для ARM. Его имя начинается с **префикса** (triplet), который передаётся сборкам через переменную `CROSS_COMPILE`:

Префикс	Для чего
`aarch64-linux-gnu-`	64-битный ARM (OGA, Raspberry Pi 4)
`arm-linux-gnueabihf-`	32-битный ARM с аппаратной плавающей точкой (BBB, Raspberry Pi 1)

`make CROSS_COMPILE=aarch64-linux-gnu-` значит, что вместо `gcc` будет вызван `aarch64-linux-gnu-gcc`, вместо `ld` — `aarch64-linux-gnu-ld` и так далее.

В SDK используются **два вида** тулчейнов:

1. **Системный** (`sudo apt install gcc-aarch64-linux-gnu gcc-arm-linux-gnueabihf`) — для TF-A, U-Boot и ядра. Им не нужна стандартная библиотека C, так что подходит любой свежий GCC.
2. **Buildroot-тулчейн** — Buildroot собирает его сам вместе с glibc для rootfs. Им же собираются все программы в rootfs. Его можно упаковать для разработки приложений (`make rootfs-sdk`, глава 9).

> **ПОЧЕМУ НЕ ОДИН ТУЛЧЕЙН НА ВСЁ**
> Можно и ядро собирать Buildroot-тулчейном. Но тогда сборка ядра будет зависеть от сборки rootfs (сначала нужно собрать тулчейн — это 30–60 минут), а компоненты перестанут быть независимыми. Системный тулчейн ставится за минуту.

> **ЛОВУШКА: `ARM-NONE-EABI-GCC` — НЕ ТО**
> Тулчейн `arm-none-eabi-` предназначен для «голого железа» (микроконтроллеров) без ОС. Для ядра и U-Boot Linux-систем берите `arm-linux-gnueabihf-`. На хосте в этом проекте поначалу был только `arm-none-eabi-gcc`, поэтому для BBB и RPi 1 пришлось ставить `gcc-arm-linux-gnueabihf`.

Целевой процессор

Для rootfs важно указать точное ядро CPU, чтобы компилятор использовал его инструкции и тюнинг:

Плата	SoC	CPU	Buildroot
OGA	RK3326	Cortex-A35	`BR2_cortex_a35=y`
BeagleBone Black	AM335x	Cortex-A8 + NEON	`BR2_cortex_a8=y`, `BR2_ARM_FPU_NEON=y`
Raspberry Pi 1	BCM2835	ARM1176JZF-S (ARMv6)	`BR2_arm1176jzf_s=y`
Raspberry Pi 4/400	BCM2711	Cortex-A72	`BR2_cortex_a72=y`

> **УПРАЖНЕНИЕ**
> В прошлой попытке в Buildroot был указан `BR2_cortex_a53`. Найдите в документации RK3326, какие ядра в нём стоят, и объясните, почему код для A53 на A35 всё равно работает, но это не лучший выбор. (Подсказка: оба — ARMv8-A, но микроархитектура и оптимальное расписание инструкций разные.)

ГЛАВА 3

Отправная точка: разбор прошлых попыток

Хороший SDK начинается не с кода, а с инвентаризации: что уже есть, что работало, что нет и почему. В каталоге `~/Documents/Odroid` лежали шесть экспериментов.

Что было в каталоге

Каталог	Что внутри	Вывод
`Kernel_test/linux`	клон torvalds/linux, v7.2-rc5, собран `Image`	mainline-ядро собирается
`Kernel_test/SD`, `NewSD`	файлы с карты EmuELEC: `boot.ini`, `extlinux/*.conf`, dtb	ядро грузилось **загрузчиком Hardkernel**
`Buildroot_test`	Buildroot 2026.05.1, `rootfs.ext4`	rootfs собирается
`Uboot_test0`	U-Boot от Hardkernel + Linaro GCC 6.3 в Docker	старый вендорский загрузчик
`u-boot_test_2`, `u-boot-test`	тот же U-Boot Hardkernel, свой форк	U-Boot Hardkernel, консоль была
`u-boot_test_mainline`	U-Boot v2026.07 (архив), rkbin, `sdmap.txt`	mainline-загрузчик собран, но **не грузился**

Из этого сразу видно главное: ядро и rootfs собирались и работали, но запускал их вендорский U-Boot от Hardkernel с карты EmuELEC (через `boot.ini` и `sysboot`). Mainline U-Boot был собран — в каталоге лежал готовый `u-boot-rockchip.bin` — но загрузиться не смог.

Как искать причину по следам

Две вещи дали больше всего информации: история shell и файл `sdmap.txt`.

История команд

`grep` по `~/.bash_history` нашёл, как загрузчик писали на карту:

```
sudo dd if=u-boot-rockchip.bin of=/dev/sda bs=512 seek=0 ...
sudo dd if=idbloader.img of=/dev/sda bs=512 seek=64 ...
sudo dd if=u-boot.itb of=/dev/sda bs=512 seek=2048 ...
sudo dd if=u-boot.itb of=/dev/sda bs=512 seek=16384 ...
sudo dd if=trust.img of=/dev/sda bs=512 seek=4096 ...
```

Видно, что пробовались разные смещения. Для mainline-U-Boot на Rockchip правильно только одно из двух:

- либо `u-boot-rockchip.bin` целиком с **сектора 64**;
- либо `idbloader.img` с сектора 64 и `u-boot.itb` с сектора **16384** (значение `CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR=0x4000` по умолчанию).

`seek=0` перетирает таблицу разделов и кладёт idbloader не туда, где его ищет BootROM. `seek=2048` для `u-boot.itb` не совпадает с тем, где его ищет SPL.

Карта разделов

```
label: gpt
/dev/sda1 : start=2048,  size=2048,    name="uboot"
/dev/sda2 : start=4096,  size=4096,    name="trust"
/dev/sda3 : start=8192,  size=20480,   name="boot"
/dev/sda4 : start=28672, size=1048576, name="rootfs"
```

Раздел `boot` занимает сектора 8192–28671 и **перекрывает сектор 16384**, где SPL ищет `u-boot.itb`. Стоит отформатировать такой раздел — и загрузчик стёрт.

Кроме того, это **GPT**. Таблица GPT занимает сектора 1–33, а на Rockchip `CONFIG_ENV_OFFSET=0x4000` — окружение U-Boot хранится с 16 КиБ (сектора 32–63). Сектора 32–33 общие: `saveenv` испортит таблицу разделов, а таблица — окружение.

> **ВЫВОД НА ТОТ МОМЕНТ**
>
> Похоже, проблема в раскладке: разные смещения и раздел поверх `u-boot.itb`. Это была правильная, но **неполная** гипотеза: в главе 11 выяснится, что даже с идеальной раскладкой mainline-загрузчик с SD на этой консоли не стартует вообще.

Что взять с собой

Что	Откуда	Как
Ядро	`Kernel_test/linux` (git, полная история)	локальный `git clone`, затем дотянуть тег из upstream
Кэш загрузок Buildroot	`Buildroot_test/.../dl`	скопировать в `dl/`, чтобы не качать заново
BL31 от Rockchip	`rkbin-master`	как запасной вариант к mainline TF-A
`loaderimage`, `trust_merger`	`rkbin-master/tools`	понадобились позже (глава 12)
Знания о загрузчике Hardkernel	форк `odroid-u-boot`	`build_recovery.sh` раскрыл тайну SPI (глава 11)

Остальное (архивы, собранные деревья, старый тулчейн Linaro GCC 6.3) в новом SDK не нужно: всё берётся из upstream свежих версий.

> **СОВЕТ: ПРЕЖДЕ ЧЕМ ПЕРЕПИСЫВАТЬ — ПРОЧИТАЙТЕ СВОИ СЛЕДЫ**
>
> `~/.bash_history`, файлы `*.txt` рядом с экспериментами, `git log` в форках и `.config` собранных деревьев — это бесплатный журнал того, что уже пробовали. `grep -E 'dd if|seek=|BL31' ~/.bash_history` за секунду показал всю историю попыток записать загрузчик.

Требования к SDK

Сформулируем, что должно получиться:

1. **Независимые компоненты.** TF-A, U-Boot, Linux, Buildroot — каждый в своём git-репозитории (сабмодуле), со своей веткой для изменений.
2. **Out-of-tree.** Сборка не пишет в исходники.
3. **Конфиги прошивок.** Один файл описывает, какие defconfig и фрагменты брать для каждого компонента и как разметить SD-карту. Несколько прошивок сосуществуют.
4. **Склейка.** После сборки компонентов одна команда собирает `.img`.
5. **Mainline.** Официальные версии проектов. Бинарные блобы — только там, где без них нельзя, и с явной пометкой.
6. **Без root для сборки.** `sudo` нужен только для записи на карту.

ГЛАВА 4

Исходники и версии

Какие версии брать

Правило простое: **последний стабильный релиз** каждого проекта. Не `master`, не `-rc`, если только не нужна свежая поддержка железа, которой нет в релизе.

Узнать последние теги можно, не клонируя репозиторий, — командой `git ls-remote`:

```
ls-tags() {
  git ls-remote --tags --refs "$1" | awk -F/ '{print $NF}' |
    grep -E "$2" | sort -V | tail -4
}
ls-tags https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git '^v7\.[0-9]+(\.
[0-9]+)?$'
ls-tags https://source.denx.de/u-boot/u-boot.git '^v20[0-9]{2}\.[0-9]{2}(-rc[0-9])?$'
ls-tags https://github.com/ARM-software/arm-trusted-firmware.git '^v[0-9]+\.[0-9]+\.[0-9]+$'
ls-tags https://gitlab.com/buildroot.org/buildroot.git '^20[0-9]{2}\.[0-9]{2}(\.[0-9]+)?$'
```

На момент сборки SDK ответ был таким:

Проект	Последнее	Выбрано	Почему
Linux stable	v7.2.9; у Торвальдса v7.3-rc6	**v7.2.9**	стабильная ветка с исправлениями
U-Boot	v2026.10-rc5	**v2026.07**	последний релиз; 2026.10 ещё rc
TF-A	v2.15.0	**v2.15.0**	последний релиз
Buildroot	2026.08	**2026.08**	последний релиз

> **STABLE VS MAINLINE**
> «Mainline» в широком смысле — официальный upstream, а не вендорский форк. Ветка `stable` ядра (v7.2.9) — тоже mainline: это v7.2 от Торвальдса плюс только исправления ошибок. Для SDK она лучше, чем свежий `-rc`.

Откуда клонировать

Компонент	Upstream	Замечание
Linux	`git.kernel.org/.../stable/linux.git`	полная история ≈ 5 ГБ
U-Boot	`source.denx.de/u-boot/u-boot.git`	зеркало: `github.com/u-boot/u-boot`
TF-A	`github.com/ARM-software/arm-trusted-firmware`	
Buildroot	`gitlab.com/buildroot.org/buildroot`	

Ядро: переиспользовать локальный клон

Полная история ядра весит гигабайты, а в `Kernel_test/linux` уже лежал клон torvalds/linux. Его можно склонировать локально и дотянуть недостающее:

```
git clone --no-hardlinks ~/Documents/Odroid/Kernel_test/linux src/linux
cd src/linux
```

```
git remote set-url origin https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git
git fetch origin tag v7.2.9 --no-tags      # только этот тег и нужные объекты
git checkout -b oga v7.2.9
```

Локальный клон игнорирует собранные файлы в рабочем дереве (их нет в git) и копирует только историю. `git fetch ... tag v7.2.9` докачивает лишь разницу от v7.2-rc5 до v7.2.9 — мегабайты вместо гигабайт.

> **ПРО --NO-HARDLINKS**
> Без этого ключа локальный `git clone` создаёт для объектов **жёсткие ссылки**, а не копии: клон почти не занимает места, пока оба каталога на одной файловой системе. Это удобно при нехватке места. С `--no-hardlinks` получается полностью независимая копия: старый каталог можно удалять, не думая о связях.

U-Boot: когда upstream медленный

Клон с `source.denx.de` шёл очень медленно (сервер перенаправлял на `git.u-boot-project.org`). Решение — параллельно клонировать официальное зеркало GitHub, взять то, что закончится первым, и **вернуть канонический URL** в `origin`:

```
git clone https://github.com/u-boot/u-boot.git src/u-boot
cd src/u-boot
git remote set-url origin https://source.denx.de/u-boot/u-boot.git
git checkout -b oga v2026.07
```

Содержимое репозитория одинаковое, коммиты те же, хеши те же — неважно, откуда они скачаны.

Ветка `oga`

В каждом репозитории создана локальная ветка `oga` от выбранного тега. Ваши изменения (патчи драйверов, свои defconfig) коммитятся в неё. Пока изменений нет, она просто указывает на upstream-тег.

Сабмодули

Верхний репозиторий SDK хранит компоненты как **git-сабмодули**. Сабмодуль — это ссылка на конкретный коммит в другом репозитории плюс URL, откуда его брать.

```
git init -b main
git submodule add https://source.denx.de/u-boot/u-boot.git src/u-boot
```

Если в `src/u-boot` уже лежит репозиторий, `git submodule add` не клонирует заново, а пишет «Adding existing repo at 'src/u-boot' to the index». Результат — файл `.gitmodules`:

```
[submodule "src/u-boot"]
    path = src/u-boot
    url = https://source.denx.de/u-boot/u-boot.git
```

> **ЛОВУШКА: `BRANCH =` В `.GITMODULES`**
> `git submodule add -b oga` записал бы `branch = oga`. Но в upstream ветки `oga` нет, и `git submodule update --remote` упадёт. Сабмодуль и так пинуется по **коммиту**, поэтому строку `branch` убрали: `git config -f .gitmodules --unset submodule.src/u-boot.branch`.

Склонировать SDK целиком, не скачивая всю историю ядра:

```
git clone --recurse-submodules --shallow-submodules git@github.com:sobs1992/linux-sdk.git
```

Как перевести сабмодуль на свой форк, когда появятся изменения, — в главе 15.

Кэш загрузок

Buildroot скачивает исходники сотен пакетов. Их кэш вынесен в общий каталог `dl/` (переменная `BR2_DL_DIR`), который используется всеми прошивками. Кэш из старой попытки (`Buildroot_test/.../dl`, 342 МБ) скопировали туда, чтобы не качать заново. В `dl/` же складываются блобы прошивки Raspberry Pi (глава 13).

Место на диске

На практике место кончалось дважды. Ориентиры для одной прошивки:

Что	Размер
`output/<fw>/buildroot` (с тулчейном)	5–9 ГБ
`output/<fw>/linux` (arm64 `defconfig`)	≈ 3 ГБ
`output/<fw>/linux` (`omap2plus`, `bcm2835`)	1–2 ГБ
`src/linux` (полная история)	≈ 5,5 ГБ
`dl/`	1–2 ГБ

Проверяйте `df -h` перед полной сборкой новой платы.

ГЛАВА 5

Архитектура SDK

Идея в одном абзаце

Исходники компонентов лежат в `src/` как сабмодули и **никогда** не меняются сборкой. Всё, что специфично для платы, — конфиги, фрагменты, файлы для загрузочного раздела — лежит в `board/<плата>/`. **Конфиг прошивки** `configs/<имя>_defconfig` связывает одно с другим: какие исходники, какие defconfig и фрагменты, какой тулчейн, как разметить карту. Верхний `Makefile` по этому конфигу собирает каждый компонент отдельно в `output/<имя>/`, а скрипт `mkimage.sh` склеивает результаты в `.img`.

Дерево каталогов

```
linux-sdk/
├── Makefile                оркестратор: выбор прошивки, сборка компонентов, образ
├── configs/
│   ├── oga_defconfig       конфиг прошивки ODROID-GO Advance
│   ├── bbb_defconfig       BeagleBone Black
│   ├── rpi1_defconfig      Raspberry Pi 1 / Zero
│   └── rpi4_defconfig      Raspberry Pi 4 / 400
├── board/
│   ├── common/
│   │   ├── buildroot/      общий BR2_EXTERNAL "SDK": пакеты, overlay, post-build
│   │   └── rpi-firmware.sha256
│   ├── oga/
│   │   ├── u-boot/oga.config     фрагмент U-Boot
│   │   ├── linux/oga.config      фрагмент ядра
│   │   ├── buildroot/defconfig   defconfig Buildroot
│   │   └── rkbin/                блобы и утилиты Rockchip
│   ├── bbb/ rpi1/ rpi4/          то же для других плат (+ boot/config.txt у RPi)
├── scripts/
│   ├── mkimage.sh          склейка образа
│   ├── kconfig-check.sh    проверка фрагментов
│   ├── rk-legacy-pack.sh   упаковка uboot.img/trust.img (OGA)
│   ├── fetch-files.sh      загрузка блобов с проверкой sha256
│   └── flash.sh            запись на SD
├── src/                    сабмодули: tf-a, u-boot, linux, buildroot
├── dl/                     кэш загрузок         (в .gitignore)
└── output/                 результаты сборки    (в .gitignore)
```

Поток данных

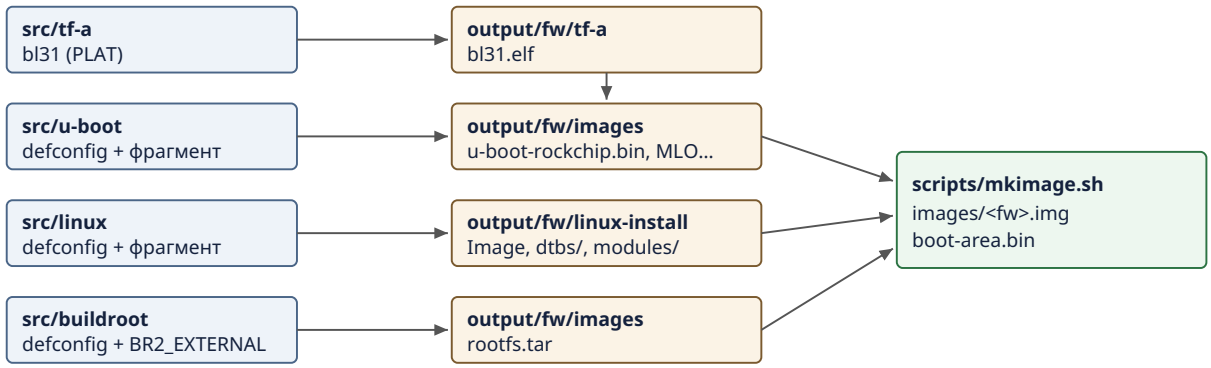


Рис. 5.1. От исходников к образу

Принципы

Компонент собирается одной командой и не знает о других. `make linux` не трогает U-Boot, `make rootfs` не трогает ядро. Единственная связь — U-Boot получает готовый BL31 через переменную `BL31=`.

Модули ядра попадают в rootfs только при склейке. Buildroot ничего не знает о ядре. Ядро ставит модули в `output/<fw>/linux-install/modules`, а `mkimage.sh` распаковывает `rootfs.tar` и докладывает туда модули. Поэтому можно пересобрать ядро и сделать `make image`, не трогая Buildroot.

Конфиг прошивки — единственное место, где описана плата. В Makefile нет ни одного `ifeq ($(BOARD),oga)`. Всё, чем платы отличаются, — переменные в конфиге.

Повторяемость. Версии зафиксированы сабмодулями, блобы — хешами, загрузки — кэшем `dl/`. Один и тот же коммит SDK даёт один и тот же образ.

Без root. Образ собирается через `fakeroot`, `mke2fs -d`, `mtools` и `sfdisk` по файлу. `sudo` нужен только в `flash.sh`.

Конфиг прошивки

Это обычный фрагмент Makefile, который подключается через `include`. Вот конфиг OGA целиком:

configs/oga_defconfig

```
# Firmware: ODROID-GO Advance (rev 1.0 / 1.1 Black Edition) and ODROID-GO Super.
# The board revision is detected by U-Boot at runtime (SARADC ch0) and the
# matching dtb is picked from the boot partition via extlinux "fdtdir".
#
# Paths are relative to the SDK root. *_DEFCONFIG is either an in-tree
# defconfig name or a path to a defconfig file (anything containing '/').
# $(IMAGES_DIR) and $(LINUX_INSTALL) point to this firmware's build outputs.

BOARD_DIR     := board/oga
CROSS_COMPILE := aarch64-linux-gnu-
```

```
# --- Trusted Firmware-A (BL31) ---
TFA_SRC  := src/tf-a
TFA_PLAT := px30

# --- U-Boot ---
UBOOT_SRC       := src/u-boot
UBOOT_DEFCONFIG := odroid-go2_defconfig
UBOOT_FRAGMENTS := $(BOARD_DIR)/u-boot/oga.config
# "tf-a" builds BL31 from TFA_SRC; otherwise a path to a bl31 ELF,
# e.g. $(BOARD_DIR)/rkbin/bin/rk3326_bl31_v1.34.elf (Rockchip rkbin blob)
UBOOT_BL31      := tf-a
# Files copied from the U-Boot build dir to images/
UBOOT_IMAGES    := u-boot-rockchip.bin

# Rockchip legacy boot path. The factory SPI flash of the OGA Black Edition
# holds Rockchip's DDR blob + miniloader, which BootROM runs before the SD
# card; the miniloader then loads uboot.img (sector 16384) and trust.img
# (sector 24576) from SD. With RK_LEGACY=y both are packed from the same
# mainline U-Boot. With an empty SPI flash BootROM runs the mainline
# idbloader at sector 64 instead, so the image boots either way.
RK_LEGACY     := y
RKBIN_DIR     := $(BOARD_DIR)/rkbin
# BL31 for trust.img (the miniloader hands over in Rockchip's own format)
RK_TRUST_BL31 := $(BOARD_DIR)/rkbin/bin/rk3326_bl31_v1.34.elf

# --- Linux ---
LINUX_SRC       := src/linux
LINUX_ARCH      := arm64
LINUX_IMAGE     := Image
LINUX_DEFCONFIG := defconfig
LINUX_FRAGMENTS := $(BOARD_DIR)/linux/oga.config
LINUX_DTBS      := rockchip/rk3326-odroid-go2.dtb \
                   rockchip/rk3326-odroid-go2-v11.dtb \
                   rockchip/rk3326-odroid-go3.dtb
LINUX_CMDLINE   := rootwait rw console=ttyS2,115200n8 console=tty1 fbcon=rotate:3 consoleblan
k=0

# --- Root filesystem (Buildroot) ---
BUILDROOT_SRC       := src/buildroot
BUILDROOT_EXTERNAL  := board/common/buildroot
BUILDROOT_DEFCONFIG := $(BOARD_DIR)/buildroot/defconfig

# --- SD card image ---
# Layout: [MBR][U-Boot env @16K][idbloader @32K][u-boot.itb @1M]
#         [uboot.img @8M][trust.img @12M][p1 FAT32 BOOT @16M][p2 ext4 rootfs]
IMAGE_NAME := oga
# Raw blobs below 16M: file@sector
IMAGE_RAW  := $(IMAGES_DIR)/u-boot-rockchip.bin@64
ifeq ($(RK_LEGACY),y)
IMAGE_RAW  += $(IMAGES_DIR)/uboot.img@16384 $(IMAGES_DIR)/trust.img@24576
endif
# U-Boot builds fdtfile as rockchip/<name>.dtb
IMAGE_DTB_LAYOUT := tree
IMAGE_BOOT_SIZE_MB  := 128
# Free space added on top of the rootfs contents
IMAGE_ROOTFS_FREE_MB := 256
# MBR disk identifier; root= is passed as PARTUUID=<id>-02
IMAGE_DISK_ID := 0x4f474131
```

Обратите внимание:

- `*_DEFCONFIG` бывает двух видов: имя из дерева исходников (`odroid-go2_defconfig`, `defconfig`) или путь к файлу в SDK (`$(BOARD_DIR)/buildroot/defconfig`). Makefile различает их по наличию `/`.

- `$(IMAGES_DIR)` и `$(LINUX_INSTALL)` — каталоги с результатами сборки этой прошивки. Makefile определяет их **до** подключения конфига, поэтому ими можно пользоваться в `IMAGE_RAW` и `IMAGE_BOOT_FILES`.
- `ifeq` внутри конфига работает: это настоящий Makefile.

Полный справочник переменных — в приложении B.

Выбор прошивки

```
make oga_defconfig        # запоминает выбор в output/.fw
make                      # собирает выбранную
make FW=bbb linux         # разово собрать для другой, не меняя выбор
make info                 # что выбрано и какие версии компонентов
```

`make info` для OGA:

```
Firmware:  oga (configs/oga_defconfig)
Compiler:  aarch64-linux-gnu-gcc
TF-A:      src/tf-a [v2.15.0] PLAT=px30
U-Boot:    src/u-boot [v2026.07] odroid-go2_defconfig oga.config
BL31:      output/oga/tf-a/px30/release/bl31/bl31.elf
Linux:     src/linux [v7.2.9] ARCH=arm64 defconfig oga.config
Buildroot: src/buildroot [2026.08] board/oga/buildroot/defconfig
Image:     output/oga/images/oga.img
```

Версия в квадратных скобках — `git describe --dirty`. Если в сабмодуле есть незакоммиченные изменения, к ней добавится `-dirty`.

ГЛАВА 6

Верхний Makefile

Makefile SDK — это «дирижёр»: он ничего не компилирует сам, а вызывает make компонентов с правильными параметрами. Разберём его по частям. Полный текст — в приложении D.

Шапка и глобальные переменные

Makefile, строки 1–21

```
# OdroidLinux SDK: top-level build orchestrator.
#
# Every component (TF-A, U-Boot, Linux, Buildroot) lives in its own git
# repository under src/ and is built out-of-tree into output/<fw>/.
# A firmware config (configs/<fw>_defconfig) selects component configs and
# the SD card layout; "make image" glues the built components into an .img.
#
#   make oga_defconfig      select firmware config
#   make                    build everything and assemble the image
#   make help               list all targets

SDK_DIR    := $(patsubst %/,%,$(dir $(abspath $(lastword $(MAKEFILE_LIST)))))
OUTPUT_DIR ?= $(SDK_DIR)/output
DL_DIR     ?= $(SDK_DIR)/dl
CROSS_COMPILE ?= aarch64-linux-gnu-
JOBS       ?= $(shell nproc)

# Command-line variables of this Makefile must not leak into component builds
MAKEOVERRIDES :=

.DEFAULT_GOAL := all
```

- `SDK_DIR` вычисляется из пути к самому Makefile, поэтому `make -C ~/linux-sdk` работает из любого каталога.
- `?=` — «присвоить, если ещё не задано». Значение можно переопределить из командной строки (`make JOBS=4`) или из окружения.
- `.DEFAULT_GOAL := all` — что делает голый `make`.

Зачем `MAKEOVERRIDES :=`

Переменные, переданные в командной строке (`make FW=oga JOBS=8`), GNU make передаёт **всем вложенным make** через `MAKEFLAGS`. То есть `make -C src/linux ...` тоже получил бы `FW=oga` и `JOBS=8`. Пока имена не совпадают — не страшно, но если вы назовёте переменную так же, как она называется в Kbuild (`O`, `V`, `ARCH`…), сборка ядра сломается самым загадочным образом. `MAKEOVERRIDES :=` обнуляет список переменных командной строки для вложенных make. Всё нужное компонентам передаётся явно (`CROSS_COMPILE=...`, `O=...`).

Выбор прошивки

Makefile, строки 23–48

```
# ------------------------------------------------------------------------
# Firmware config selection

FW ?= $(shell cat $(OUTPUT_DIR)/.fw 2>/dev/null)
```

```
# Output locations, usable in firmware configs
OUT           := $(OUTPUT_DIR)/$(FW)
IMAGES_DIR    := $(OUT)/images
LINUX_INSTALL := $(OUT)/linux-install

ifeq ($(filter $(notdir $(wildcard $(SDK_DIR)/configs/*_defconfig)),$(MAKECMDGOALS)),)
ifneq ($(FW),)
FW_CONFIG := $(SDK_DIR)/configs/$(FW)_defconfig
ifeq ($(wildcard $(FW_CONFIG)),)
$(error Firmware config $(FW_CONFIG) not found)
endif
include $(FW_CONFIG)
endif
endif

FW_DEFCONFIGS := $(notdir $(wildcard $(SDK_DIR)/configs/*_defconfig))
.PHONY: $(FW_DEFCONFIGS)
$(FW_DEFCONFIGS): %_defconfig:
    @mkdir -p $(OUTPUT_DIR)
    @echo $* > $(OUTPUT_DIR)/.fw
    @echo "Firmware config: $* (output: $(OUTPUT_DIR)/$*)"
```

1. `FW` берётся из командной строки или из файла `output/.fw`.
2. `OUT`, `IMAGES_DIR`, `LINUX_INSTALL` определяются **до** `include`, чтобы конфиг прошивки мог ими пользоваться.
3. Конфиг подключается, только если в целях нет `*_defconfig` (иначе при первом `make oga_defconfig` подключать было бы нечего).
4. Цели `oga_defconfig`, `bbb_defconfig`… создаются автоматически по файлам в `configs/` через **static pattern rule** `$(FW_DEFCONFIGS): %_defconfig:`.

ЛОВУШКА: `UNEXPORT` СОЗДАЁТ ПУСТУЮ ПЕРЕМЕННУЮ

В первой версии стояло `unexport FW` перед `FW ?= ...`. В GNU make 4.4 `unexport` **определяет** переменную (пустой), и `?=` её уже не трогает: `FW` всегда была пустой, а `make info` говорил «No firmware config selected», хотя `output/.fw` содержал `oga`. Нашлось через `make -p info | grep '^FW'`, которое показало `# makefile (from 'Makefile', line 20)` — строку с `unexport`.

ЛОВУШКА: СЛИШКОМ ШИРОКИЙ ШАБЛОН

Первая версия правила была просто `%_defconfig:`. Но defconfig Buildroot — путь вида `.../configs/oga_defconfig` — тоже совпадает с шаблоном и стоит в зависимостях. Static pattern rule со списком из `configs/` ограничивает правило только конфигами прошивок.

Пути

Makefile, строки 50–70

```
# Absolute path for a path relative to the SDK root
sdkpath = $(if $(filter /%,$(1)),$(1),$(SDK_DIR)/$(1))
# In-tree defconfig names stay as they are, paths become absolute
cfgpath = $(if $(findstring /,$(1)),$(call sdkpath,$(1)),$(1))
# "path<sep>suffix" -> "/abs/path<sep>suffix" ($(1)=list $(2)=separator)
sdkpath-pairs = $(foreach e,$(1),$(call sdkpath,$(firstword $(subst $(2), ,$(e))))$(if $(find
string $(2),$(e)),$(2)$(lastword $(subst $(2), ,$(e)))))

LINUX_ARCH  ?= arm64
LINUX_IMAGE ?= Image
```

```
TFA_SRC   := $(call sdkpath,$(TFA_SRC))
UBOOT_SRC := $(call sdkpath,$(UBOOT_SRC))
LINUX_SRC := $(call sdkpath,$(LINUX_SRC))
BUILDROOT_SRC      := $(call sdkpath,$(BUILDROOT_SRC))
BUILDROOT_EXTERNAL := $(call sdkpath,$(BUILDROOT_EXTERNAL))

UBOOT_DEFCONFIG     := $(call cfgpath,$(UBOOT_DEFCONFIG))
LINUX_DEFCONFIG     := $(call cfgpath,$(LINUX_DEFCONFIG))
BUILDROOT_DEFCONFIG := $(call cfgpath,$(BUILDROOT_DEFCONFIG))
UBOOT_FRAGMENTS     := $(foreach f,$(UBOOT_FRAGMENTS),$(call sdkpath,$(f)))
LINUX_FRAGMENTS     := $(foreach f,$(LINUX_FRAGMENTS),$(call sdkpath,$(f)))
```

- `sdkpath` делает путь абсолютным относительно корня SDK, если он ещё не абсолютный.
- `cfgpath` различает «имя defconfig из дерева» и «путь к файлу» по наличию `/`.
- `sdkpath-pairs` нужна для списков вида `файл@сектор` и `источник:имя`: путь делается абсолютным, а суффикс сохраняется.

Все пути становятся абсолютными, потому что вложенные make запускаются с `-C` в другом каталоге, и относительный путь там значил бы другое.

Kconfig-помощники

Makefile, строки 72–98

```
# ------------------------------------------------------------------------
# Kconfig helpers (U-Boot and Linux)
#
# $(OBJ)/.config is regenerated when its inputs change: the defconfig file,
# a fragment, or the list of inputs itself (stored in .sdk-inputs). Changes
# made with *-menuconfig are kept until then; use *-savedefconfig to keep
# them for good.

# $(1)=objdir $(2)=inputs: rewrite the stamp only when the inputs differ
define inputs-stamp
$(shell mkdir -p $(1) && { test "$$(cat $(1)/.sdk-inputs 2>/dev/null)" = "$(strip $(2))" || e
cho "$(strip $(2))" > $(1)/.sdk-inputs; })
endef

# $(1)=make command $(2)=objdir $(3)=defconfig $(4)=fragments $(5)=src
define kconfig-configure
    @echo ">>> Configuring $(2)"
    $(if $(findstring /,$(3)),cp $(3) $(2)/.config && $(1) olddefconfig,$(1) $(3))
    $(if $(strip $(4)),$(5)/scripts/kconfig/merge_config.sh -m -O $(2) $(2)/.config $(4) && $
(1) olddefconfig && $(SDK_DIR)/scripts/kconfig-check.sh $(2)/.config $(4))
endef

# $(1)=make command $(2)=objdir $(3)=defconfig
define kconfig-savedefconfig
    $(1) savedefconfig
    @if [ -n "$(findstring /,$(3))" ]; then cp $(2)/defconfig $(3) && echo "Saved to
$(3)"; else \
        echo "Saved to $(2)/defconfig. The firmware config uses the in-tree '$(3)':"; \
        echo "move the changes to a fragment, or point the *_DEFCONFIG at a file."; fi
endef
```

Когда пересоздавать `.config`

`.config` компонента — это файл-цель make с зависимостями: defconfig-файл и фрагменты. Если любой из них новее `.config`, make пересоздаст его. Но есть тонкость: если вы **поменяли имя** defconfig в конфиге прошивки (например, `omap2plus_defconfig` → `multi_v7_defconfig`), файлов, которые бы обновились, нет.

Для этого `inputs-stamp` пишет список входов в файл `.sdk-inputs` и переписывает его **только если список изменился**. `.config` зависит от `.sdk-inputs`, поэтому смена списка тоже вызывает пересборку конфигурации. Вызов стоит прямо в теле Makefile (через `$(shell ...)`) и выполняется при каждом разборе.

> **КОМПРОМИСС**
> Изменения из `menuconfig` живут, пока не поменялся вход. Потом `.config` перегенерируется, и изменения пропадут. Это осознанное правило (так же устроен Buildroot): постоянные изменения должны жить во фрагменте или defconfig-файле. Для этого есть `*-savedefconfig`.

kconfig-check.sh

scripts/kconfig-check.sh

```
#!/bin/sh
# Warn about fragment options that did not end up in the final .config
# (usually an unmet dependency). Usage: kconfig-check.sh .config frag...
cfg=$1; shift
for frag; do
    grep -E '^(CONFIG_[A-Za-z0-9_]+=|# CONFIG_[A-Za-z0-9_]+ is not set)' "$frag" | while
read -r line; do
        case "$line" in
        "# "*)
            sym=${line#\# }; sym=${sym%% *}
            grep -q "^$sym=" "$cfg" && echo "WARNING: ${frag##*/}: $sym is still enabled"
            ;;
        *)
            grep -qxF "$line" "$cfg" && continue
            sym=${line%%=*}
            got=$(grep -E "^$sym=|^# $sym is not set" "$cfg" || echo "not available")
            echo "WARNING: ${frag##*/}: requested $line, got: $got"
            ;;
        esac
    done
done
exit 0
```

Скрипт проходит по строкам фрагмента. Для `CONFIG_X=...` проверяет, что точно такая же строка есть в `.config`; для `# CONFIG_X is not set` — что опция не включена. Если нет — печатает, что просили и что получилось. Это и поймало панель OGA и VC4 на Raspberry Pi 4.

Компоненты

TF-A

Makefile, строки 110–124

```
# ------------------------------------------------------------------------------
# Trusted Firmware-A (only for boards with TFA_PLAT)

TFA_OUT  := $(OUT)/tf-a
TFA_BL31 := $(TFA_OUT)/$(TFA_PLAT)/release/bl31/bl31.elf
TFA_MAKE  = $(MAKE) -C $(TFA_SRC) BUILD_BASE=$(TFA_OUT) PLAT=$(TFA_PLAT) \
            CROSS_COMPILE=$(CROSS_COMPILE) $(TFA_MAKE_ARGS)

.PHONY: tfa tfa-clean
tfa: check-cc
    @test -n "$(TFA_PLAT)" || { echo "$(FW) does not use TF-A (TFA_PLAT is not set)"; exit
1; }
    $(TFA_MAKE) -j$(JOBS) bl31
```

```
tfa-clean: check-fw
    rm -rf $(TFA_OUT)
```

TF-A — не Kconfig-проект, параметры передаются переменными: `PLAT` (платформа), `BUILD_BASE` (аналог `O=`), `CROSS_COMPILE`. Цель `bl31` собирает только BL31.

U-Boot

Makefile, строки 126–171

```
# ------------------------------------------------------------------------
# U-Boot

UBOOT_OUT := $(OUT)/u-boot
UBOOT_MAKE = $(MAKE) -C $(UBOOT_SRC) O=$(UBOOT_OUT) CROSS_COMPILE=$(CROSS_COMPILE) $(UBOOT_MA
KE_ARGS)

ifeq ($(UBOOT_BL31),tf-a)
UBOOT_BL31_FILE := $(TFA_BL31)
UBOOT_DEPS := tfa
else ifneq ($(UBOOT_BL31),)
UBOOT_BL31_FILE := $(call sdkpath,$(UBOOT_BL31))
endif

# Rockchip legacy uboot.img/trust.img for the SPI miniloader
ifeq ($(RK_LEGACY),y)
RKBIN_DIR     := $(call sdkpath,$(RKBIN_DIR))
RK_TRUST_BL31 := $(call sdkpath,$(RK_TRUST_BL31))
RK_LEGACY_IMAGES := $(IMAGES_DIR)/uboot.img $(IMAGES_DIR)/trust.img
endif

ifneq ($(FW),)
$(call inputs-stamp,$(UBOOT_OUT),$(UBOOT_DEFCONFIG) $(UBOOT_FRAGMENTS))
endif

$(UBOOT_OUT)/.config: $(UBOOT_OUT)/.sdk-inputs $(filter /%,$(UBOOT_DEFCONFIG)) $(UBOOT_FRAGME
NTS)
    $(call kconfig-configure,$(UBOOT_MAKE),$(UBOOT_OUT),$(UBOOT_DEFCONFIG),$
(UBOOT_FRAGMENTS),$(UBOOT_SRC))

.PHONY: uboot uboot-configure uboot-menuconfig uboot-savedefconfig uboot-clean
uboot: check-cc $(UBOOT_DEPS) $(UBOOT_OUT)/.config
    $(UBOOT_MAKE) -j$(JOBS) $(if $(UBOOT_BL31_FILE),BL31=$(UBOOT_BL31_FILE))
    @mkdir -p $(IMAGES_DIR)
    cp $(addprefix $(UBOOT_OUT)/,$(UBOOT_IMAGES)) $(IMAGES_DIR)/
ifeq ($(RK_LEGACY),y)
    $(SDK_DIR)/scripts/rk-legacy-pack.sh $(RKBIN_DIR) $(UBOOT_OUT) $(RK_TRUST_BL31) $(IMAGES_
DIR)
endif

uboot-configure: check-cc $(UBOOT_OUT)/.config

uboot-menuconfig: check-cc $(UBOOT_OUT)/.config
    $(UBOOT_MAKE) menuconfig

uboot-savedefconfig: check-cc $(UBOOT_OUT)/.config
    $(call kconfig-savedefconfig,$(UBOOT_MAKE),$(UBOOT_OUT),$(UBOOT_DEFCONFIG))

uboot-clean: check-fw
    rm -rf $(UBOOT_OUT) $(addprefix $(IMAGES_DIR)/,$(notdir $(UBOOT_IMAGES))) $(RK_LEGACY_IMA
GES)
```

- `UBOOT_BL31` может быть `tf-a` (собрать из исходников и взять результат), путём к готовому ELF или пустым (платам без TF-A).
- `UBOOT_IMAGES` — какие файлы забрать из каталога сборки в `images/`: у OGA это `u-boot-rockchip.bin`, у BBB — `MLO` и `u-boot.img`, у RPi — `u-boot.bin`.

- `RK_LEGACY` включает дополнительную упаковку для заводского загрузчика OGA (глава 12).

Блобы прошивки

Makefile, строки 173–188

```
# ------------------------------------------------------------------------------
# Boot firmware blobs (e.g. Raspberry Pi GPU firmware), fetched by URL and
# checked against BOOTFW_HASH, then copied to images/

ifneq ($(strip $(BOOTFW_FILES)),)
BOOTFW_DEPS := bootfw
endif

.PHONY: bootfw bootfw-clean
bootfw: check-fw
    @test -n "$(strip $(BOOTFW_FILES))" || { echo "$(FW) has no boot firmware (BOOTFW_FILES
is not set)"; exit 1; }
    $(SDK_DIR)/scripts/fetch-files.sh $(BOOTFW_URL) $(DL_DIR)/$(BOOTFW_NAME) \
        $(call sdkpath,$(BOOTFW_HASH)) $(IMAGES_DIR) $(BOOTFW_FILES)

bootfw-clean: check-fw
    rm -f $(addprefix $(IMAGES_DIR)/,$(BOOTFW_FILES))
```

Для Raspberry Pi нужны закрытые файлы прошивки GPU. Цель `bootfw` скачивает их по URL в кэш `dl/<BOOTFW_NAME>/`, сверяет sha256 и копирует в `images/`.

Linux

Makefile, строки 190–221

```
# ------------------------------------------------------------------------------
# Linux

LINUX_OUT := $(OUT)/linux
LINUX_MAKE = $(MAKE) -C $(LINUX_SRC) O=$(LINUX_OUT) ARCH=$(LINUX_ARCH) CROSS_COMPILE=$(CROSS_
COMPILE) $(LINUX_MAKE_ARGS)

ifneq ($(FW),)
$(call inputs-stamp,$(LINUX_OUT),$(LINUX_DEFCONFIG) $(LINUX_FRAGMENTS))
endif

$(LINUX_OUT)/.config: $(LINUX_OUT)/.sdk-inputs $(filter /%,$(LINUX_DEFCONFIG)) $(LINUX_FRAGME
NTS)
    $(call kconfig-configure,$(LINUX_MAKE),$(LINUX_OUT),$(LINUX_DEFCONFIG),$
(LINUX_FRAGMENTS),$(LINUX_SRC))

.PHONY: linux linux-configure linux-menuconfig linux-savedefconfig linux-clean
linux: check-cc $(LINUX_OUT)/.config
    $(LINUX_MAKE) -j$(JOBS) $(LINUX_IMAGE) modules $(LINUX_DTBS)
    rm -rf $(LINUX_INSTALL)
    mkdir -p $(LINUX_INSTALL)/modules
    cp $(LINUX_OUT)/arch/$(LINUX_ARCH)/boot/$(LINUX_IMAGE) $(LINUX_OUT)/System.map $(LINUX_OU
T)/.config $(LINUX_INSTALL)/
    $(foreach d,$(LINUX_DTBS),install -D -m 644 $(LINUX_OUT)/arch/$(LINUX_ARCH)/boot/dts/$
(d) $(LINUX_INSTALL)/dtbs/$(d) &&) true
    $(LINUX_MAKE) INSTALL_MOD_PATH=$(LINUX_INSTALL)/modules INSTALL_MOD_STRIP=1 modules_insta
ll

linux-configure: check-cc $(LINUX_OUT)/.config

linux-menuconfig: check-cc $(LINUX_OUT)/.config
    $(LINUX_MAKE) menuconfig

linux-savedefconfig: check-cc $(LINUX_OUT)/.config
    $(call kconfig-savedefconfig,$(LINUX_MAKE),$(LINUX_OUT),$(LINUX_DEFCONFIG))
```

```
linux-clean: check-fw
    rm -rf $(LINUX_OUT) $(LINUX_INSTALL)
```

- `ARCH` и имя образа (`Image` / `zImage`) берутся из конфига прошивки.
- Собираются только нужные `.dtb` из `LINUX_DTBS`, а не все тысячи.
- `modules_install` с `INSTALL_MOD_PATH` кладёт модули в `linux-install/modules/lib/modules/<версия>/` и запускает `depmod`. `INSTALL_MOD_STRIP=1` вырезает отладочную информацию — модули становятся в разы меньше.

Buildroot

Makefile, строки 223–263

```
# ------------------------------------------------------------------------------
# Root filesystem (Buildroot)

ROOTFS_OUT := $(OUT)/buildroot
ROOTFS_MAKE = BR2_DL_DIR=$(DL_DIR) $(MAKE) -C $(BUILDROOT_SRC) O=$(ROOTFS_OUT) \
              BR2_EXTERNAL=$(BUILDROOT_EXTERNAL) $(BUILDROOT_MAKE_ARGS)

ifneq ($(FW),)
$(call inputs-stamp,$(ROOTFS_OUT),$(BUILDROOT_DEFCONFIG) $(BUILDROOT_EXTERNAL))
endif

$(ROOTFS_OUT)/.config: $(ROOTFS_OUT)/.sdk-inputs $(filter /%,$(BUILDROOT_DEFCONFIG))
    @echo ">>> Configuring $(ROOTFS_OUT)"
    $(if $(findstring /,$(BUILDROOT_DEFCONFIG)),$(ROOTFS_MAKE) defconfig BR2_DEFCONFIG=$(BUIL
DROOT_DEFCONFIG),$(ROOTFS_MAKE) $(BUILDROOT_DEFCONFIG))

.PHONY: rootfs rootfs-configure rootfs-menuconfig rootfs-savedefconfig rootfs-sdk
rootfs-make rootfs-clean
rootfs: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE)
    @mkdir -p $(IMAGES_DIR)
    cp $(ROOTFS_OUT)/images/rootfs.tar $(IMAGES_DIR)/

rootfs-configure: check-fw $(ROOTFS_OUT)/.config

rootfs-menuconfig: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) menuconfig

rootfs-savedefconfig: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) savedefconfig

# Relocatable cross toolchain + sysroot for application development
rootfs-sdk: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) sdk
    @mkdir -p $(IMAGES_DIR)
    cp $(ROOTFS_OUT)/images/*_sdk-buildroot.tar.gz $(IMAGES_DIR)/

# Run any Buildroot target: make rootfs-make BR=busybox-menuconfig
rootfs-make: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) $(BR)

rootfs-clean: check-fw
    rm -rf $(ROOTFS_OUT) $(IMAGES_DIR)/rootfs.tar
```

- Buildroot не любит, когда его вызывают с чужими переменными, поэтому `BR2_DL_DIR` передаётся через окружение (префикс перед `$(MAKE)`).
- `make defconfig BR2_DEFCONFIG=<путь>` разворачивает defconfig из произвольного места и запоминает путь — потом `savedefconfig` запишет изменения **обратно в этот файл**.
- `rootfs-make BR=<цель>` даёт доступ к любой цели Buildroot, например `busybox-menuconfig`, `linux-headers-rebuild`, `<пакет>-dirclean`.

Образ и прошивка карты

Makefile, строки 265–300

```
# ------------------------------------------------------------------------
# SD card image

IMAGE_FILE := $(IMAGES_DIR)/$(IMAGE_NAME).img
IMAGE_DTB_LAYOUT ?= tree

.PHONY: image all flash flash-boot clean info help
image: check-fw
    KERNEL=$(LINUX_INSTALL)/$(LINUX_IMAGE) \
    LINUX_DIR=$(LINUX_INSTALL) \
    DTB_LAYOUT=$(IMAGE_DTB_LAYOUT) \
    RAW="$(call sdkpath-pairs,$(IMAGE_RAW),@)" \
    BOOT_FILES="$(call sdkpath-pairs,$(IMAGE_BOOT_FILES),:)" \
    ROOTFS_TAR=$(IMAGES_DIR)/rootfs.tar \
    CMDLINE="$(LINUX_CMDLINE)" \
    BOOT_SIZE_MB=$(IMAGE_BOOT_SIZE_MB) \
    ROOTFS_FREE_MB=$(IMAGE_ROOTFS_FREE_MB) \
    DISK_ID=$(IMAGE_DISK_ID) \
    WORK_DIR=$(OUT)/image-work \
    HOST_TOOLS=$(ROOTFS_OUT)/host/bin \
    $(SDK_DIR)/scripts/mkimage.sh $(IMAGE_FILE)

all: uboot linux rootfs $(BOOTFW_DEPS)
    $(MAKE) FW=$(FW) image

# make flash DEV=/dev/sdX
flash: check-fw
    $(SDK_DIR)/scripts/flash.sh $(IMAGE_FILE) $(DEV) 0

# Rewrite the bootloader and the BOOT partition (kernel, dtbs, extlinux),
# keeping the MBR and the rootfs: make flash-boot DEV=/dev/sdX.
# The card must hold an image of the same firmware config.
flash-boot: check-fw
    @test -f $(IMAGES_DIR)/boot-area.sector || { echo "Run 'make image' first"; exit 1; }
    $(SDK_DIR)/scripts/flash.sh $(IMAGES_DIR)/boot-area.bin $(DEV) $$(cat $(IMAGES_DIR)/boot-
area.sector)
```

Makefile только передаёт переменные в `mkimage.sh` через окружение (глава 10). `all` сначала собирает компоненты, потом вызывает себя же для `image` — так `image` гарантированно запускается после всех компонентов, даже при `make -j`.

> **УПРАЖНЕНИЕ**
>
> Добавьте цель `image-xz`, которая после `image` сжимает образ (`xz -T0 -k images/<fw>.img`). Подумайте, от чего она должна зависеть и как не пересжимать образ, если он не менялся.

ГЛАВА 7

TF-A и U-Boot

TF-A: BL31 для PX30

RK3326 в TF-A — это платформа `px30` (RK3326 — вариант PX30). Проверить, что платформа поддерживается, можно просто по наличию каталога:

```
ls src/tf-a/plat/rockchip/px30
```

Сборка одной командой (её и выполняет `make tfa`):

```
make -C src/tf-a BUILD_BASE=output/oga/tf-a PLAT=px30 \
     CROSS_COMPILE=aarch64-linux-gnu- bl31
# результат: output/oga/tf-a/px30/release/bl31/bl31.elf
```

U-Boot нужен именно **ELF**, а не `.bin`: binman разбирает ELF на сегменты и кладёт каждый в FIT со своим адресом загрузки (те самые `atf-1`, `atf-2`, `atf-3` из главы 1).

Скорость консоли TF-A

В `plat/rockchip/px30/px30_def.h` написано `#define PX30_BAUDRATE 1500000` — стандарт Rockchip. Но в `plat/rockchip/common/params_setup.c` TF-A читает `stdout-path` из device tree, который ей передаёт SPL, и берёт скорость оттуда. В DTS OGA стоит `serial2:115200n8`, поэтому TF-A печатает на 115200.

> **КАК ВЫЯСНЯТЬ ТАКИЕ ВЕЩИ**
> `grep -rn -i 'baud' src/tf-a/plat/rockchip/px30/ src/tf-a/plat/rockchip/common/` — две минуты чтения кода вместо часов гаданий на осциллографе.

Запасной вариант: BL31 от Rockchip

В rkbin есть бинарный `rk3326_bl31_v1.34.elf`. Он лежит в `board/oga/rkbin/bin/` и подключается так: `UBOOT_BL31 := $(BOARD_DIR)/rkbin/bin/rk3326_bl31_v1.34.elf`. Mainline TF-A заработала сразу, но запасной вариант полезно иметь — и он понадобился в другом месте (глава 12).

U-Boot для ODROID-GO Advance

В mainline U-Boot уже есть плата: `configs/odroid-go2_defconfig` и `board/hardkernel/odroid_go2/go2.c`, а в документации она значится как «ODROID-GO Advance (odroid-go2)»: `doc/board/rockchip/rockchip.rst`. Кто её сопровождает, написано в `board/hardkernel/odroid_go2/MAINTAINERS` — туда же отправлять патчи.

Что умеет go2.c

Главное — **определение ревизии**: на канал 0 SARADC повешен резистор, разный для разных ревизий. U-Boot читает АЦП, находит ревизию в таблице `oga_model_details[]` и ставит переменные окружения (упрощённо):

```
env_set("board", ...);
env_set("board_name", ...);
env_set("fdtfile", "rockchip/rk3326-odroid-go2-v11.dtb");   /* для BE */
```

`fdtfile` потом использует extlinux: в `extlinux.conf` написано `fdtdir /`, и U-Boot загружает `/` + `fdtfile`. Благодаря этому **один образ подходит для всех трёх ревизий**, и никаких `if test ${hwrev} = 'v11'` из старого `boot.ini` не нужно.

Важные опции defconfig

```
CONFIG_DEFAULT_DEVICE_TREE="rockchip/rk3326-odroid-go2"
CONFIG_OF_LIST="rockchip/rk3326-odroid-go2 rockchip/rk3326-odroid-go2-v11 rockchip/rk3326-
odroid-go3"
CONFIG_SPL_ATF=y             SPL умеет запускать BL31
CONFIG_TPL_RAM=y             TPL сам инициализирует DDR (без блоба Rockchip)
CONFIG_ENV_OFFSET=0x4000     окружение по смещению 16 КиБ
CONFIG_ROCKCHIP_SPI_IMAGE=y  дополнительно собрать образ для SPI-флеша
```

`CONFIG_TPL_RAM=y` — важная деталь: инициализация DDR в mainline TPL **открытая** (`drivers/ram/rockchip/sdram_px30.c`), без бинарного DDR-блоба Rockchip.

Фрагмент SDK

board/oga/u-boot/oga.config

```
# ODROID-GO Advance: options on top of odroid-go2_defconfig.

# Same serial speed as TF-A (taken from stdout-path) and the kernel
CONFIG_BAUDRATE=115200
CONFIG_BOOTDELAY=2

# u-boot.itb at sector 2048 (1 MiB) instead of 16384: sectors 16384 and
# 24576 hold uboot.img/trust.img for the Rockchip miniloader (SPI flash)
CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR=0x800
CONFIG_SPL_PAD_TO=0xf8000
```

Первые две опции появились из-за консоли, вторые две — из-за заводского загрузчика в SPI (глава 12).

История про 1500000 бод

По умолчанию U-Boot для Rockchip ставит `CONFIG_BAUDRATE=1500000`. В итоге:

Стадия	Скорость без фрагмента
TPL, SPL, U-Boot	1 500 000 (CONFIG_BAUDRATE)
TF-A	115 200 (из stdout-path)
Ядро	115 200 (console=ttyS2,115200)

Лог «разъезжался» бы по скоростям. Фрагмент выставляет 115200 везде. Это ещё и практично: у многих USB-UART адаптеров (например, CP2102) максимум — 921600 бод, 1,5 Мбод они физически не умеют (глава 11).

ОТЛАДОЧНЫЙ UART В TPL/SPL

TPL и SPL печатают через «debug UART» — простейший драйвер без device tree. Его параметры — `CONFIG_DEBUG_UART_BASE=0xFF160000` (UART2), `CONFIG_DEBUG_UART_CLOCK=24000000`, а скорость — тот же `CONFIG_BAUDRATE`. Функция `board_debug_uart_init()` в `arch/arm/mach-rockchip/px30/px30.c` переключает пины на UART и выбирает источник тактирования 24 МГц.

Сборка

```
make -C src/u-boot O=output/oga/u-boot CROSS_COMPILE=aarch64-linux-gnu- odroid-go2_defconfig
# + merge_config.sh с фрагментом, olddefconfig
make -C src/u-boot O=output/oga/u-boot CROSS_COMPILE=aarch64-linux-gnu- \
     BL31=output/oga/tf-a/px30/release/bl31/bl31.elf -j16
```

В конце сборки работает **binman** — инструмент U-Boot, который по описанию в `arch/arm/dts/rockchip-u-boot.dtsi` собирает итоговые образы:

Файл	Что это
`idbloader.img`	TPL + SPL с заголовком Rockchip
`u-boot.itb`	FIT: U-Boot + BL31 + dtb
`u-boot-rockchip.bin`	idbloader + FIT по смещению `CONFIG_SPL_PAD_TO` — **пишется с сектора 64**
`u-boot-rockchip-spi.bin`	то же для SPI-флеша

Сборка напечатала предупреждение:

```
Image 'simple-bin' is missing optional external blobs but is still functional: tee-os
```

`tee-os` — это OP-TEE (BL32), необязательный компонент. «Still functional» — образ рабочий.

ОПАСНО: ОБРАЗ БЕЗ BL31 ВСЁ РАВНО ПОЯВИТСЯ

U-Boot всегда вызывает binman с `--allow-missing --fake-ext-blobs`. Если BL31 не передан, binman подставит **фальшивый** блоб, запишет `u-boot-rockchip.bin` и только потом сообщит об ошибке. Make завершится неуспешно, но файл на диске останется — и выглядит он как настоящий. А с `BINMAN_ALLOW_MISSING=1` (добавляет `--ignore-missing`) не будет даже ошибки. Если вы запускали сборку руками и пропустили сообщение, можно получить «загрузчик», который молча не грузится. Проверяйте `dumpimage -l u-boot.itb` или код возврата make.

Проверка результата

```
dumpimage -l output/oga/u-boot/u-boot.itb | grep -E 'Image [0-9]|Description|Load'
grep -E '^CONFIG_(BAUDRATE|SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR|SPL_PAD_TO)=' output/oga/u-boot/.config
```

Проверяем три вещи: в FIT есть U-Boot, сегменты BL31 и все три dtb; скорость — 115200; сектор FIT совпадает с тем, куда он реально попадёт в образе.

Адреса загрузки ядра

Ещё одна проверка, которую стоит делать для каждой новой платы: **влезет ли ядро в память между адресами, которые использует U-Boot**. Переменные адресов для PX30 — в `include/configs/px30_common.h`:

```
fdt_addr_r=0x01e00000        dtb
kernel_addr_r=0x02080000     ядро
ramdisk_addr_r=0x06000000    initrd
```

Между `kernel_addr_r` и `ramdisk_addr_r` — `0x06000000 - 0x02080000 = 0x3F80000`, то есть 63,5 МиБ. `Image` из arm64 `defconfig` весит ≈ 40,5 МиБ — влезает. Если бы ядро было больше, оно перекрыло бы область, куда загрузится что-то ещё, и упало бы непредсказуемо.

> **УПРАЖНЕНИЕ**
>
> Найдите в `include/configs/` адреса для am335x и для Raspberry Pi 4 и проверьте, сколько места там под ядро. Сравните с размером `zImage` BBB (≈ 5,6 МиБ) и `Image` RPi 4 (≈ 41 МиБ).

ГЛАВА 8

Ядро Linux

Какой defconfig взять за основу

Есть три пути:

1. **Общий defconfig архитектуры** (`arch/arm64/configs/defconfig`). Включает поддержку сотен плат, почти всё — модулями. Большой (`Image` ≈ 40 МиБ, модули ≈ 88 МиБ), но гарантированно содержит всё нужное для SoC.
2. **Defconfig семейства SoC** (`omap2plus_defconfig`, `bcm2835_defconfig`). Меньше и точнее, есть только для части платформ.
3. **Свой минимальный defconfig.** Самый компактный, но его нужно составить и поддерживать — и легко забыть что-нибудь важное.

Для OGA и Raspberry Pi 4 взят общий arm64 `defconfig`, для BBB — `omap2plus_defconfig`, для Raspberry Pi 1 — `bcm2835_defconfig`. Поверх каждого — короткий фрагмент. Ужать ядро можно позже, когда всё заработает (упражнение в конце главы).

Что добавить для OGA

Чтобы понять, чего не хватает, сравниваем то, что использует DTS платы, с тем, что включено в defconfig. Список `compatible` из DTS:

```
grep -h -o 'compatible = "[a-z,0-9-]*"' \
    arch/arm64/boot/dts/rockchip/rk3326-odroid-go*.dts* | sort -u
```

Среди них `elida,kd35t133` (панель rev 1.0/1.1), `elida,kd50t048a` (панель Super), `adc-joystick` (аналоговый стик), `gpio-keys` (кнопки), `rockchip,rk817` (PMIC, звук, зарядка). Для каждого ищем драйвер и его опцию:

```
grep -rn 'elida,kd35t133' drivers/       # → panel-elida-kd35t133.c
grep -n 'kd35t133' drivers/gpu/drm/panel/Makefile   # → CONFIG_DRM_PANEL_ELIDA_KD35T133
grep -E 'ELIDA|JOYSTICK|KEYBOARD_GPIO' arch/arm64/configs/defconfig
```

Выяснилось, что в arm64 `defconfig` нет драйверов обеих панелей и ADC-джойстика, а DRM Rockchip включён модулем. Отсюда фрагмент:

board/oga/linux/oga.config

```
# ODROID-GO Advance / Super: options on top of arm64 defconfig.
# Display stack is built in so the boot console shows up on the panel
# without waiting for modules.

CONFIG_LOCALVERSION="-oga"
# CONFIG_LOCALVERSION_AUTO is not set

# Display: VOP + MIPI-DSI + panels
CONFIG_DRM=y
CONFIG_DRM_ROCKCHIP=y
CONFIG_ROCKCHIP_VOP=y
CONFIG_ROCKCHIP_DW_MIPI_DSI=y
CONFIG_PHY_ROCKCHIP_INNO_DSIDPHY=y
CONFIG_DRM_PANEL_ELIDA_KD35T133=y
```

```
CONFIG_DRM_PANEL_SITRONIX_ST7701=y
CONFIG_BACKLIGHT_CLASS_DEVICE=y
CONFIG_BACKLIGHT_PWM=y
CONFIG_FRAMEBUFFER_CONSOLE=y
# Without it fbcon=rotate: on the command line is silently ignored
CONFIG_FRAMEBUFFER_CONSOLE_ROTATION=y

# Gamepad: gpio-keys + analog stick on SARADC
CONFIG_INPUT_JOYSTICK=y
CONFIG_JOYSTICK_ADC=y
CONFIG_ROCKCHIP_SARADC=y
CONFIG_KEYBOARD_GPIO=y

# PMIC RK817: battery/charger, RTC, audio codec
CONFIG_CHARGER_RK817=m
CONFIG_RTC_DRV_RK808=y
CONFIG_SND_SOC_RK817=m
CONFIG_SND_SOC_ROCKCHIP_I2S_TDM=m
CONFIG_SND_SIMPLE_CARD=m
```

Почему дисплей встроен, а не модулем

Если драйверы DRM — модули, они загрузятся только когда init дойдёт до `mdev` и `modprobe`. До этого экран чёрный, и если что-то пошло не так раньше, вы ничего не увидите. Встроенный дисплей показывает лог ядра с первых секунд. Для портативной консоли без UART это ценнее, чем пара мегабайт размера ядра.

Случай с панелью

После первой сборки `kconfig-check.sh` напечатал:

```
WARNING: oga.config: requested CONFIG_DRM_PANEL_ELIDA_KD35T133=y, got: ...=m
WARNING: oga.config: requested CONFIG_DRM_PANEL_SITRONIX_ST7701=y, got: ...=m
WARNING: oga.config: requested CONFIG_BACKLIGHT_PWM=y, got: CONFIG_BACKLIGHT_PWM=m
```

Смотрим зависимости:

```
config DRM_PANEL_ELIDA_KD35T133
    tristate "Elida KD35T133 panel driver"
    depends on OF
    depends on DRM_MIPI_DSI
    depends on BACKLIGHT_CLASS_DEVICE
```

`BACKLIGHT_CLASS_DEVICE=m` в defconfig ограничивает все три опции до `m`. Добавили во фрагмент `CONFIG_BACKLIGHT_CLASS_DEVICE=y` — и всё встало как надо. Без `kconfig-check.sh` это всплыло бы только на железе: «почему экран загорается так поздно?».

Сборка

```
make -C src/linux O=output/oga/linux ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j16 \
     Image modules rockchip/rk3326-odroid-go2.dtb rockchip/rk3326-odroid-go2-v11.dtb \
     rockchip/rk3326-odroid-go3.dtb
make ... INSTALL_MOD_PATH=output/oga/linux-install/modules INSTALL_MOD_STRIP=1 modules_instal
l
```

Результат в `output/oga/linux-install/`:

```
Image                        ядро
System.map, .config          для отладки и справки
```

```
dtbs/rockchip/*.dtb           три dtb
modules/lib/modules/7.2.9-oga/    модули + modules.dep (depmod)
```

`CONFIG_LOCALVERSION="-oga"` делает версию `7.2.9-oga`, а `# CONFIG_LOCALVERSION_AUTO is not set` убирает автоматический хвост `+` или `-gXXXXXXX` от git. Версия важна: модули лежат в `/lib/modules/$(uname -r)`, и если версия собранного ядра не совпадает с каталогом модулей, `modprobe` их не найдёт.

ЛОВУШКА: ПЕРЕСБОРКА ЯДРА БЕЗ ПЕРЕСБОРКИ ОБРАЗА

Модули попадают в rootfs только в `make image`. Если пересобрали ядро с другими опциями, но записали на карту только раздел BOOT, модули в rootfs останутся старыми. Самый надёжный путь — `make linux image` и полная перезапись карты.

Командная строка ядра

```
rootwait rw console=ttyS2,115200n8 console=tty1 fbcon=rotate:3 consoleblank=0
```

Параметр	Зачем
`root=PARTUUID=...-02`	добавляет `mkimage.sh`: rootfs — второй раздел этого диска
`rootwait`	ждать, пока появится устройство (SD инициализируется не мгновенно)
`rw`	монтировать сразу на запись
`console=ttyS2,...`	лог и консоль на UART
`console=tty1`	и на экран
`fbcon=rotate:3`	панель OGA физически повёрнута — поворачиваем текстовую консоль на 90° против часовой
`consoleblank=0`	не гасить экран по таймауту

ЛОВУШКА: `FBCON=ROTATE` БЕЗ ПОДДЕРЖКИ В ЯДРЕ

Параметр `fbcon=rotate:N` работает, только если в ядре включён `CONFIG_FRAMEBUFFER_CONSOLE_ROTATION`. В arm64 `defconfig` он выключен, и ядро **молча игнорирует** параметр. В первой версии SDK так и было: в командной строке стояло `fbcon=rotate:3`, а консоль оставалась в родной портретной ориентации панели. Теперь опция включена во фрагменте OGA.

Значения: 0 — без поворота, 1 — 90° по часовой, 2 — 180°, 3 — 90° против часовой. Без параметра fbcon берёт подсказку от DRM: у панели OGA в DTS `rotation = <270>`, DRM превращает это в `FB_ROTATE_CW`, то есть `rotate:1`. Но на консоли с ней картинка **вверх ногами**: правильная ориентация — `rotate:3`, проверено на Black Edition. Поэтому параметр задан явно. Урок: подсказка из DTS — это гипотеза, окончательный ответ даёт только железо. Текущий поворот виден в `/sys/class/graphics/fbcon/rotate`, и его можно менять на лету: `echo 3 > /sys/class/graphics/fbcon/rotate`.

Сообщения ядра идут на все устройства из `console=`, а `/dev/console` — то, куда пишет и откуда читает init, — это **последний** `console=`. Поэтому порядок имеет значение: последним ставим UART, чтобы сообщения init и аварийная консоль были там, куда подключён кабель.

Устройства ввода

После загрузки кнопки и стик «не реагировали в консоли». При этом лог ядра говорил, что всё на месте:

```
input: adc-joystick as /devices/platform/adc-joystick/input/input0
input: gpio-keys as /devices/platform/gpio-keys/input/input1
```

Дело в том, что это **геймпад**: драйвер `gpio-keys` по DTS генерирует коды `BTN_SOUTH`, `BTN_DPAD_UP` и т. п., а не клавиши клавиатуры. Текстовая консоль понимает только клавиатуру. Проверка — утилитой `evtest`, которая есть в rootfs:

```
evtest /dev/input/event1     # нажимайте кнопки — события печатаются
```

Игры и эмуляторы (SDL и другие) читают `/dev/input/event*` и видят геймпад как есть.

> **УПРАЖНЕНИЕ: УМЕНЬШИТЬ ЯДРО**
> На работающей OGA выполните `lsmod > /boot/lsmod.txt` (после того как погоняли всё железо: звук, зарядку, USB). На ПК: `make -C src/linux O=output/oga/linux ARCH=arm64 LSMOD=lsmod.txt localmodconfig` отключит все модули, которые не загружены. Затем `make linux-savedefconfig`, сохраните результат как `board/oga/linux/oga_defconfig` и укажите `LINUX_DEFCONFIG := $(BOARD_DIR)/linux/oga_defconfig`. Сравните размер `Image` и модулей до и после.

ГЛАВА 9

Корневая файловая система: Buildroot

Почему Buildroot

Buildroot собирает из исходников кросс-тулчейн, библиотеку C, busybox и выбранные пакеты и выдаёт готовую rootfs. Он прост (Makefile + Kconfig), воспроизводим и хорошо подходит для встраиваемых систем. Альтернатива — Yocto: мощнее, но гораздо тяжелее в освоении.

Buildroot умеет собирать и ядро, и U-Boot сам. Но по требованию SDK компоненты независимы, поэтому в нашем defconfig Buildroot **не собирает** ни ядро, ни загрузчик — только rootfs. На выходе — `rootfs.tar`.

BR2_EXTERNAL: свои файлы вне дерева Buildroot

Менять файлы внутри `src/buildroot` не нужно. Для своих defconfig, пакетов и overlay у Buildroot есть механизм **BR2_EXTERNAL** — внешнее дерево с минимальной структурой:

```
board/common/buildroot/
├── external.desc        имя дерева: "SDK" → переменная $(BR2_EXTERNAL_SDK_PATH)
├── external.mk          подключает package/*/*.mk
├── Config.in            подключает package/*/Config.in (меню пакетов)
├── package/             ваши пакеты
├── rootfs-overlay/      файлы, которые копируются в rootfs как есть
├── post-build.sh        скрипт, который правит rootfs перед упаковкой
└── busybox.fragment     фрагмент конфига busybox
```

board/common/buildroot/external.desc

```
name: SDK
desc: OdroidLinux SDK: packages, rootfs overlay and scripts shared by all boards
```

Сначала у OGA было своё внешнее дерево `board/oga/buildroot`. При добавлении других плат оно стало **общим** (`board/common/buildroot`, имя `SDK`), а у каждой платы остался только defconfig `board/<плата>/buildroot/defconfig`. Так свои пакеты пишутся один раз и доступны всем платам.

Defconfig платы

board/oga/buildroot/defconfig

```
BR2_aarch64=y
BR2_cortex_a35=y
BR2_TOOLCHAIN_BUILDROOT_GLIBC=y
BR2_TOOLCHAIN_BUILDROOT_CXX=y
BR2_PACKAGE_HOST_GDB=y
BR2_TARGET_GENERIC_HOSTNAME="odroid-go"
BR2_TARGET_GENERIC_ISSUE="Welcome to ODROID-GO Advance"
BR2_ROOTFS_DEVICE_CREATION_DYNAMIC_MDEV=y
BR2_TARGET_GENERIC_GETTY_PORT="ttyS2"
BR2_TARGET_GENERIC_GETTY_BAUDRATE_115200=y
BR2_SYSTEM_DHCP=""
BR2_ROOTFS_OVERLAY="$(BR2_EXTERNAL_SDK_PATH)/rootfs-overlay"
BR2_ROOTFS_POST_BUILD_SCRIPT="$(BR2_EXTERNAL_SDK_PATH)/post-build.sh"
BR2_ROOTFS_POST_SCRIPT_ARGS="tty1"
BR2_PACKAGE_BUSYBOX_CONFIG_FRAGMENT_FILES="$(BR2_EXTERNAL_SDK_PATH)/busybox.fragment"
```

```
BR2_PACKAGE_ALSA_UTILS=y
BR2_PACKAGE_EVTEST=y
BR2_PACKAGE_GDB=y
BR2_PACKAGE_KMOD=y
BR2_PACKAGE_KMOD_TOOLS=y
BR2_TARGET_ROOTFS_TAR=y
BR2_PACKAGE_HOST_MTOOLS=y
```

Строки	Смысл
`BR2_aarch64`, `BR2_cortex_a35`	архитектура и точное ядро CPU
`BR2_TOOLCHAIN_BUILDROOT_GLIBC`, `_CXX`	собрать свой тулчейн с glibc и C++
`BR2_PACKAGE_HOST_GDB`, `BR2_PACKAGE_GDB`	gdb на ПК и gdbserver на плате
`BR2_ROOTFS_DEVICE_CREATION_DYNAMIC_MDEV`	`/dev` через devtmpfs + mdev, модули грузятся по modalias
`BR2_TARGET_GENERIC_GETTY_PORT`	основная консоль входа (UART)
`BR2_ROOTFS_OVERLAY`, `_POST_BUILD_SCRIPT`, `_POST_SCRIPT_ARGS`	наши правки rootfs
`BR2_PACKAGE_BUSYBOX_CONFIG_FRAGMENT_FILES`	фрагмент конфига busybox
`BR2_TARGET_ROOTFS_TAR`	на выходе `rootfs.tar` (ext4 собирает `mkimage.sh`)
`BR2_PACKAGE_HOST_MTOOLS`	запасной `mcopy`, если его нет в системе

mdev и модули

С `DYNAMIC_MDEV` скрипт `/etc/init.d/S10mdev` при старте проходит по `/sys` и для каждого устройства вызывает `modprobe` по его `modalias`. Поэтому модули ядра (звук RK817, Ethernet, USB) подгружаются сами, без списков в `/etc/modules`. Пакет `kmod` даёт полноценные `modprobe`/`lsmod`, хотя busybox справился бы и сам.

post-build.sh

board/common/buildroot/post-build.sh

```
#!/bin/sh
# Buildroot post-build hook shared by all boards.
#   $1     target dir
#   $2...  extra login consoles, "tty[:baud]" (BR2_ROOTFS_POST_SCRIPT_ARGS),
#          e.g. "tty1" for the display next to the serial getty
# The /boot fstab entry is added by scripts/mkimage.sh, which knows the layout.
set -e
TARGET_DIR="$1"
shift

for spec; do
    tty=${spec%%:*}
    baud=${spec#*:}; [ "$baud" != "$spec" ] || baud=0
    grep -q "^$tty::" "$TARGET_DIR/etc/inittab" || \
        sed -i "/# GENERIC_SERIAL\$/a $tty::respawn:/sbin/getty -L $tty $baud linux" "$TARGET
_DIR/etc/inittab"
done
```

Buildroot вызывает скрипт после сборки всех пакетов, передавая путь к будущей rootfs и аргументы из `BR2_ROOTFS_POST_SCRIPT_ARGS`. Здесь он добавляет `getty` на экран (`tty1`) рядом с основным на UART: Buildroot из коробки умеет только один getty.

Обратите внимание на комментарий: запись `/boot` в `/etc/fstab` делает **не** post-build, а `mkimage.sh`. Это правило «знание живёт там, где его источник»: разметку карты знает только сборка образа.

Фрагмент busybox

board/common/buildroot/busybox.fragment

```
# fstab mounts the BOOT partition by label (the mmcblk number differs per board)
CONFIG_FEATURE_MOUNT_LABEL=y
```

`/boot` монтируется по метке `LABEL=BOOT`, потому что номер `mmcblkN` у разных плат разный (на Raspberry Pi 4 SD-карта может стать `mmcblk1`). Но в стандартной конфигурации busybox `mount` **не понимает** ни `LABEL=`, ни `PARTUUID=` (`# CONFIG_FEATURE_MOUNT_LABEL is not set`). Фрагмент включает эту возможность.

ЛОВУШКА: ИЗМЕНЕНИЕ ФРАГМЕНТА НЕ ПЕРЕСОБИРАЕТ ПАКЕТ

Buildroot пересобирает пакет только если его нет или вы попросили явно. После изменения фрагмента busybox для уже собранного rootfs нужно: `make rootfs-make BR=busybox-reconfigure`, затем `make rootfs`. То же касается любых изменений в конфигурации уже собранного пакета (`<пакет>-reconfigure`, `<пакет>-rebuild`, `<пакет>-dirclean`).

ЛОВУШКА: УДАЛЁННЫЕ ФАЙЛЫ OVERLAY ОСТАЮТСЯ

Buildroot копирует overlay в `output/<fw>/buildroot/target/`, но никогда оттуда **не удаляет**. Когда `profile.d/oga.sh` переименовали в `sdk.sh`, в rootfs оказались оба файла. Лечится удалением файла из `target/` руками (или полной пересборкой `make rootfs-clean rootfs`).

Время и место

Первая сборка — самая долгая: Buildroot собирает **тулчейн** (binutils, GCC в два прохода, glibc, заголовки ядра) — 30–60 минут и 5–9 ГБ в `output/<fw>/buildroot`. Повторные сборки быстрые.

ОПАСНО: ПАРАЛЛЕЛЬНЫЕ СБОРКИ BUILDROOT

Buildroot по умолчанию собирает каждый пакет в `nproc + 1` поток. Три сборки Buildroot одновременно на 16 потоках и 14 ГБ ОЗУ, плюс ядро, привели к нехватке памяти: OOM killer убил графическую сессию вместе с IDE и всеми сборками (глава 14). Собирайте тулчейны **по одному**.

Можно сократить первую сборку, взяв готовый внешний тулчейн Bootlin (`BR2_TOOLCHAIN_EXTERNAL_BOOTLIN_*`): он скачивается за минуту и экономит 3–5 ГБ. В SDK остались собственные тулчейны, чтобы у всех плат были одинаковые версии GCC/glibc и не было готовых бинарников. Решение стоит пересмотреть, если важнее скорость.

SDK для приложений

```
make rootfs-sdk
# → output/<fw>/images/aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz
```

Это переносимый тулчейн с sysroot — в нём уже есть заголовки и библиотеки всех пакетов из rootfs. Использование:

```
tar xf aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz -C ~/opt
SDK=~/opt/aarch64-buildroot-linux-gnu_sdk-buildroot
$SDK/relocate-sdk.sh                    # прописать новый путь внутрь SDK
$SDK/bin/aarch64-buildroot-linux-gnu-gcc hello.c -o hello
scp hello root@board:/root/             # на BBB/RPi есть dropbear
```

Для CMake в SDK есть готовый toolchain-файл: `cmake -DCMAKE_TOOLCHAIN_FILE=$SDK/share/buildroot/toolchainfile.cmake ..`. Если включить в defconfig `BR2_PACKAGE_HOST_ENVIRONMENT_SETUP=y`, в SDK появится скрипт `environment-setup`, который выставляет `CC`, `CFLAGS` и прочее: `source $SDK/environment-setup`.

Отладка — `gdbserver :2345 ./hello` на плате и `aarch64-buildroot-linux-gnu-gdb` на ПК (`target remote board:2345`).

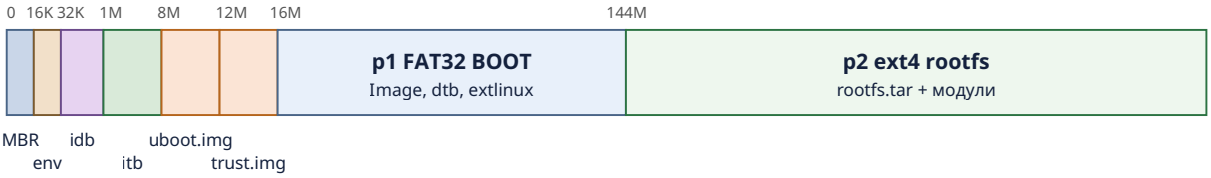
> **УПРАЖНЕНИЕ: СВОЙ ПАКЕТ**
>
> Создайте `board/common/buildroot/package/hello/` с `Config.in` и `hello.mk` (тип `generic-package`, исходник — локальный каталог через `HELLO_SITE_METHOD = local`), подключите его строкой в `board/common/buildroot/Config.in` и включите в defconfig платы. Проверьте, что `make rootfs` кладёт программу в rootfs.

ГЛАВА 10

Образ SD-карты

Все компоненты собраны по отдельности. Теперь их нужно разложить на «виртуальную SD-карту» — файл `.img`, который потом побайтово пишется на настоящую карту. Делает это `scripts/mkimage.sh`, и делает **без root**.

Разметка



Масштаб условный. Сырые области (слева) у каждой платы свои и задаются IMAGE_RAW; разделы — общие для всех.

Рис. 10.1. Разметка образа OGA

Где	Что	Кто читает
сектор 0	MBR: таблица разделов, disk id	все
16–32 КиБ	окружение U-Boot (`saveenv`)	U-Boot
`IMAGE_RAW` (до 16 МиБ)	сырые блобы загрузчика	BootROM, SPL, miniloader
16 МиБ	p1, FAT32, метка `BOOT`, активный	U-Boot, прошивка RPi, ROM BBB
дальше	p2, ext4, метка `rootfs`	ядро (`root=PARTUUID=…-02`)

Почему MBR, а не GPT

GPT — современный формат, но занимает сектора 1–33 (заголовок и таблица записей) и ещё копию в конце диска. На Rockchip окружение U-Boot по умолчанию лежит с 16 КиБ (сектор 32) — пересечение. Кроме того, ROM AM335x ищет **активный** (bootable) раздел в MBR, а Raspberry Pi 1–3 вообще понимают только MBR. MBR занимает один сектор и подходит всем.

Почему FAT для BOOT

FAT читают U-Boot, ROM AM335x (`MLO` с FAT) и прошивка Raspberry Pi (`config.txt`, `start.elf`). И его можно открыть на любом ПК, чтобы поправить `extlinux.conf`.

Почему 16 МиБ до первого раздела

Это запас на все сырые блобы. Самая «толстая» раскладка — у OGA: `trust.img` с сектора 24576 (12 МиБ) занимает 4 МиБ и кончается ровно на 16 МиБ.

mkimage.sh по шагам

Скрипт получает всё через переменные окружения (их выставляет `make image`) и проходит четыре шага.

1. Проверки и сырые блобы

scripts/mkimage.sh, строки 24–52

```
P1_START=32768  # 16 MiB, in 512-byte sectors

die() { echo "mkimage: $*" >&2; exit 1; }

# System tools first, Buildroot host tools as a fallback
export PATH="$PATH:/usr/sbin:/sbin${HOST_TOOLS:+:$HOST_TOOLS}"
need() { command -v "$1" >/dev/null || die "'$1' not found (package: $2)"; }
need sfdisk fdisk
need mkfs.vfat dosfstools
need mcopy mtools
need mke2fs e2fsprogs
need fakeroot fakeroot

for f in "$KERNEL" "$ROOTFS_TAR"; do
    [ -e "$f" ] || die "missing $f: build the component first"
done

# --- raw blobs: sorted by sector, must not overlap each other or p1 ---
raw_sorted=$(for r in $RAW; do echo "${r##*@} ${r%@*}"; done | sort -n)
first_sector=$P1_START
prev_end=1
while read -r sector file; do
    [ -n "$sector" ] || continue
    [ -f "$file" ] || die "missing $file: build the component first"
    [ "$sector" -ge "$prev_end" ] || die "$(basename "$file") at sector $sector overlaps the
previous blob"
    prev_end=$(( sector + ($(stat -c %s "$file") + 511) / 512 ))
    [ "$prev_end" -le $P1_START ] || die "$(basename "$file") at sector $sector overlaps the
first partition"
    if [ "$sector" -lt "$first_sector" ]; then first_sector=$sector; fi
done <<< "$raw_sorted"
```

`set -euo pipefail` (в начале файла) останавливает скрипт на первой ошибке, на неопределённой переменной и на ошибке внутри конвейера.

`IMAGE_RAW` — список вида `файл@сектор`. Скрипт сортирует блобы по сектору и проверяет, что они не перекрывают друг друга и первый раздел. Это защита от ошибки, которая погубила прошлую попытку (глава 3). Размер блобов зависит от раскладки: `u-boot-rockchip.bin` весит ≈ 9 МиБ, когда FIT лежит на 8 МиБ, и ≈ 2 МиБ, когда на 1 МиБ. Сдвинули одно — и без проверки легко не заметить, что блоб налез на соседний.

2. Раздел BOOT

scripts/mkimage.sh, строки 54–93

```
rm -rf "$WORK_DIR"
mkdir -p "$WORK_DIR/boot/extlinux"

# --- boot partition ---
BOOT_DIR=$WORK_DIR/boot
cp "$KERNEL" "$BOOT_DIR/"
if [ -d "$LINUX_DIR/dtbs" ]; then
    case $DTB_LAYOUT in
    tree|both) cp -r "$LINUX_DIR/dtbs/." "$BOOT_DIR/" ;;&
    flat|both) find "$LINUX_DIR/dtbs" -name '*.dtb' -exec cp {} "$BOOT_DIR/" \; ;;
    tree) ;;
    *) die "unknown DTB_LAYOUT '$DTB_LAYOUT' (tree, flat or both)" ;;
    esac
fi

for f in $BOOT_FILES; do
    src=${f%%:*}
    dst=${f#*:}; [ "$dst" != "$f" ] || dst=$(basename "$src")
```

```
    [ -f "$src" ] || die "missing $src: build the component first"
    cp "$src" "$BOOT_DIR/$dst"
done

disk_id=$(printf '%08x' $(( DISK_ID )))
# U-Boot sets ${fdtfile} from the board model; fdtdir prepends the path.
cat > "$BOOT_DIR/extlinux/extlinux.conf" <<EOT
default linux

label linux
    kernel /$(basename "$KERNEL")
    fdtdir /
    append root=PARTUUID=$disk_id-02 $CMDLINE
EOT

boot_kb=$(du -sk "$BOOT_DIR" | cut -f1)
[ "$boot_kb" -lt $(( BOOT_SIZE_MB * 1024 * 95 / 100 )) ] || \
    die "boot files ($((boot_kb / 1024)) MiB) do not fit into IMAGE_BOOT_SIZE_MB=$BOOT_SIZE_M
B"

BOOT_IMG=$WORK_DIR/boot.vfat
mkfs.vfat -F 32 -n BOOT -i "$disk_id" -C "$BOOT_IMG" $(( BOOT_SIZE_MB * 1024 )) >/dev/null
mcopy -i "$BOOT_IMG" -s -p -m "$BOOT_DIR"/* ::/
```

- Ядро, dtb и `IMAGE_BOOT_FILES` складываются в каталог.
- Раскладка dtb (`DTB_LAYOUT`) зависит от того, как U-Boot платы строит `fdtfile`: `rockchip/rk3326-odroid-go2-v11.dtb` (с каталогом) у OGA, `am335x-boneblack.dtb` (без каталога) у BBB. Конструкция `;;&` в `case` значит «выполнить и проверить следующие шаблоны» — так `both` выполняет обе ветки.
- `extlinux.conf` минимальный: одна метка, `fdtdir /`, корень по PARTUUID.
- `mkfs.vfat -C` **создаёт файл** нужного размера с файловой системой, а `mcopy` из пакета mtools копирует в него файлы — без `mount` и без root. Серийный номер тома (`-i`) — тот же disk id, просто для красоты.

3. Раздел rootfs

scripts/mkimage.sh, строки 95–114

```
# --- rootfs partition ---
ROOT_DIR=$WORK_DIR/rootfs
ROOT_IMG=$WORK_DIR/rootfs.ext4
fakeroot -- bash -euo pipefail -c '
    root=$1 tar=$2 mods=$3 free_mb=$4 img=$5
    mkdir -p "$root"
    tar -xpf "$tar" -C "$root"
    if [ -d "$mods/lib/modules" ]; then
        mkdir -p "$root/lib/modules"
        cp -r "$mods/lib/modules/." "$root/lib/modules/"
        chown -R 0:0 "$root/lib/modules"
    fi
    # BOOT partition on /boot, by label: the mmcblk number differs per board
    mkdir -p "$root/boot"
    sed -i "/[[:space:]]\/boot[[:space:]]/d" "$root/etc/fstab"
    printf "LABEL=BOOT\t/boot\tvfat\tdefaults,noatime\t0\t0\n" >> "$root/etc/fstab"
    size_mb=$(( $(du -sm "$root" | cut -f1) * 11 / 10 + free_mb ))
    mke2fs -q -F -t ext4 -L rootfs -d "$root" -E root_owner=0:0 "$img" "${size_mb}M"
' _ "$ROOT_DIR" "$ROOTFS_TAR" "$LINUX_DIR/modules" "$ROOTFS_FREE_MB" "$ROOT_IMG"
rm -rf "$ROOT_DIR"
```

Это самая интересная часть. Rootfs должна содержать файлы, принадлежащие root (`/bin/busybox` с битом setuid, `/root`, `/etc/shadow`…). Обычный пользователь не может создать

файл, принадлежащий root. Решение — **fakeroot**: он перехватывает вызовы `chown`, `stat` и т. п. и **запоминает** «как будто» установленных владельцев. Внутри одной сессии fakeroot:

1. `tar -xp` распаковывает `rootfs.tar` с владельцами из архива (fakeroot их запомнит);
2. в `/lib/modules` копируются модули ядра и «отдаются» root;
3. в `/etc/fstab` дописывается строка для `/boot` по метке;
4. `mke2fs -d <каталог>` создаёт ext4 **сразу с содержимым**. Он читает владельцев через `stat` — а fakeroot отвечает «root».

Все четыре шага должны идти **в одной** сессии fakeroot: между сессиями память о владельцах теряется (если не использовать файл состояния `-s`/`-i`).

Размер раздела — содержимое плюс 10 % плюс `IMAGE_ROOTFS_FREE_MB` свободного места.

4. Сборка диска

scripts/mkimage.sh, строки 116–148

```
# --- disk ---
boot_sectors=$(( BOOT_SIZE_MB * 2048 ))
p2_start=$(( P1_START + boot_sectors ))
root_sectors=$(( $(stat -c %s "$ROOT_IMG") / 512 ))
total_sectors=$(( p2_start + root_sectors + 2048 ))

TMP_IMG=$OUT_IMG.tmp
rm -f "$TMP_IMG"
truncate -s $(( total_sectors * 512 )) "$TMP_IMG"
sfdisk -q "$TMP_IMG" <<EOT
label: dos
label-id: $DISK_ID
start=$P1_START, size=$boot_sectors, type=c, bootable
start=$p2_start, size=$root_sectors, type=83
EOT

put() { dd if="$1" of="$TMP_IMG" bs=1M seek=$(( $2 * 512 )) oflag=seek_bytes conv=notrunc sta
tus=none; }
while read -r sector file; do
    if [ -n "$sector" ]; then put "$file" "$sector"; fi
done <<< "$raw_sorted"
put "$BOOT_IMG" "$P1_START"
put "$ROOT_IMG" "$p2_start"
mv "$TMP_IMG" "$OUT_IMG"

# Bootloader blobs + BOOT partition for "make flash-boot"
out_dir=$(dirname "$OUT_IMG")
dd if="$OUT_IMG" of="$out_dir/boot-area.bin" bs=1M skip=$(( first_sector * 512 )) \
    count=$(( (p2_start - first_sector) * 512 )) iflag=skip_bytes,count_bytes status=none
echo "$first_sector" > "$out_dir/boot-area.sector"

echo "Image: $OUT_IMG ($(( total_sectors / 2048 )) MiB)"
sfdisk -l "$OUT_IMG" | sed -n '/^Device/,$p'
echo "root=PARTUUID=$disk_id-02"
```

- `truncate` создаёт разреженный файл нужного размера (на диске почти не занимает места).
- `sfdisk` умеет работать с обычным файлом как с диском: записывает MBR с disk id `label-id` и двумя разделами; у первого — флаг `bootable`.
- `dd ... oflag=seek_bytes conv=notrunc` кладёт каждый кусок по точному байтовому смещению, не обрезая файл. `bs=1M` с `seek` в байтах — быстро и без арифметики в блоках.

- `boot-area.bin` — всё от первого сырого блоба до конца раздела BOOT. Его пишет `make flash-boot`, чтобы обновить загрузчик и ядро, не трогая rootfs.

PARTUUID и LABEL

Ядро должно найти rootfs, а init — раздел BOOT. Варианты:

Способ	Пример	Плюсы и минусы
Имя устройства	`root=/dev/mmcblk0p2`	зависит от порядка инициализации контроллеров
PARTUUID	`root=PARTUUID=4f474131-02`	понимает **ядро** без initramfs; для MBR = disk id + номер раздела
UUID ФС	`root=UUID=...`	ядро без initramfs **не** понимает
LABEL	`LABEL=BOOT` в fstab	понимает `mount` (busybox — с `FEATURE_MOUNT_LABEL`)

Поэтому: корень — по PARTUUID (его разбирает само ядро), `/boot` в fstab — по метке (её разбирает `mount`). Disk id задаётся в конфиге (`IMAGE_DISK_ID`), разный у разных плат — так их образы легко различить.

Проверка образа без железа

Прежде чем писать на карту, образ стоит проверить. Всё делается без root:

```
O=output/oga
sfdisk -l $O/images/oga.img                              # таблица разделов
mdir -i $O/image-work/boot.vfat -/ ::/                   # что в BOOT
mtype -i $O/image-work/boot.vfat ::/extlinux/extlinux.conf
debugfs -R 'cat /etc/fstab' $O/image-work/rootfs.ext4    # файл из ext4
debugfs -R 'ls -l /lib/modules' $O/image-work/rootfs.ext4
# загрузчик лежит там, где надо?
dd if=$O/images/oga.img bs=512 skip=64 count=$(( $(stat -c%s $O/images/u-boot-rockchip.bin)/5
12 )) \
   status=none | cmp - $O/images/u-boot-rockchip.bin && echo "u-boot @64 OK"
dd if=$O/images/oga.img bs=512 skip=2048 count=1 status=none | xxd -l 8   # d00dfeed = FIT
```

СОВЕТ: ПРОГОН НА ФИКТИВНОМ ROOTFS

Пока Buildroot собирается (а это час), скрипт образа можно отладить на пустышке: `mkdir -p fake/{bin,etc} && tar -C fake -cf fake.tar .` и запустить `mkimage.sh` с `ROOTFS_TAR=fake.tar`. Ошибки в разметке всплывут за секунды. Именно так в проекте нашли, что в системе нет `mcopy` (пакет `mtools`).

СИГНАТУРЫ, КОТОРЫЕ ПОЛЕЗНО ЗНАТЬ

`d0 0d fe ed` — FIT или dtb (это одна и та же структура). `3b 8c dc fc` — начало idbloader Rockchip (заголовок зашифрован RC4, и так выглядит его первая часть). `LOADER` — `uboot.img` формата Rockchip; `BL3X` — `trust.img` (глава 12); `55 aa` в конце сектора 0 — сигнатура MBR.

Запись на карту

scripts/flash.sh

```
#!/bin/bash
# Write a file to a removable block device at a sector offset.
# Usage: flash.sh <file> <device> <sector>
set -euo pipefail

FILE=$1 DEV=${2:-} SECTOR=${3:-0}
die() { echo "flash: $*" >&2; exit 1; }

if [ -z "$DEV" ]; then
    echo "Specify the SD card: DEV=/dev/sdX. Removable devices:"
    lsblk -d -o NAME,SIZE,TRAN,RM,MODEL | awk 'NR==1 || $4==1'
    exit 1
fi
[ -f "$FILE" ] || die "$FILE not found: build it first"
[ -b "$DEV" ] || die "$DEV is not a block device"
[ "$(lsblk -dno TYPE "$DEV")" = disk ] || die "$DEV is a partition, pass the whole disk"
if [ "$(lsblk -dno RM "$DEV")" != 1 ] && [ "$(lsblk -dno TRAN "$DEV")" != usb ] && [[ $DEV !
= /dev/mmcblk* ]]; then
    die "$DEV is not removable, refusing to write"
fi
if lsblk -lno MOUNTPOINT "$DEV" | grep -qxE '/|/boot|/home|/usr|/var'; then
    die "$DEV holds a system mount point"
fi

lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT "$DEV"
echo
read -r -p "Write $(basename "$FILE") to $DEV at sector $SECTOR? Data on $DEV will be lost.
Type 'yes': " answer
[ "$answer" = yes ] || die "aborted"

for part in $(lsblk -lnpo NAME,MOUNTPOINT "$DEV" | awk '$2 != "" {print $1}'); do
    sudo umount "$part"
done
sudo dd if="$FILE" of="$DEV" bs=4M seek=$(( SECTOR * 512 )) oflag=seek_bytes conv=fsync statu
s=progress
sync
echo "Done."
```

Запись на неправильное устройство — самая дорогая ошибка в этой области, поэтому скрипт осторожен:

- отказывается писать на раздел (`/dev/sda1`) вместо диска;
- отказывается писать на **несъёмный** диск (не `RM=1`, не USB, не `mmcblk`);
- отказывается, если на устройстве смонтирован `/`, `/boot`, `/home`…;
- показывает `lsblk` и требует набрать `yes`;
- отмонтирует разделы карты, пишет с `conv=fsync` и делает `sync`.

```
make flash DEV=/dev/sdX          # весь образ
make flash-boot DEV=/dev/sdX     # только загрузчик + BOOT (rootfs остаётся)
```

ЛОВУШКА: ОБРАЗ КАКОЙ ПЛАТЫ БУДЕТ ЗАПИСАН

`make flash` пишет образ **выбранной** прошивки — той, что записана в `output/.fw` последней командой `make <fw>_defconfig`. `FW=<fw>` в командной строке действует только на одну команду и выбор не меняет. Образы разных плат лежат рядом (`output/oga/images/oga.img`, `output/bbb/images/bbb.img`, `output/rpi1/images/rpi1.img`, `output/rpi4/images/rpi4.img`), поэтому легко собрать одну плату через `FW=`, а записать — другую.

```
bash make info # строка Firmware: — что выбрано make FW=rpi4 flash DEV=/dev/sdX # записать rpi4 разово make rpi4_defconfig && make flash DEV=/dev/sdX # переключиться на rpi4 насовсем
```

ОПАСНО

Перед `make flash` проверьте `lsblk`: какое устройство появилось, когда вы вставили карту. `/dev/sda` на одной машине — SD-кардридер, на другой — системный диск.

ГЛАВА 11

Отладка: «прошил, вставил — ничего не происходит»

Первый образ OGA собрался, прошёл все проверки из главы 10 и был записан на карту. Консоль включили — экран чёрный, горят только красные светодиоды. Эта глава — подробный разбор того, как от этого симптома дойти до причины. Методика важнее результата: она пригодится на любой плате.

Шаг 1. Получить UART

Без последовательной консоли отладка загрузчика — гадание. У OGA есть разъём UART (TX, RX, GND на торце, 3,3 В). Адаптер USB-UART подключается к ПК:

```
ls -l /dev/ttyUSB*                  # crw-rw---- root dialout ... /dev/ttyUSB0
id | grep dialout                   # пользователь должен быть в группе dialout
lsusb | grep -i -E 'cp210|ch34|ftdi|pl23'
lsusb -v -d 10c4:ea60 | grep iProduct    # → CP2102 USB to UART Bridge Controller
```

ЛОВУШКА: МОДЕЛЬ АДАПТЕРА ИМЕЕТ ЗНАЧЕНИЕ

Классический **CP2102** умеет максимум **921600 бод**. Rockchip любит 1 500 000. CP2102N, CP2104, FT232R, CH340 умеют 1,5–3 Мбод. Модель видна в `lsusb -v`.

Шаг 2. Порт занят

```
$ stty -F /dev/ttyUSB0 115200 raw
stty: Device or resource busy
$ fuser -v /dev/ttyUSB0
/dev/ttyUSB0:  sobs  696728 F.... screen
$ screen -ls
    696728.pts-2.sobs-pc    (Detached)
```

Порт держала **отсоединённая** сессия `screen`, запущенная командой `screen /dev/ttyUSB0` — **без указания скорости**. Без скорости `screen` открывает порт на 9600 бод, а загрузка печатает на 115200: на экране был бы мусор или ничего.

КАК СНЯТЬ СОДЕРЖИМОЕ ЧУЖОЙ СЕССИИ SCREEN

`screen -S <pid> -X hardcopy -h файл` сохраняет экран и историю прокрутки в файл, не трогая сессию. Здесь он показал пустоту — подтверждение, что на 9600 ничего полезного не пришло. Правильный запуск: `screen /dev/ttyUSB0 115200`, выход — `Ctrl-A`, `K`.

Шаг 3. Записать загрузку в файл

Глазами быстрый поток не прочитать, а мусор не скопировать. Пишем порт в файл **до** включения платы:

```
stty -F /dev/ttyUSB0 115200 raw -echo cs8 -cstopb -parenb -crtscts
timeout 180 cat /dev/ttyUSB0 > boot.log      # включаем плату, ждём
```

```
wc -c boot.log
cat -v boot.log | head                          # непечатные байты как ^X и M-x
```

`raw` выключает всю обработку символов терминалом, `cs8 -cstopb -parenb` — формат 8N1, `-crtscts` — без аппаратного управления потоком. Перед захватом полезно отправить Enter (`printf '\r\n' > /dev/ttyUSB0`): если плата уже висит в U-Boot или Linux, придёт приглашение.

Шаг 4. Читать мусор

Захват на 115200 при включении:

```
68 байт
^@^@^@^@M-xSM-{.M-{M-g6M-oM-:M-nM-^[M-^?&M-~wM-/M-~M-^MM-^ZM-#Q^K...
```

Четыре нулевых байта в начале — переходный процесс на линии TX при включении питания. Дальше — **мусор**. Мусор — это не «ничего»: плата что-то передаёт, но на другой скорости. Какой?

Оценка скорости по мусору

Когда приёмник медленнее передатчика, он видит «старт-бит» на любом переходе в ноль, а остальные биты выбирает редко — получаются байты, в которых много единиц (`0xFF`, `0xFE`, `0xF8`…). Именно такие байты в захвате (`M-x` = `0xF8`, `M-{` = `0xFB`, `M-~` = `0xFE`). Значит, источник **быстрее** 115200.

Порядок величины можно оценить. Если источник передаёт на скорости `S`, а приёмник настроен на `R < S`, то один принятый «байт» (10 бит на скорости `R`) покрывает примерно `S/R` переданных байтов. Для 1 500 000 / 115 200 ≈ 13:

```
68 принятых байт × 13 ≈ 880 байт, переданных на 1,5 Мбод
```

Затем запись на «1 500 000». CP2102 такую скорость не умеет; драйвер `cp210x` ограничивает скорость максимумом модели, так что реально порт, по-видимому, работал на 921 600 (`stty` при этом молча «соглашается» на 1 500 000). Получили 707 байт мусора. Пересчёт: 1 500 000 / 921 600 ≈ 1,63, 707 × 1,63 ≈ 1150 байт.

Две независимые оценки сходятся: **около 1 КиБ текста на ≈ 1,5 Мбод**.

> **ЭТО ГРУБАЯ ОЦЕНКА**
> Она предполагает непрерывный поток и ничего не говорит о содержимом. Но для гипотез её достаточно: «быстро» против «медленно» и «полстраницы» против «трёх строк».

Шаг 5. Что должно было печататься

Что напечатал бы наш загрузчик? Смотрим код, а не гадаем:

```
grep -n -E 'printascii|printf' src/u-boot/drivers/ram/rockchip/sdram_px30.c
#   715:    printascii("out\n");          ← TPL после инициализации DDR
```

TPL после инициализации DDR печатает `out`, SPL — баннер `U-Boot SPL 2026.07 (...)`. Оба — на `CONFIG_BAUDRATE` = 115200 через debug UART, тактируемый от 24 МГц (`board_debug_uart_init()` в `px30.c`). В захвате на 115200 нет ни одного читаемого символа. Вывод: **наш TPL вообще не запускается**. А ≈ 1 КиБ на 1,5 Мбод — это типичный лог DDR-блоба и miniloader из rkbin Rockchip (`DDR Version ...`, `Boot1 Release Time ...`, `SdmmcInit ...`).

Шаг 6. Исключить очевидное

Прежде чем строить сложные гипотезы, проверяем простое: а на карте точно наш образ?

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,PARTUUID /dev/sda
#  sda1 128M vfat BOOT   4f474131-01
#  sda2 363M ext4 rootfs 4f474131-02         ← таблица разделов наша
cmp /run/media/$USER/BOOT/Image output/oga/linux-install/Image && echo "Image OK"
sudo dd if=/dev/sda bs=512 skip=64 count=18496 status=none \
  | cmp - output/oga/images/u-boot-rockchip.bin && echo BOOTLOADER OK
```

Всё совпало: и разделы, и ядро, и загрузчик по сектору 64.

Шаг 7. Индикатор без UART

Ещё одна проверка без всякого UART: у U-Boot OGA в DTS стоит

```
&blue_led { default-state = "on"; };      /* rk3326-odroid-go2-u-boot.dtsi */
```

Если U-Boot proper стартует, загорается **синий** светодиод. Он не горел — до U-Boot дело не доходило. Это сходится с шагом 5.

> **СОВЕТ**
> Ищите в DTS для U-Boot (`*-u-boot.dtsi`) светодиоды с `default-state`, подсветку, вибромотор — всё, что меняет видимое состояние платы. Это бесплатный индикатор «до какой стадии дошли».

Шаг 8. Кто печатает на 1,5 Мбод?

Если наш код с SD не стартует, но кто-то печатает лог загрузчика Rockchip — значит, BootROM нашёл загрузчик **где-то ещё**. На плате есть SPI-флеш: в DTS OGA есть узел `jedec,spi-nor`. А BootROM RK3326 проверяет SPI **раньше** SD.

Во SPI-флеш ничего не писали. Но в наработках лежал форк U-Boot от Hardkernel, и в нём нашлись два очень говорящих файла:

```
$ ls sd_fuse/
boot.ini  idbloader.img  rk3326_header_miniloader_spiboot.img  sd_fusing.sh  ...
$ strings -n 6 sd_fuse/rk3326_header_miniloader_spiboot.img | head
EFI PART
...
DDR Version 1.12 20190528
```

И скрипт `build_recovery.sh`:

```
pack_spi_full_image() {
  dd if=rk3326_header_miniloader_spiboot.img of=spi_recovery.img seek=0 count=2048
  dd if=uboot_spi.img of=spi_recovery.img seek=2048 count=2048
  dd if=trust_spi.img of=spi_recovery.img seek=4096 count=4096
  # + dtb и картинки: logo_hardkernel, low_battery, no_sdcard, recovery ...
}
```

Hardkernel собирает **образ для SPI-флеша**: DDR-блоб + miniloader + свой U-Boot + картинки «нет SD-карты», «низкий заряд». У Black Edition этот recovery-загрузчик **стоит во SPI с завода**. Картина сложилась:

1. BootROM находит загрузчик во SPI и запускает его (DDR-блоб + miniloader печатают ≈ 1 КиБ на 1,5 Мбод — это наш мусор).
2. Miniloader видит SD-карту и пытается загрузить с неё `uboot.img` (сектор 16384) и `trust.img` (сектор 24576) **в формате Rockchip**.
3. По сектору 16384 на нашей карте лежит mainline-FIT `u-boot.itb`, а не `uboot.img`. Miniloader его не принимает и зависает. Горят только красные светодиоды.
4. Старая карта EmuELEC/Hardkernel работала именно потому, что на ней `uboot.img` и `trust.img` лежали в формате Rockchip по этим секторам.

> **ПОДТВЕРЖДЕНИЕ**
> Окончательно гипотезу подтвердил гибридный образ (глава 12): как только на карте появился `uboot.img` в формате Rockchip, консоль загрузилась. Ещё один простой тест, который стоит знать: включить плату **без SD-карты**. Если мусор на 1,5 Мбод тот же — загрузчик живёт во SPI/eMMC.

Варианты решения

Вариант	Как	Плюсы	Минусы
Стереть заголовок SPI	бэкап `sf read` → `fatwrite`, затем `sf erase 0 0x100000`	полностью mainline, всё на SD	меняет заводскую прошивку; карта не загрузится на стоковой консоли
Mainline U-Boot во SPI	`u-boot-rockchip-spi.bin`	быстро	загрузчик уходит из образа; сломанный SPI-загрузчик сложно восстановить
`uboot.img` / `trust.img` в формате Rockchip на SD	`loaderimage`, `trust_merger`	SPI не трогаем; работает на любой стоковой BE	начало цепочки — блобы Rockchip
Гибрид	и mainline idbloader, и RK-формат на одной карте	работает и со SPI, и без него	чуть сложнее разметка

Выбран гибрид — глава 12.

ЕСЛИ ВСЁ ЖЕ ТРОГАТЬ SPI

Всегда сначала бэкап: в U-Boot консоли `sf probe`, `sf read 0x02000000 0 0x1000000`, `fatwrite mmc 1:1 0x02000000 spi-backup.bin 0x1000000` — и проверить файл на ПК (`strings` должен показать `EFI PART` и `DDR Version`). Стирайте только заголовок (`sf erase 0 0x100000`), а не весь флеш. Если записать во SPI валидный, но зависающий загрузчик, BootROM будет запускать его вместо SD, и восстанавливать придётся через режим MaskROM по USB.

Методика в сжатом виде

1. **Получить данные**: UART в файл, с момента включения, на нужной скорости.
2. **Читать мусор**: мусор значит «не та скорость»; характер байтов говорит, быстрее или медленнее; количество — порядок объёма.
3. **Знать, что должно быть**: найти в коде, что и на какой скорости печатает каждая стадия.
4. **Исключить очевидное**: на карте то, что вы думаете? `cmp` по секторам.
5. **Искать побочные индикаторы**: светодиоды, подсветка, USB-устройства в `lsusb`.
6. **Читать чужой код**: вендорские скрипты (`build_recovery.sh`) рассказывают про железо больше, чем документация.
7. **Проверять гипотезу минимальным изменением**: тест без SD, один новый файл на карте.

ГЛАВА 12

Гибридный образ для OGA

Задача: одна SD-карта, которая грузится и на консоли с заводским загрузчиком во SPI, и на консоли с пустым SPI. Для этого на карте должны лежать **обе** цепочки:

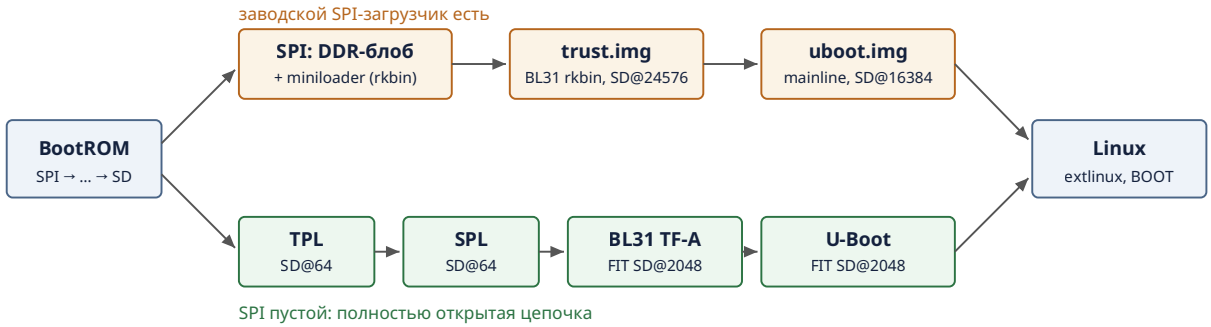


Рис. 12.1. Две цепочки загрузки на одной карте

Разметка

```
сектор 64      u-boot-rockchip.bin: idbloader (TPL+SPL)        ← mainline
сектор 2048    u-boot.itb (FIT: U-Boot + TF-A BL31 + dtb)      ← mainline, перенесён
сектор 16384   uboot.img  (U-Boot в формате Rockchip)          ← для miniloader
сектор 24576   trust.img  (BL31 в формате Rockchip)            ← для miniloader
сектор 32768   раздел BOOT
```

Конфликт был один: и SPL, и miniloader хотят свой файл по сектору 16384. Miniloader изменить нельзя (он во SPI, закрытый), а SPL — можно.

Перенос FIT

board/oga/u-boot/oga.config

```
# ODROID-GO Advance: options on top of odroid-go2_defconfig.

# Same serial speed as TF-A (taken from stdout-path) and the kernel
CONFIG_BAUDRATE=115200
CONFIG_BOOTDELAY=2

# u-boot.itb at sector 2048 (1 MiB) instead of 16384: sectors 16384 and
# 24576 hold uboot.img/trust.img for the Rockchip miniloader (SPI flash)
CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR=0x800
CONFIG_SPL_PAD_TO=0xf8000
```

- `CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR=0x800` — SPL ищет FIT в секторе 2048 (1 МиБ).
- `CONFIG_SPL_PAD_TO` — смещение FIT **внутри** `u-boot-rockchip.bin`. Файл пишется с сектора 64, значит смещение в нём = `(0x800 − 0x40) × 512 = 0xF8000`.

Откуда известно, что binman берёт именно `SPL_PAD_TO`? Из описания образа:

```
grep -n -B2 -A6 'SPL_PAD_TO' src/u-boot/arch/arm/dts/rockchip-u-boot.dtsi
#   u-boot-img { offset = <CONFIG_SPL_PAD_TO>; };
```

По умолчанию `SPL_PAD_TO=0x7f8000` = `(0x4000 - 0x40) × 512` — это и есть «FIT по сектору 16384». Две опции **связаны**, и менять их нужно вместе. Если поменять только сектор, SPL будет искать FIT там, где его нет.

Форматы Rockchip

uboot.img

Это бинарник U-Boot с заголовком `LOADER`: адрес загрузки, размер, CRC и хеш. Miniloader проверяет заголовок, кладёт U-Boot по адресу из заголовка и запоминает его как BL33. Упаковывает утилита `loaderimage` из rkbin:

```
loaderimage --pack --uboot output/oga/u-boot/u-boot-dtb.bin uboot.img 0x00800000
```

Адрес — **`CONFIG_TEXT_BASE`** нашего U-Boot: тот адрес, по которому он слинкован. Скрипт берёт его из `.config` автоматически. У рабочего `uboot.img` Hardkernel адрес был `0x200000` — их U-Boot слинкован иначе. Взять «как у них» было бы ошибкой.

```
$ loaderimage --info uboot.img          (наш)      $ loaderimage --info uboot.img
(Hardkernel)
Load Addr is 0x800000                              Load Addr is 0x200000
```

`u-boot-dtb.bin` — U-Boot proper с приклеенным **одним** dtb (по умолчанию `rk3326-odroid-go2`, rev 1.0). В mainline-цепочке SPL выбирает dtb из FIT по ревизии, а здесь SPL нет. Это не страшно: драйверы, нужные U-Boot (SD, UART, ADC), у всех ревизий одинаковые, а `fdtfile` для **ядра** go2.c всё равно ставит по АЦП — и в логе видно, что для BE выбран `rk3326-odroid-go2-v11.dtb`.

trust.img

Контейнер с заголовком `BL3X` для BL31 (и опционально BL32). Собирается утилитой `trust_merger` по ini-файлу:

```
[BL31_OPTION]
SEC=1
PATH=/abs/path/rk3326_bl31_v1.34.elf
ADDR=0x00040000
[BL32_OPTION]
SEC=0
```

`ADDR` — точка входа BL31. Её стоит проверить, а не копировать из примера:

```
aarch64-linux-gnu-readelf -h rk3326_bl31_v1.34.elf | grep Entry   # 0x40000
```

Почему BL31 от Rockchip, а не mainline

Mainline TF-A ждёт параметры от BL2 в формате, который ей передаёт U-Boot SPL. Miniloader передаёт параметры в своём формате, и будет ли mainline TF-A с ним работать — не проверено. BL31 из rkbin гарантированно работает с miniloader: Hardkernel использует именно эту пару.

Почему без BL32

В rkbin-шаблоне `RK3326TRUST.ini` есть BL32 (OP-TEE) по адресу `0x08400000`. Его не включили. OP-TEE занимает кусок DRAM и закрывает его аппаратно (TrustZone). Ядро должно знать об этой области (узел `reserved-memory` в device tree), иначе оно может обратиться к защищённой памяти и упасть. В mainline-DTS PX30 такого узла нет — это легко проверить: `grep -n 'reserved-memory\|optee' px30.dtsi rk3326*.dtsi` ничего не находит. Без BL32 этой проблемы нет, а OP-TEE нашему SDK не нужен.

Скрипт упаковки

scripts/rk-legacy-pack.sh

```
#!/bin/bash
# Pack mainline U-Boot proper and BL31 in Rockchip legacy format, for boards
# whose SPI flash holds Rockchip's DDR blob + miniloader (e.g. the factory
# loader of ODROID-GO Advance Black Edition). The miniloader loads
# uboot.img from sector 16384 and trust.img from sector 24576 of the SD card.
#
# Usage: rk-legacy-pack.sh <rkbin dir> <u-boot objdir> <bl31.elf> <out dir>
set -euo pipefail

RKBIN=$1 UBOOT_OUT=$2 BL31=$3 OUT=$4

die() { echo "rk-legacy-pack: $*" >&2; exit 1; }
[ -x "$RKBIN/tools/loaderimage" ] || die "$RKBIN/tools/loaderimage not found"
[ -x "$RKBIN/tools/trust_merger" ] || die "$RKBIN/tools/trust_merger not found"
[ -f "$BL31" ] || die "BL31 $BL31 not found"

load_addr=$(sed -n 's/^CONFIG_TEXT_BASE=//p' "$UBOOT_OUT/.config")
[ -n "$load_addr" ] || die "CONFIG_TEXT_BASE not found in $UBOOT_OUT/.config"

mkdir -p "$OUT"
"$RKBIN/tools/loaderimage" --pack --uboot "$UBOOT_OUT/u-boot-dtb.bin" "$OUT/uboot.img" "$load
_addr" >/dev/null

# BL31 only: without a BL32 the kernel needs no OP-TEE memory reservation
work=$UBOOT_OUT/rk-legacy
mkdir -p "$work"
cat > "$work/trust.ini" <<EOF
[VERSION]
MAJOR=1
MINOR=0
[BL30_OPTION]
SEC=0
[BL31_OPTION]
SEC=1
PATH=$(realpath "$BL31")
ADDR=0x00040000
[BL32_OPTION]
SEC=0
[BL33_OPTION]
SEC=0
[OUTPUT]
PATH=$(realpath "$OUT")/trust.img
EOF
(cd "$work" && "$(realpath "$RKBIN")/tools/trust_merger" --pack trust.ini >/dev/null)

echo "uboot.img: $(stat -c %s "$OUT/uboot.img") bytes, load address $load_addr"
echo "trust.img: $(stat -c %s "$OUT/trust.img") bytes, BL31 $(basename "$BL31")"
```

`trust_merger` запускается из рабочего каталога, потому что пишет временные файлы рядом с собой. Пути в ini — абсолютные (`realpath`).

В конфиге прошивки всё включается тремя переменными (`RK_LEGACY`, `RKBIN_DIR`, `RK_TRUST_BL31`), а в `IMAGE_RAW` при `RK_LEGACY=y` добавляются два блоба:

```
IMAGE_RAW  := $(IMAGES_DIR)/u-boot-rockchip.bin@64
ifeq ($(RK_LEGACY),y)
IMAGE_RAW  += $(IMAGES_DIR)/uboot.img@16384 $(IMAGES_DIR)/trust.img@24576
endif
```

Проверка

```
I=output/oga/images
loaderimage --info $I/uboot.img                 # Load Addr is 0x800000
xxd -l 16 $I/uboot.img                          # 4c4f 4144 4552 2020  "LOADER  "
xxd -l 16 $I/trust.img                          # 424c 3358            "BL3X"
for b in u-boot-rockchip.bin@64 uboot.img@16384 trust.img@24576; do
  f=${b%@*} s=${b#*@}
  dd if=$I/oga.img bs=512 skip=$s count=$(( ($(stat -c%s $I/$f)+511)/512 )) status=none |
    cmp -n $(stat -c%s $I/$f) - $I/$f && echo "$f @ $s OK"
done
dd if=$I/oga.img bs=512 skip=2048 count=1 status=none | xxd -l 4   # d00dfeed: FIT
```

Формат заголовков сравнили с рабочими `uboot.img`/`trust.img` Hardkernel: та же сигнатура и структура.

Результат

Карту перепрошили только в загрузочной области (раздел BOOT и rootfs не менялись) — и консоль загрузилась. Ключевые строки лога UART (сначала короткий мусор от miniloader на 1,5 Мбод, дальше всё читаемо на 115200):

```
U-Boot 2026.07 (Oct 06 2026 - 22:29:26 +0200)
Model: ODROID-GO Advance
DRAM:  1 GiB (total 1022 MiB)
** Booting bootflow 'mmc@ff370000.bootdev.part_1' with extlinux
Retrieving file: /Image
Retrieving file: /rockchip/rk3326-odroid-go2-v11.dtb
Starting kernel ...
[    0.000000] Linux version 7.2.9-oga ...
[    0.000000] Machine model: ODROID-GO Advance Black Edition
[    3.588084] VFS: Mounted root (ext4 filesystem) on device 179:2.
[    3.619962] Run /sbin/init as init process
Welcome to ODROID-GO Advance
odroid-go login:
```

Что тут подтверждается:

- `DRAM: 1 GiB` — U-Boot правильно узнал размер памяти, хотя DDR инициализировал не наш TPL, а DDR-блоб Rockchip;
- `rk3326-odroid-go2-v11.dtb` — go2.c определил Black Edition по АЦП;
- `Machine model: ... Black Edition` — ядро получило правильный dtb;
- `Mounted root ... 179:2` — PARTUUID сработал;
- загорелся синий светодиод, на экране — лог и приглашение.

Безобидные строки в том же логе:

Строка	Почему не страшно
`Loading Boot0000 'mmc 0' failed`	U-Boot сначала пробует EFI-загрузку, не находит и идёт к extlinux
`mmc0: error -110 whilst initialising SD card`	первая попытка инициализации SD, вторая успешна
`FAT-fs (mmcblk0p1): Volume was not properly unmounted`	раздел был смонтирован на ПК при записи
`deferred probe pending: ... wifi-pwrseq`	Wi-Fi ESP8089 Black Edition не поддерживается в mainline

ГЛАВА 13

Несколько плат: BeagleBone Black и Raspberry Pi

SDK, сделанный под одну плату, почти всегда содержит «зашитые» допущения. Перед добавлением новых плат полезно честно их перечислить.

Что было зашито под OGA

Место	Было	Стало
Makefile	`ARCH=arm64`, `arch/arm64/boot/Image`	`LINUX_ARCH`, `LINUX_IMAGE` в конфиге
Makefile	`CROSS_COMPILE ?= aarch64-linux-gnu-`	`CROSS_COMPILE` в конфиге + проверка `check-cc`
TF-A	цель всегда, BL31 всегда передаётся	только при `TFA_PLAT`/`UBOOT_BL31`
U-Boot	один `UBOOT_IMAGE`	список `UBOOT_IMAGES` (BBB: `MLO` `u-boot.img`)
mkimage.sh	обязательный загрузчик с сектора 64, отдельный код для RK legacy	общий список `IMAGE_RAW` (`файл@сектор`)
mkimage.sh	dtb всегда с каталогом	`IMAGE_DTB_LAYOUT`: `tree`, `flat`, `both`
mkimage.sh	—	`IMAGE_BOOT_FILES` (`путь[:имя]`) для файлов в BOOT
Buildroot	external дерево OGA	общее `board/common/buildroot`
fstab	`/dev/mmcblk0p1 /boot` в post-build	`LABEL=BOOT` из `mkimage.sh`
Блобы	—	компонент `bootfw` (URL + sha256)

Регрессионная проверка

После такого рефакторинга нужно доказать, что OGA не сломалась. Самый строгий способ — сравнить загрузочную область нового образа с той, что **уже загрузилась на железе**:

```
cp output/oga/images/bootloader.bin /tmp/old-boot-area.bin   # сохранено до рефакторинга
make FW=oga image
dd if=output/oga/images/oga.img bs=512 skip=64 count=$((32768-64)) status=none \
  | cmp - /tmp/old-boot-area.bin && echo "boot area identical"
```

Результат — побайтовое совпадение. Изменился только rootfs (fstab и busybox), и это изменение осознанное.

BeagleBone Black

Как он грузится

ROM AM335x перебирает носители в порядке, который задают выводы SYSBOOT. У BBB по умолчанию **eMMC раньше SD**. Кнопка S2 (у слота SD) при включении меняет порядок: SD первой.

ROM ищет на карте **активный FAT-раздел** и файл `MLO` в его корне — это U-Boot SPL с заголовком TI. SPL инициализирует DDR и загружает с того же раздела `u-boot.img`. Есть и «сырой» режим (MLO по смещению 128 КиБ), но файловый проще и отлично ложится в нашу разметку.

Проверка в собранном U-Boot:

```
CONFIG_SPL_FS_FAT=y
CONFIG_SYS_MMCSD_FS_BOOT=y
CONFIG_SPL_FS_LOAD_PAYLOAD_NAME="u-boot.img"
CONFIG_CMD_BOOTZ=y                ← нужен для zImage
```

Выбор dtb

U-Boot читает EEPROM платы и в скрипте `findfdt` (`include/configs/am335x_evm.h`) ставит `fdtfile=am335x-boneblack.dtb` — **без каталога**. Поэтому `IMAGE_DTB_LAYOUT := flat`.

Конфиг

configs/bbb_defconfig

```
# Firmware: BeagleBone Black (TI AM335x, Cortex-A8).
# The ROM loads MLO (U-Boot SPL) from the first active FAT partition, SPL
# loads u-boot.img from it, and U-Boot picks the dtb from the board EEPROM
# (fdtfile=am335x-boneblack.dtb, without a vendor dir).
# The BBB boots its eMMC first: hold S2 (the button next to the SD slot)
# at power-on to boot from SD, or wipe the bootloader on the eMMC.
#
# See configs/oga_defconfig for the meaning of the variables.

BOARD_DIR     := board/bbb
CROSS_COMPILE := arm-linux-gnueabihf-

# --- U-Boot ---
UBOOT_SRC       := src/u-boot
UBOOT_DEFCONFIG := am335x_evm_defconfig
UBOOT_IMAGES    := MLO u-boot.img

# --- Linux ---
LINUX_SRC       := src/linux
LINUX_ARCH      := arm
LINUX_IMAGE     := zImage
LINUX_DEFCONFIG := omap2plus_defconfig
LINUX_FRAGMENTS := $(BOARD_DIR)/linux/bbb.config
LINUX_DTBS      := ti/omap/am335x-boneblack.dtb \
                   ti/omap/am335x-boneblack-wireless.dtb \
                   ti/omap/am335x-bonegreen.dtb \
                   ti/omap/am335x-bonegreen-wireless.dtb
LINUX_CMDLINE   := rootwait rw console=tty1 console=ttyS0,115200n8

# --- Root filesystem (Buildroot) ---
BUILDROOT_SRC       := src/buildroot
BUILDROOT_EXTERNAL  := board/common/buildroot
BUILDROOT_DEFCONFIG := $(BOARD_DIR)/buildroot/defconfig

# --- SD card image ---
```

```
# Layout: [MBR][p1 FAT32 BOOT @16M: MLO, u-boot.img, zImage, dtbs][p2 ext4 rootfs]
IMAGE_NAME       := bbb
IMAGE_BOOT_FILES := $(IMAGES_DIR)/MLO $(IMAGES_DIR)/u-boot.img
IMAGE_DTB_LAYOUT := flat
IMAGE_BOOT_SIZE_MB   := 64
IMAGE_ROOTFS_FREE_MB := 256
IMAGE_DISK_ID        := 0x42424231
```

Ядро — `omap2plus_defconfig` (в нём уже есть всё для AM335x: CPSW Ethernet, MMC, HDMI через tilcdc + tda998x модулями), фрагмент добавляет только `LOCALVERSION`. Консоль — `ttyS0` (разъём J1).

Raspberry Pi: другая модель загрузки

У Raspberry Pi загрузку начинает **GPU**, а не ARM:

	Pi 1 / Zero	Pi 4 / 400
Первая стадия	ROM GPU читает `bootcode.bin` с FAT	загрузчик в EEPROM на плате
Прошивка GPU	`start.elf` + `fixup.dat`	`start4.elf` + `fixup4.dat`
Конфиг	`config.txt`	`config.txt`
Что запускается на ARM	`kernel=` из config.txt	`kernel=` из config.txt

Сырые сектора не используются вообще: всё — файлы на FAT. Прошивка GPU закрытая, её берём с `github.com/raspberrypi/firmware` (тег `1.20260915`) и проверяем по sha256:

board/common/rpi-firmware.sha256

```
4245ec23b58158dc3e55f3e065eb4ecc0c0705d76c8db506e9f824941feee32e  bootcode.bin
32f990801e7765bae6fe5cb940ba5eeff02dd0dac8dc5474db5b3fd345281790  start.elf
7853f2b51e4489f8b3ce6f9c52fd0afdda38c7383961524e0883d34afcb4827b  fixup.dat
560ead37044d849325755d3684ac1f74654b3ce6450604a42df6dd832cda9ede  start4.elf
d181d5055daebac22513d6904fa9ce21d1e99b852cc391af095ade6f0b45c400  fixup4.dat
c7283ff51f863d93a275c66e3b4cb08021a5dd4d8c1e7acc47d872fbe52d3d6b  LICENCE.broadcom
```

scripts/fetch-files.sh

```
#!/bin/bash
# Download files from <base url>/<file> into a cache dir, check them against
# a sha256sum-format hash file and copy them to <out dir>.
#
# Usage: fetch-files.sh <base url> <cache dir> <hash file> <out dir> <file>...
set -euo pipefail

URL=$1 CACHE=$2 HASHES=$3 OUT=$4
shift 4

die() { echo "fetch-files: $*" >&2; exit 1; }
[ -f "$HASHES" ] || die "hash file $HASHES not found"

mkdir -p "$CACHE" "$OUT"
for f in "$@"; do
    want=$(awk -v f="$f" '$2 == f { print $1 }' "$HASHES")
    [ -n "$want" ] || die "no hash for $f in $HASHES"
    if [ ! -f "$CACHE/$f" ] || [ "$(sha256sum "$CACHE/$f" | cut -d' ' -f1)" != "$want" ]; the
n
        echo ">>> Downloading $URL/$f"
        wget -q -O "$CACHE/$f.tmp" "$URL/$f" || die "download of $URL/$f failed"
        got=$(sha256sum "$CACHE/$f.tmp" | cut -d' ' -f1)
        [ "$got" = "$want" ] || { rm -f "$CACHE/$f.tmp"; die "$f: sha256 $got, expected
$want"; }
        mv "$CACHE/$f.tmp" "$CACHE/$f"
```

```
    fi
    cp "$CACHE/$f" "$OUT/$f"
done
```

Хеши посчитаны один раз при добавлении платы. Если файл в кэше испорчен или upstream подменил его по тому же тегу, сборка остановится.

U-Boot вместо прямой загрузки ядра

Прошивка умеет грузить ядро сама (`kernel=zImage`, `cmdline.txt`). Но мы запускаем mainline U-Boot (`kernel=u-boot.bin`), чтобы цепочка была одинаковой на всех платах: U-Boot выбирает dtb по модели и грузит `extlinux.conf`.

board/rpi4/boot/config.txt

```
# Raspberry Pi firmware: load mainline U-Boot (64-bit), which boots extlinux.conf
arm_64bit=1
kernel=u-boot.bin
# Fixed core clock for the mini UART on GPIO14/15 (console ttyS1)
enable_uart=1
```

Ловушка: U-Boot на Pi не стартует без device tree от прошивки

В `board/raspberrypi/rpi/rpi.c`:

```
int board_fdt_blob_setup(void **fdtp)
{
    if (fdt_magic(fw_dtb_pointer) != FDT_MAGIC)
        return -ENXIO;
    *fdtp = (void *)fw_dtb_pointer;
    return 0;
}
```

А в `lib/fdtdec.c` ошибку игнорируют, только если это `-EEXIST`:

```
ret = board_fdt_blob_setup(&blob);
if (ret) {
    if (ret != -EEXIST)
        return ret;          /* -ENXIO → U-Boot не стартует */
```

То есть если прошивка GPU **не передала** U-Boot device tree, U-Boot остановится на самом раннем этапе. Прошивка же передаёт dtb, только если найдёт его в корне FAT **под своим именем**:

Плата	Имя, которое ищет прошивка	Имя в mainline
Pi 4 B / Pi 400	`bcm2711-rpi-4-b.dtb`, `bcm2711-rpi-400.dtb`	то же
Pi 1 B rev 2, A	`bcm2708-rpi-b.dtb`	`bcm2835-rpi-b-rev2.dtb`
Pi 1 B+, A+	`bcm2708-rpi-b-plus.dtb`	`bcm2835-rpi-b-plus.dtb`
Zero / Zero W	`bcm2708-rpi-zero(-w).dtb`	`bcm2835-rpi-zero(-w).dtb`

Отсюда два разных решения:

- **Pi 4/400:** U-Boot для arm64 строит `fdtfile` с каталогом (`broadcom/...`), а прошивке dtb нужен в корне. `IMAGE_DTB_LAYOUT := both` кладёт и туда, и туда.

- **Pi 1:** U-Boot для arm32 строит `fdtfile` без каталога (`flat`), а для прошивки mainline-dtb копируются ещё и под «старыми» именами через `IMAGE_BOOT_FILES`:

configs/rpi1_defconfig, строки 42–52

```
IMAGE_NAME := rpi1
RPI1_DTBS  := $(LINUX_INSTALL)/dtbs/broadcom/bcm2835-rpi
IMAGE_BOOT_FILES := $(addprefix $(IMAGES_DIR)/,$(BOOTFW_FILES) u-boot.bin) \
                    $(BOARD_DIR)/boot/config.txt \
                    $(RPI1_DTBS)-b-rev2.dtb:bcm2708-rpi-b.dtb \
                    $(RPI1_DTBS)-b.dtb:bcm2708-rpi-b-rev1.dtb \
                    $(RPI1_DTBS)-b-plus.dtb:bcm2708-rpi-b-plus.dtb \
                    $(RPI1_DTBS)-zero.dtb:bcm2708-rpi-zero.dtb \
                    $(RPI1_DTBS)-zero-w.dtb:bcm2708-rpi-zero-w.dtb \
                    $(RPI1_DTBS)-cm1-io1.dtb:bcm2708-rpi-cm.dtb
IMAGE_DTB_LAYOUT := flat
```

Каталог в `fdtfile` определяет одна строка в `rpi.c`:

```
#ifdef CONFIG_ARM64
#define DTB_DIR "broadcom/"
#else
#define DTB_DIR ""
#endif
```

> **СОВЕТ**
> Узнавать такие вещи надёжнее всего из кода загрузчика, а не из чужих инструкций: `grep -n 'fdtfile\|DTB_DIR' board/<vendor>/<board>/*.c include/configs/<board>.h`.

Консоль

Плата	Консоль на GPIO14/15	Замечание
Pi 1	`ttyAMA0` (PL011)	`stdout-path = "serial0:115200n8"`
Pi 4/400	`ttyS1` (mini-UART)	`stdout-path = "serial1:115200n8"`; частота mini-UART зависит от частоты ядра GPU — `enable_uart=1` её фиксирует

У Pi 400 главная консоль — HDMI и встроенная клавиатура. Для этого во фрагменте ядра встроены HDMI (VC4), PCIe, xHCI (контроллер USB VL805 висит на PCIe) и HID:

board/rpi4/linux/rpi4.config

```
# Raspberry Pi 4 / 400: options on top of arm64 defconfig.
# HDMI console, USB keyboard (Pi 400) and SD are built in, so the console
# works before modules load.
CONFIG_LOCALVERSION="-rpi4"
# CONFIG_LOCALVERSION_AUTO is not set

# HDMI: VC4 KMS
CONFIG_DRM=y
CONFIG_DRM_VC4=y
CONFIG_SOUND=y
CONFIG_SND=y
CONFIG_SND_SOC=y
CONFIG_FRAMEBUFFER_CONSOLE=y

# USB (VL805 xHCI on PCIe) and keyboard
CONFIG_PCIE_BRCMSTB=y
CONFIG_RESET_RASPBERRYPI=y
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_PCI=y
```

```
CONFIG_USB_HID=y
CONFIG_HID_GENERIC=y

# SD card (emmc2) and Ethernet
CONFIG_MMC_SDHCI_IPROC=y
CONFIG_BCMGENET=y
CONFIG_BROADCOM_PHY=y
```

`CONFIG_SOUND=y` появился после предупреждения `kconfig-check.sh` (глава 2): без него VC4 оставался модулем.

Сборка нескольких плат

Тулчейны для arm32 нужно поставить отдельно:

```
sudo apt install gcc-arm-linux-gnueabihf
make FW=bbb uboot linux rootfs image
make FW=rpi1 uboot bootfw linux rootfs image
make FW=rpi4 uboot bootfw linux rootfs image
```

ПО ОДНОЙ

Не запускайте несколько сборок Buildroot параллельно на машине с 14–16 ГБ ОЗУ. Подробности — в главе 14.

Проверка образов

```
for f in rpi1 rpi4; do mdir -b -i output/$f/image-work/boot.vfat ::/; done
```

```
rpi1: bcm2708-rpi-b.dtb bcm2708-rpi-b-plus.dtb ... bcm2835-rpi-zero-w.dtb bootcode.bin
      config.txt extlinux/ fixup.dat LICENCE.broadcom start.elf u-boot.bin zImage
rpi4: bcm2711-rpi-400.dtb bcm2711-rpi-4-b.dtb bcm2711-rpi-cm4-io.dtb broadcom/ config.txt
      extlinux/ fixup4.dat Image LICENCE.broadcom start4.elf u-boot.bin
```

СТАТУС

На момент написания образы BBB, RPi 1 и RPi 4/400 собраны и проверены по содержимому, но на железе ещё не запускались. Первые запуски — хорошее упражнение на методику главы 11.

ГЛАВА 14

Ошибки и уроки

Ошибки по дороге — нормальная часть работы. Важно, чтобы они быстро обнаруживались и не повторялись. Здесь собраны все промахи, сделанные при построении SDK, — с симптомом, причиной и уроком. Многие из них типичны и встретятся вам самим.

Нехватка памяти: OOM убил графическую сессию

Симптом. Посреди сборки трёх новых плат IDE закрылась, все фоновые сборки исчезли.

Причина. Одновременно шли три сборки Buildroot (каждая собирала GCC в 17 потоков) и сборка ядра в 16 потоков — около 70 тяжёлых процессов компилятора на машине с 14 ГБ ОЗУ. Средняя нагрузка доходила до 100. Журнал ядра:

```
kernel: systemd invoked oom-killer: ...
kernel: Out of memory: Killed process 5485 (gvfsd-metadata) ...
systemd[4422]: org.gnome.Shell@ubuntu.service: Failed with result 'oom-kill'.
```

OOM killer убил GNOME Shell, с ним закрылись сеанс, IDE и все дочерние процессы.

Как найти. `journalctl -k --since "20:50" | grep -i -E 'out of memory|oom'` и `uptime` (средняя нагрузка за 5 и 15 минут).

Урок. Параллельность должна учитывать память, а не только ядра. GCC при сборке самого себя может требовать до 1 ГБ на процесс. На 14 ГБ — одна сборка Buildroot за раз. Make и Buildroot продолжают с места остановки, поэтому повторный запуск ничего не потерял.

`pkill -f` убил собственную команду

Симптом. Команда остановить медленный `git clone` и подставить зеркало завершилась с кодом 144, половина действий не выполнилась.

Причина. `pkill -f 'git clone https://source.denx.de/...'` ищет шаблон в **полной командной строке** процессов. Строка оболочки, которая выполняла `pkill`, тоже содержала этот текст — `pkill` убил свою же оболочку.

Урок. Убивать по PID (`pgrep -a`, затем `kill <pid>`) или делать шаблон, который не совпадёт с вашей командой (`pkill -f '[g]it clone'` — классический трюк).

`cd` в несуществующий каталог

Симптом. Файлы BR2_EXTERNAL оказались в `src/`, а не в `board/oga/buildroot/`.

Причина. `cd board/oga/buildroot && ...` — каталога ещё не было. `cd` упал, но следующие команды были через `;`, а не через `&&`, и выполнились в текущем каталоге.

Урок. Сначала `mkdir -p`, потом `cd`. Связывайте команды через `&&` или включайте `set -e`. После массового создания файлов — `find` и проверить, где они оказались.

`unexport FW` создал пустую переменную

Симптом. `make info` после `make oga_defconfig` говорил «No firmware config selected».

Причина. `unexport FW` перед `FW ?= $(shell cat ...)` определяет переменную `FW` пустой; `?=` после этого ничего не присваивает.

Как найти. `make -p info | grep -B2 '^FW :*='` — make сам показывает, в какой строке определена переменная.

Урок. `make -p` (база данных переменных и правил) — главный инструмент отладки Makefile.

`git mv -k` молча ничего не сделал

Симптом. `rmdir blobs` — «Directory not empty», файл остался на месте.

Причина. Файл ещё не был добавлен в git. `git mv` такие файлы не трогает, а `-k` подавляет ошибку. Код возврата — 0, поэтому запасной `|| mv ...` тоже не сработал.

Урок. Ключи вида «игнорировать ошибки» прячут проблему. Для неотслеживаемых файлов — обычный `mv`.

`[ ... ] && x` в конце цикла под `set -e`

Симптом. Потенциальный: `mkimage.sh` мог молча завершиться при пустом `IMAGE_RAW`.

Причина. Код возврата цикла `while` — код возврата его последней команды. Если последняя команда — `[ -n "$x" ] && put ...` и условие ложно, цикл вернёт 1, и `set -e` завершит скрипт.

Урок. Под `set -e` используйте `if ...; then ...; fi` вместо `cond && action`, особенно в конце функций и циклов.

Забытый `chmod +x`

Симптом. `make: scripts/fetch-files.sh: Permission denied`, цепочка сборки RPi остановилась.

Урок. Новые скрипты сразу делать исполняемыми. Git хранит бит исполнения, так что после `git add` его будет видно в `git diff` как `new file mode 100755`.

Опции Kconfig, которые не включились

Два случая — панель OGA (`BACKLIGHT_CLASS_DEVICE=m`) и VC4 на Pi 4 (`SOUND=m`) — разобраны в главах 2, 8 и 13. Урок: **всегда проверять итоговый `.config`** против фрагмента. Для этого в SDK есть `kconfig-check.sh`.

Параметр ядра, который ни на что не влиял

Симптом. Пользователь хотел перевернуть экран OGA, поменял `fbcon=rotate:3` на `fbcon=rotate:0`, пересобрал, прошил — ничего не изменилось.

Причина. `CONFIG_FRAMEBUFFER_CONSOLE_ROTATION` в arm64 `defconfig` выключен, и без него ядро игнорирует `fbcon=rotate:` целиком. Параметр был скопирован из командной строки Hardkernel без проверки, что поддержка собрана. Консоль всё время была в родной ориентации панели; заметили это только когда попытались её поменять.

Как найти. Убедиться, что изменение дошло до карты (`extlinux.conf` в образе), а затем найти, кто читает параметр: `grep -n initial_rotation drivers/video/fbdev/core/fbcon.c` — код под `#ifdef CONFIG_FRAMEBUFFER_CONSOLE_ROTATION`.

Попутно выяснилось, что подсказка из DTS (`rotation = <270>` → `rotate:1`) даёт перевёрнутую картинку, а правильное значение — исходное `rotate:3`.

Урок. Параметр командной строки — это тоже код, который может быть не собран. Для каждого нестандартного параметра проверяйте, какая опция Kconfig его включает. И проверяйте, что изменение **видно** на железе, а не только что оно есть в конфиге.

Файл, удалённый из overlay, остался в rootfs

Симптом. В rootfs два файла: `profile.d/oga.sh` и `profile.d/sdk.sh`.

Причина. Buildroot копирует overlay в `target/` и никогда не удаляет из `target/`.

Урок. После переименований и удалений в overlay — `rootfs-clean` или ручное удаление из `output/<fw>/buildroot/target/`.

Захват UART мешал работать с консолью

Симптом. После загрузки нельзя было ввести команду в консоли UART.

Причина. Фоновый `cat /dev/ttyUSB0` продолжал читать порт. Два процесса, читающие один tty, делят входящие данные между собой случайным образом.

Урок. Захват — с ограничением по времени (`timeout`), и останавливать его, когда он больше не нужен.

Неполная первая гипотеза

Симптом. Раскладка карты была исправлена, а образ всё равно не грузился.

Причина. Гипотеза «дело в смещениях» объясняла прошлые неудачи, но не учитывала загрузчик во SPI.

Урок. Проверяйте гипотезу, которая объясняет **все** наблюдения. Мусор на 1,5 Мбод в первом же захвате не объяснялся «неправильными смещениями» — это и был ключ.

Общие выводы

1. **Проверка после каждого шага.** `cmp`, `dumpimage`, `kconfig-check`, `make info` — дешёвые проверки экономят часы.
2. **Код вместо догадок.** Скорость консоли, имена dtb, смещения — всё есть в исходниках.
3. **Логи — в файлы.** Сборки — в `logs/*.log`, UART — в файл с момента включения.
4. **Ресурсы.** Перед долгой сборкой — `df -h` и `free -h`.
5. **Осторожность с необратимым.** Запись на карту — с проверками; изменения SPI — только с бэкапом; убийство процессов — по PID.

ГЛАВА 15

Git: сабмодули, форки и обновления

Как устроен репозиторий SDK

Верхний репозиторий (`linux-sdk`) хранит Makefile, конфиги, скрипты, файлы плат — и **ссылки** на четыре сабмодуля. Сабмодуль в верхнем репозитории — это одна запись: путь + хеш коммита. URL лежит в `.gitmodules`.

```
$ git submodule status
 d518030...  src/buildroot (2026.08)
 5fce161...  src/linux (v7.2.9)
 da738d5...  src/tf-a (v2.15.0)
 ece349a...  src/u-boot (v2026.07)
```

Склонировать SDK со всем содержимым:

```
git clone --recurse-submodules --shallow-submodules git@github.com:sobs1992/linux-sdk.git
```

`--shallow-submodules` берёт только нужный коммит каждого сабмодуля без истории — это экономит гигабайты на ядре. Если позже понадобится история: `git -C src/linux fetch --unshallow`.

Вносить изменения в компонент

Допустим, нужен патч драйвера панели.

```
cd src/linux
git switch oga                        # работаем в своей ветке
$EDITOR drivers/gpu/drm/panel/panel-elida-kd35t133.c
git commit -am "drm/panel: kd35t133: ..."
cd ../..
make linux image                      # SDK собирает ваш код
```

Пока коммит есть только у вас локально. Чтобы SDK мог на него ссылаться из любого клона, его нужно опубликовать в **свой форк**:

```
# один раз: форк на GitHub (kernel.org не позволяет push, поэтому форк обязателен)
cd src/linux
git remote add mine git@github.com:<you>/linux.git
git push mine oga

cd ../..
git config -f .gitmodules submodule.src/linux.url git@github.com:<you>/linux.git
git submodule sync src/linux
git add .gitmodules src/linux
git commit -m "linux: kd35t133 fix"
```

Теперь `src/linux` в SDK указывает на ваш коммит в вашем форке.

> **ЛОВУШКА: КОММИТ В САБМОДУЛЕ БЕЗ PUSH**
> Если закоммитить новый хеш сабмодуля в SDK, но не запушить сам коммит в форк, у всех, кто склонирует SDK, `git submodule update` упадёт с «reference is not a tree». Порядок всегда: push сабмодуля → commit/push SDK.

Обновить версию компонента

Например, перейти на U-Boot v2026.10, когда он выйдет:

```
cd src/u-boot
git fetch origin tag v2026.10 --no-tags
git switch oga
git rebase v2026.10                  # ваши патчи — поверх нового релиза
cd ../..
make FW=oga uboot-clean uboot        # чистая сборка: конфиг мог измениться
make image
# проверить на железе, затем:
git add src/u-boot && git commit -m "u-boot: v2026.10"
```

Что проверить после обновления:

- `make uboot` не показывает новых предупреждений `kconfig-check.sh` (фрагмент мог перестать применяться, если опцию переименовали);
- для OGA не изменились значения `CONFIG_SPL_PAD_TO` и `CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR` (глава 12);
- `dumpimage -l u-boot.itb` показывает BL31 и все dtb.

Что хранить в SDK, а что — в компоненте

Изменение	Где хранить
Опции ядра/U-Boot для платы	фрагмент в `board/<плата>/`
Новый драйвер, исправление ошибки	коммит в сабмодуле (ветка `oga`), в идеале — отправить upstream
Новый DTS платы	коммит в сабмодуле
Пакет для rootfs	`board/common/buildroot/package/`
Файлы в rootfs	`board/common/buildroot/rootfs-overlay/`
Разметка карты, версия прошивки RPi	конфиг прошивки `configs/<fw>_defconfig`

Чем меньше изменений в сабмодулях, тем проще обновляться. Если изменение полезно другим — отправьте его upstream (`scripts/get_maintainer.pl` в ядре и U-Boot подскажет, кому писать).

.gitignore

```
/output/
/dl/
/logs/
```

Результаты сборки, кэш загрузок и логи в репозиторий не попадают. Бинарники, которые всё же лежат в репозитории (`board/oga/rkbin/`), лежат вместе со своей лицензией: лицензия Rockchip разрешает распространение без изменения уведомлений. Прошивку Raspberry Pi в репозиторий не кладём: хранятся только её sha256, а сами файлы скачивает `make bootfw`.

ГЛАВА 16

Своя плата: пошаговый план и упражнения

Эта глава — рабочий план. Пройдите его для любой новой платы: Orange Pi, Rock Pi, Pine64, своей разработки на известном SoC.

Шаг 1. Собрать факты

Ответьте на вопросы, **по возможности из исходников**, а не из форумов.

Вопрос	Где искать
Есть ли плата в mainline U-Boot?	`ls src/u-boot/configs | grep -i <плата>`, `doc/board/<vendor>/`
Как BootROM находит загрузчик и в каком порядке носителей?	документация SoC, `doc/board/<vendor>/*.rst` в U-Boot
Нужен ли TF-A / BL31?	`ls src/tf-a/plat/<vendor>/`, `CONFIG_SPL_ATF` в defconfig U-Boot
Какие файлы выдаёт сборка U-Boot?	`ls output/<fw>/u-boot/` после первой сборки, `binman` в `*-u-boot.dtsi`
Куда писать загрузчик (сектора или файлы на FAT)?	`CONFIG_SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR`, `SPL_FS_*`, документация
Как U-Boot строит `fdtfile`?	`grep -rn fdtfile board/<vendor>/ include/configs/<плата>.h`
Где DTS в ядре?	`ls src/linux/arch/arm*/boot/dts/<vendor>/ | grep <плата>`
Какая консоль и скорость?	`stdout-path` в DTS, `CONFIG_BAUDRATE` в U-Boot
Какой defconfig ядра взять?	`ls arch/arm*/configs/`, проверить `CONFIG_ARCH_<SOC>=y`
Что-то уже стоит во флеше / eMMC?	осмотреть плату, вендорские скрипты, тест «без SD»
Есть ли закрытые блобы (DDR, прошивка GPU)?	документация, `binman` (`rockchip-tpl`, `ddr-*`)

Шаг 2. Создать файлы платы

```
configs/<fw>_defconfig
board/<плата>/linux/<плата>.config       # хотя бы LOCALVERSION
board/<плата>/u-boot/<плата>.config      # при необходимости
board/<плата>/buildroot/defconfig        # скопируйте ближайший и поменяйте CPU и getty
board/<плата>/boot/...                   # если загрузчику нужны файлы (config.txt и т. п.)
```

Шаблон конфига прошивки:

```
BOARD_DIR     := board/myboard
CROSS_COMPILE := aarch64-linux-gnu-
```

```
# TFA_SRC  := src/tf-a          # если нужен BL31
# TFA_PLAT := <plat>

UBOOT_SRC       := src/u-boot
UBOOT_DEFCONFIG := myboard_defconfig
UBOOT_IMAGES    := u-boot-xxx.bin
# UBOOT_BL31    := tf-a

LINUX_SRC       := src/linux
LINUX_ARCH      := arm64
LINUX_IMAGE     := Image
LINUX_DEFCONFIG := defconfig
LINUX_FRAGMENTS := $(BOARD_DIR)/linux/myboard.config
LINUX_DTBS      := vendor/myboard.dtb
LINUX_CMDLINE   := rootwait rw console=tty1 console=ttyS0,115200n8

BUILDROOT_SRC       := src/buildroot
BUILDROOT_EXTERNAL  := board/common/buildroot
BUILDROOT_DEFCONFIG := $(BOARD_DIR)/buildroot/defconfig

IMAGE_NAME           := myboard
IMAGE_RAW            := $(IMAGES_DIR)/u-boot-xxx.bin@<сектор>
IMAGE_BOOT_FILES     :=
IMAGE_DTB_LAYOUT     := tree
IMAGE_BOOT_SIZE_MB   := 128
IMAGE_ROOTFS_FREE_MB := 256
IMAGE_DISK_ID        := 0x<8 hex>
```

Шаг 3. Собирать по частям и проверять каждую

```
make myboard_defconfig
make info                     # пути и версии верные?
make uboot                    # нет ли предупреждений binman о missing blobs?
make linux                    # нет ли WARNING от kconfig-check?
make rootfs                   # долго: тулчейн
make image                    # не перекрываются ли блобы?
```

После каждой команды — проверки из глав 7, 8 и 10: `dumpimage`, `grep` по `.config`, `mdir`, `debugfs`, `cmp` по секторам.

Шаг 4. Первый запуск

1. UART подключён, скорость верная, захват в файл запущен **до** включения.
2. Записали образ (`make flash`), вставили, включили.
3. Смотрим, до какой стадии дошли: ничего / мусор / SPL / U-Boot / ядро / init.
4. Если ничего или мусор — методика главы 11.

Где остановилось	Что проверить
Тишина	скорость UART, правильные ли TX/RX, тест «без SD», не грузится ли что-то из флеша/eMMC
Мусор	скорость; кто печатает (наш код или чужой загрузчик)
SPL, но не U-Boot	сектор FIT (`SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR`), есть ли BL31 в FIT
U-Boot, но не ядро	`fdtfile` и раскладка dtb, `bootflow scan -l` , есть ли `bootz` для zImage
Ядро, но паника на root	`root=PARTUUID` , драйвер SD встроен (`=y`)?, `rootwait`
init, но нет консоли	getty на правильном порту, `console=` последним указан UART

ПОЛЕЗНЫЕ КОМАНДЫ В КОНСОЛИ U-BOOT

`bootflow scan -l` — что U-Boot нашёл для загрузки; `printenv fdtfile` — какой dtb он выберет; `mmc list` , `ls mmc 0:1 /` — видит ли он карту и файлы; `bdinfo` — память; `dm tree` — устройства.

Упражнения

1. МИНИМАЛЬНОЕ ЯДРО ДЛЯ OGA

Глава 8: `localmodconfig` , свой defconfig-файл, сравнение размеров.

2. ЦЕЛЬ IMAGE-XZ

Глава 6: сжатие образа отдельной целью без лишней работы.

3. СВОЙ ПАКЕТ BUILDROOT

Глава 9: `hello` как `generic-package` в общем external-дереве.

4. USB-ГАДЖЕТ ETHERNET НА OGA

У OGA есть USB OTG (DWC2). Включите в ядре `CONFIG_USB_CONFIGFS_ECM` (или `g_ether`), добавьте в overlay скрипт `/etc/init.d/S40usbnet` , который настраивает гаджет и адрес `192.168.7.2` , и включите dropbear. Цель — `ssh root@192.168.7.2` по кабелю USB.

5. ЗАПУСК BBB ИЛИ RASPBERRY PI НА ЖЕЛЕЗЕ

Пройдите шаг 4 этой главы для одной из новых плат. Запишите, на какой стадии что увидели, и что пришлось исправить.

6. ПЕРЕЙТИ НА СВЕЖИЙ U-BOOT

Глава 15: rebase ветки `oga` на новый релиз, проверка `SPL_PAD_TO` и `SYS_MMCSD_RAW_MODE_U_BOOT_SECTOR` , проверка на OGA.

7. ВНЕШНИЙ ТУЛЧЕЙН

Сделайте копию конфига прошивки `rpi4` с Buildroot-тулчейном Bootlin (`BR2_TOOLCHAIN_EXTERNAL=y`, `BR2_TOOLCHAIN_EXTERNAL_BOOTLIN=y`, `BR2_TOOLCHAIN_EXTERNAL_BOOTLIN_AARCH64_GLIBC_STABLE=y`). Сравните время сборки и размер `output/<fw>/buildroot`.

8. БЕЗ SPI-ЗАГРУЗЧИКА

(Только если готовы к риску и есть бэкап.) Сохраните SPI OGA, сотрите заголовок (глава 11) и проверьте, что тот же образ грузится по полностью открытой цепочке: в логе должен появиться `out` от TPL и `U-Boot SPL 2026.07`.

ПРИЛОЖЕНИЕ А

Шпаргалка команд

SDK

```
make <fw>_defconfig          # выбрать прошивку (oga, bbb, rpi1, rpi4)
make info                    # что выбрано, версии компонентов
make                         # всё + образ
make uboot | tfa | bootfw | linux | rootfs | image
make FW=<fw> <цель>          # для другой прошивки, не меняя выбор
make <comp>-menuconfig       # comp: uboot, linux, rootfs
make <comp>-savedefconfig
make <comp>-clean
make rootfs-make BR=<цель>   # любая цель Buildroot: busybox-menuconfig, <pkg>-rebuild…
make rootfs-sdk              # тулчейн + sysroot для приложений
make flash DEV=/dev/sdX      # весь образ
make flash-boot DEV=/dev/sdX # загрузчик + раздел BOOT
```

Исследование исходников

```
git ls-remote --tags --refs <url> | awk -F/ '{print $NF}' | sort -V | tail
grep -rn 'fdtfile' board/<vendor>/ include/configs/
grep -n -A8 '^config FOO$' $(git grep -l '^config FOO$')      # зависимости опции
grep -h -o 'compatible = "[^"]*"' arch/arm64/boot/dts/<v>/<board>*.dts* | sort -u
strings -n 6 blob.img | head                                    # что внутри блоба
```

Проверка результатов

```
dumpimage -l u-boot.itb                       # содержимое FIT
loaderimage --info uboot.img                  # uboot.img Rockchip
aarch64-linux-gnu-readelf -h bl31.elf         # точка входа ELF
xxd -l 16 file                                # сигнатура
sfdisk -l image.img                           # разделы
mdir -i boot.vfat -/ ::/ ; mtype -i boot.vfat ::/path
debugfs -R 'cat /etc/fstab' rootfs.ext4
dd if=img bs=512 skip=<S> count=<N> status=none | cmp - file   # блоб на месте?
scripts/kconfig-check.sh output/<fw>/linux/.config board/<b>/linux/<b>.config
```

UART

```
lsusb -v -d <vid:pid> | grep iProduct         # модель адаптера
fuser -v /dev/ttyUSB0                         # кто держит порт
screen -ls ; screen -S <id> -X hardcopy -h out.txt
screen /dev/ttyUSB0 115200                    # интерактивно; выход Ctrl-A K
stty -F /dev/ttyUSB0 115200 raw -echo cs8 -cstopb -parenb -crtscts
timeout 180 cat /dev/ttyUSB0 > boot.log       # захват с момента включения
cat -v boot.log | head ; wc -c boot.log
```

Система

```
df -h /home ; free -h ; uptime
journalctl -k --since "HH:MM" | grep -i oom
lsblk -o NAME,SIZE,TRAN,RM,FSTYPE,LABEL,PARTUUID,MOUNTPOINT
```

U-Boot (консоль платы)

```
=> bootflow scan -l          что можно загрузить
=> printenv fdtfile          какой dtb будет выбран
=> mmc list ; ls mmc 0:1 /   карта и файлы
=> bdinfo                    память
=> sf probe ; sf read <addr> <off> <len>      SPI-флеш
=> fatwrite mmc 1:1 <addr> file <len>         сохранить память в файл на FAT
```

ПРИЛОЖЕНИЕ B

Справочник переменных конфига прошивки

Пути — относительно корня SDK или абсолютные. В значениях можно использовать `$(BOARD_DIR)`, `$(IMAGES_DIR)` (`output/<fw>/images`) и `$(LINUX_INSTALL)` (`output/<fw>/linux-install`).

Общие

Переменная	Обязательна	Значение
`BOARD_DIR`	да	каталог платы, например `board/oga`
`CROSS_COMPILE`	да	префикс тулчейна для TF-A, U-Boot, ядра

TF-A

Переменная	Обязательна	Значение
`TFA_SRC`	если нужен TF-A	`src/tf-a`
`TFA_PLAT`	если нужен TF-A	платформа TF-A, например `px30`
`TFA_MAKE_ARGS`	нет	доп. аргументы make TF-A

U-Boot

Переменная	Обязательна	Значение
`UBOOT_SRC`	да	`src/u-boot`
`UBOOT_DEFCONFIG`	да	имя in-tree defconfig или путь к файлу
`UBOOT_FRAGMENTS`	нет	список фрагментов
`UBOOT_BL31`	нет	`tf-a` или путь к `bl31.elf`
`UBOOT_IMAGES`	да	файлы из каталога сборки → `images/`
`UBOOT_MAKE_ARGS`	нет	доп. аргументы make U-Boot

Rockchip legacy (OGA)

Переменная	Значение
`RK_LEGACY`	`y` — собрать `uboot.img` и `trust.img` для SPI miniloader
`RKBIN_DIR`	каталог с `tools/loaderimage`, `tools/trust_merger`
`RK_TRUST_BL31`	BL31 для `trust.img`

Блобы прошивки

Переменная	Значение
`BOOTFW_NAME`	имя каталога в кэше `dl/`
`BOOTFW_URL`	базовый URL, к нему дописывается имя файла
`BOOTFW_FILES`	список файлов
`BOOTFW_HASH`	файл в формате `sha256sum`

Linux

Переменная	Обязательна	Значение
`LINUX_SRC`	да	`src/linux`
`LINUX_ARCH`	нет (`arm64`)	`arm` или `arm64`
`LINUX_IMAGE`	нет (`Image`)	`Image` или `zImage`
`LINUX_DEFCONFIG`	да	имя in-tree defconfig или путь
`LINUX_FRAGMENTS`	нет	список фрагментов
`LINUX_DTBS`	да	dtb относительно `arch/<arch>/boot/dts/`
`LINUX_CMDLINE`	да	командная строка; `root=PARTUUID=…` добавляется автоматически
`LINUX_MAKE_ARGS`	нет	доп. аргументы make ядра

Buildroot

Переменная	Обязательна	Значение
`BUILDROOT_SRC`	да	`src/buildroot`
`BUILDROOT_EXTERNAL`	да	`board/common/buildroot`
`BUILDROOT_DEFCONFIG`	да	путь к defconfig платы
`BUILDROOT_MAKE_ARGS`	нет	доп. аргументы make Buildroot

Образ

Переменная	Обязательна	Значение
`IMAGE_NAME`	да	имя файла `images/<name>.img`
`IMAGE_RAW`	нет	`файл@сектор …`, всё должно кончаться до 16 МиБ
`IMAGE_BOOT_FILES`	нет	`путь[:имя] …` — в корень раздела BOOT
`IMAGE_DTB_LAYOUT`	нет (`tree`)	`tree`, `flat`, `both`
`IMAGE_BOOT_SIZE_MB`	да	размер раздела BOOT
`IMAGE_ROOTFS_FREE_MB`	да	свободное место в rootfs сверх содержимого
`IMAGE_DISK_ID`	да	идентификатор MBR, `0x` + 8 hex-цифр

Переменные командной строки

Переменная	Значение
`FW`	прошивка для одной команды
`JOBS`	потоков для TF-A, U-Boot, ядра (по умолчанию `nproc`)
`DEV`	устройство для `flash`, `flash-boot`
`BR`	цель Buildroot для `rootfs-make`
`OUTPUT_DIR`, `DL_DIR`	каталоги результатов и кэша

ПРИЛОЖЕНИЕ C

Глоссарий

Термин	Значение
BL31	Runtime-монитор TF-A в EL3: PSCI, SMC-сервисы
BL32	Безопасная ОС (OP-TEE) в TrustZone, опционально
BL33	Загрузчик «нормального мира» — у нас U-Boot
binman	Инструмент U-Boot, собирающий итоговые образы по описанию в dts
BootROM	Неизменяемый код в процессоре, первая стадия загрузки
BR2_EXTERNAL	Внешнее дерево Buildroot для своих defconfig, пакетов, overlay
bootstd	Механизм U-Boot «standard boot»: сам ищет `extlinux.conf`, EFI и т. п.
defconfig	Минимальная конфигурация Kconfig (только отличия от умолчаний)
device tree (DT, dtb)	Описание железа платы для ядра и U-Boot
EL0–EL3	Уровни привилегий ARMv8 (Exception Levels)
extlinux.conf	Простой конфиг загрузки (формат syslinux), понятный U-Boot
fakeroot	Утилита, имитирующая права root для создания файловых систем
FIT (.itb)	Контейнер образов U-Boot (сам устроен как device tree)
fragment	Короткий файл с изменениями Kconfig поверх defconfig
idbloader	TPL+SPL Rockchip с заголовком, по сектору 64
Kconfig	Система конфигурации Linux, U-Boot, Buildroot
MBR	Классическая таблица разделов в секторе 0
mdev	Менеджер `/dev` из busybox, подгружает модули по modalias
miniloader	Закрытый загрузчик Rockchip из rkbin (второй этап после DDR-блоба)
MLO	U-Boot SPL для TI AM335x с заголовком, грузится ROM с FAT
OOM killer	Механизм ядра, убивающий процессы при нехватке памяти
out-of-tree	Сборка с `O=`, при которой исходники не меняются
PARTUUID	Идентификатор раздела; для MBR — disk id + номер раздела
PSCI	Интерфейс управления питанием CPU через BL31
rkbin	Набор бинарных блобов и утилит Rockchip
SPL	Урезанный U-Boot, инициализирует DRAM и грузит основной U-Boot
TF-A	Trusted Firmware-A, эталонная прошивка EL3 для ARMv8
TPL	Ещё более ранняя стадия, чем SPL; на Rockchip инициализирует DDR
trust.img	Контейнер BL31/BL32 в формате Rockchip (`BL3X`)
uboot.img	U-Boot в формате Rockchip (`LOADER`), грузится miniloader
UART	Последовательный порт, консоль загрузки

ПРИЛОЖЕНИЕ D

Полные листинги

Листинги взяты из репозитория в момент сборки книги.

Makefile

Makefile

```
# OdroidLinux SDK: top-level build orchestrator.
#
# Every component (TF-A, U-Boot, Linux, Buildroot) lives in its own git
# repository under src/ and is built out-of-tree into output/<fw>/.
# A firmware config (configs/<fw>_defconfig) selects component configs and
# the SD card layout; "make image" glues the built components into an .img.
#
#   make oga_defconfig      select firmware config
#   make                    build everything and assemble the image
#   make help               list all targets

SDK_DIR    := $(patsubst %/,%,$(dir $(abspath $(lastword $(MAKEFILE_LIST)))))
OUTPUT_DIR ?= $(SDK_DIR)/output
DL_DIR     ?= $(SDK_DIR)/dl
CROSS_COMPILE ?= aarch64-linux-gnu-
JOBS       ?= $(shell nproc)

# Command-line variables of this Makefile must not leak into component builds
MAKEOVERRIDES :=

.DEFAULT_GOAL := all

# ------------------------------------------------------------------------------
# Firmware config selection

FW ?= $(shell cat $(OUTPUT_DIR)/.fw 2>/dev/null)

# Output locations, usable in firmware configs
OUT           := $(OUTPUT_DIR)/$(FW)
IMAGES_DIR    := $(OUT)/images
LINUX_INSTALL := $(OUT)/linux-install

ifeq ($(filter $(notdir $(wildcard $(SDK_DIR)/configs/*_defconfig)),$(MAKECMDGOALS)),)
ifneq ($(FW),)
FW_CONFIG := $(SDK_DIR)/configs/$(FW)_defconfig
ifeq ($(wildcard $(FW_CONFIG)),)
$(error Firmware config $(FW_CONFIG) not found)
endif
include $(FW_CONFIG)
endif
endif

FW_DEFCONFIGS := $(notdir $(wildcard $(SDK_DIR)/configs/*_defconfig))
.PHONY: $(FW_DEFCONFIGS)
$(FW_DEFCONFIGS): %_defconfig:
    @mkdir -p $(OUTPUT_DIR)
    @echo $* > $(OUTPUT_DIR)/.fw
    @echo "Firmware config: $* (output: $(OUTPUT_DIR)/$*)"

# Absolute path for a path relative to the SDK root
sdkpath = $(if $(filter /%,$(1)),$(1),$(SDK_DIR)/$(1))
# In-tree defconfig names stay as they are, paths become absolute
cfgpath = $(if $(findstring /,$(1)),$(call sdkpath,$(1)),$(1))
# "path<sep>suffix" -> "/abs/path<sep>suffix" ($(1)=list $(2)=separator)
sdkpath-pairs = $(foreach e,$(1),$(call sdkpath,$(firstword $(subst $(2), ,$(e))))$(if $(find
string $(2),$(e)),$(2)$(lastword $(subst $(2), ,$(e)))))

LINUX_ARCH  ?= arm64
```

```
LINUX_IMAGE ?= Image

TFA_SRC    := $(call sdkpath,$(TFA_SRC))
UBOOT_SRC  := $(call sdkpath,$(UBOOT_SRC))
LINUX_SRC  := $(call sdkpath,$(LINUX_SRC))
BUILDROOT_SRC      := $(call sdkpath,$(BUILDROOT_SRC))
BUILDROOT_EXTERNAL := $(call sdkpath,$(BUILDROOT_EXTERNAL))

UBOOT_DEFCONFIG     := $(call cfgpath,$(UBOOT_DEFCONFIG))
LINUX_DEFCONFIG     := $(call cfgpath,$(LINUX_DEFCONFIG))
BUILDROOT_DEFCONFIG := $(call cfgpath,$(BUILDROOT_DEFCONFIG))
UBOOT_FRAGMENTS     := $(foreach f,$(UBOOT_FRAGMENTS),$(call sdkpath,$(f)))
LINUX_FRAGMENTS     := $(foreach f,$(LINUX_FRAGMENTS),$(call sdkpath,$(f)))

# ------------------------------------------------------------------------------
# Kconfig helpers (U-Boot and Linux)
#
# $(OBJ)/.config is regenerated when its inputs change: the defconfig file,
# a fragment, or the list of inputs itself (stored in .sdk-inputs). Changes
# made with *-menuconfig are kept until then; use *-savedefconfig to keep
# them for good.

# $(1)=objdir $(2)=inputs: rewrite the stamp only when the inputs differ
define inputs-stamp
$(shell mkdir -p $(1) && { test "$$(cat $(1)/.sdk-inputs 2>/dev/null)" = "$(strip $(2))" || e
cho "$(strip $(2))" > $(1)/.sdk-inputs; })
endef

# $(1)=make command $(2)=objdir $(3)=defconfig $(4)=fragments $(5)=src
define kconfig-configure
    @echo ">>> Configuring $(2)"
    $(if $(findstring /,$(3)),cp $(3) $(2)/.config && $(1) olddefconfig,$(1) $(3))
    $(if $(strip $(4)),$(5)/scripts/kconfig/merge_config.sh -m -O $(2) $(2)/.config $(4) && $
(1) olddefconfig && $(SDK_DIR)/scripts/kconfig-check.sh $(2)/.config $(4))
endef

# $(1)=make command $(2)=objdir $(3)=defconfig
define kconfig-savedefconfig
    $(1) savedefconfig
    @if [ -n "$(findstring /,$(3))" ]; then cp $(2)/defconfig $(3) && echo "Saved to
$(3)"; else \
        echo "Saved to $(2)/defconfig. The firmware config uses the in-tree '$(3)':"; \
        echo "move the changes to a fragment, or point the *_DEFCONFIG at a file."; fi
endef

.PHONY: check-fw check-cc
check-fw:
    @test -n "$(FW)" || { echo "No firmware config selected: run 'make <name>_defconfig'."; \
        echo "Available:"; ls $(SDK_DIR)/configs | sed -n 's/_defconfig$$//p' | sed 's/
^/  /'; exit 1; }

# Cross compiler for TF-A, U-Boot and Linux (Buildroot builds its own)
check-cc: check-fw
    @command -v $(CROSS_COMPILE)gcc >/dev/null || { echo "$(CROSS_COMPILE)gcc not found."; \
        echo "Install it (e.g. 'sudo apt install gcc-$(patsubst %-,%,$(CROSS_COMPILE))') or
pass CROSS_COMPILE=..."; exit 1; }

# ------------------------------------------------------------------------------
# Trusted Firmware-A (only for boards with TFA_PLAT)

TFA_OUT  := $(OUT)/tf-a
TFA_BL31 := $(TFA_OUT)/$(TFA_PLAT)/release/bl31/bl31.elf
TFA_MAKE  = $(MAKE) -C $(TFA_SRC) BUILD_BASE=$(TFA_OUT) PLAT=$(TFA_PLAT) \
            CROSS_COMPILE=$(CROSS_COMPILE) $(TFA_MAKE_ARGS)

.PHONY: tfa tfa-clean
tfa: check-cc
    @test -n "$(TFA_PLAT)" || { echo "$(FW) does not use TF-A (TFA_PLAT is not set)"; exit
1; }
    $(TFA_MAKE) -j$(JOBS) bl31

tfa-clean: check-fw
```

```
    rm -rf $(TFA_OUT)

# ------------------------------------------------------------------------
# U-Boot

UBOOT_OUT := $(OUT)/u-boot
UBOOT_MAKE = $(MAKE) -C $(UBOOT_SRC) O=$(UBOOT_OUT) CROSS_COMPILE=$(CROSS_COMPILE) $(UBOOT_MA
KE_ARGS)

ifeq ($(UBOOT_BL31),tf-a)
UBOOT_BL31_FILE := $(TFA_BL31)
UBOOT_DEPS := tfa
else ifneq ($(UBOOT_BL31),)
UBOOT_BL31_FILE := $(call sdkpath,$(UBOOT_BL31))
endif

# Rockchip legacy uboot.img/trust.img for the SPI miniloader
ifeq ($(RK_LEGACY),y)
RKBIN_DIR     := $(call sdkpath,$(RKBIN_DIR))
RK_TRUST_BL31 := $(call sdkpath,$(RK_TRUST_BL31))
RK_LEGACY_IMAGES := $(IMAGES_DIR)/uboot.img $(IMAGES_DIR)/trust.img
endif

ifneq ($(FW),)
$(call inputs-stamp,$(UBOOT_OUT),$(UBOOT_DEFCONFIG) $(UBOOT_FRAGMENTS))
endif

$(UBOOT_OUT)/.config: $(UBOOT_OUT)/.sdk-inputs $(filter /%,$(UBOOT_DEFCONFIG)) $(UBOOT_FRAGME
NTS)
    $(call kconfig-configure,$(UBOOT_MAKE),$(UBOOT_OUT),$(UBOOT_DEFCONFIG),$
(UBOOT_FRAGMENTS),$(UBOOT_SRC))

.PHONY: uboot uboot-configure uboot-menuconfig uboot-savedefconfig uboot-clean
uboot: check-cc $(UBOOT_DEPS) $(UBOOT_OUT)/.config
    $(UBOOT_MAKE) -j$(JOBS) $(if $(UBOOT_BL31_FILE),BL31=$(UBOOT_BL31_FILE))
    @mkdir -p $(IMAGES_DIR)
    cp $(addprefix $(UBOOT_OUT)/,$(UBOOT_IMAGES)) $(IMAGES_DIR)/
ifeq ($(RK_LEGACY),y)
    $(SDK_DIR)/scripts/rk-legacy-pack.sh $(RKBIN_DIR) $(UBOOT_OUT) $(RK_TRUST_BL31) $(IMAGES_
DIR)
endif

uboot-configure: check-cc $(UBOOT_OUT)/.config

uboot-menuconfig: check-cc $(UBOOT_OUT)/.config
    $(UBOOT_MAKE) menuconfig

uboot-savedefconfig: check-cc $(UBOOT_OUT)/.config
    $(call kconfig-savedefconfig,$(UBOOT_MAKE),$(UBOOT_OUT),$(UBOOT_DEFCONFIG))

uboot-clean: check-fw
    rm -rf $(UBOOT_OUT) $(addprefix $(IMAGES_DIR)/,$(notdir $(UBOOT_IMAGES))) $(RK_LEGACY_IMA
GES)

# ------------------------------------------------------------------------
# Boot firmware blobs (e.g. Raspberry Pi GPU firmware), fetched by URL and
# checked against BOOTFW_HASH, then copied to images/

ifneq ($(strip $(BOOTFW_FILES)),)
BOOTFW_DEPS := bootfw
endif

.PHONY: bootfw bootfw-clean
bootfw: check-fw
    @test -n "$(strip $(BOOTFW_FILES))" || { echo "$(FW) has no boot firmware (BOOTFW_FILES
is not set)"; exit 1; }
    $(SDK_DIR)/scripts/fetch-files.sh $(BOOTFW_URL) $(DL_DIR)/$(BOOTFW_NAME) \
        $(call sdkpath,$(BOOTFW_HASH)) $(IMAGES_DIR) $(BOOTFW_FILES)

bootfw-clean: check-fw
    rm -f $(addprefix $(IMAGES_DIR)/,$(BOOTFW_FILES))
```

```
# ------------------------------------------------------------------------------
# Linux

LINUX_OUT := $(OUT)/linux
LINUX_MAKE = $(MAKE) -C $(LINUX_SRC) O=$(LINUX_OUT) ARCH=$(LINUX_ARCH) CROSS_COMPILE=$(CROSS_
COMPILE) $(LINUX_MAKE_ARGS)

ifneq ($(FW),)
$(call inputs-stamp,$(LINUX_OUT),$(LINUX_DEFCONFIG) $(LINUX_FRAGMENTS))
endif

$(LINUX_OUT)/.config: $(LINUX_OUT)/.sdk-inputs $(filter /%,$(LINUX_DEFCONFIG)) $(LINUX_FRAGME
NTS)
    $(call kconfig-configure,$(LINUX_MAKE),$(LINUX_OUT),$(LINUX_DEFCONFIG),$
(LINUX_FRAGMENTS),$(LINUX_SRC))

.PHONY: linux linux-configure linux-menuconfig linux-savedefconfig linux-clean
linux: check-cc $(LINUX_OUT)/.config
    $(LINUX_MAKE) -j$(JOBS) $(LINUX_IMAGE) modules $(LINUX_DTBS)
    rm -rf $(LINUX_INSTALL)
    mkdir -p $(LINUX_INSTALL)/modules
    cp $(LINUX_OUT)/arch/$(LINUX_ARCH)/boot/$(LINUX_IMAGE) $(LINUX_OUT)/System.map $(LINUX_OU
T)/.config $(LINUX_INSTALL)/
    $(foreach d,$(LINUX_DTBS),install -D -m 644 $(LINUX_OUT)/arch/$(LINUX_ARCH)/boot/dts/$
(d) $(LINUX_INSTALL)/dtbs/$(d) &&) true
    $(LINUX_MAKE) INSTALL_MOD_PATH=$(LINUX_INSTALL)/modules INSTALL_MOD_STRIP=1 modules_insta
ll

linux-configure: check-cc $(LINUX_OUT)/.config

linux-menuconfig: check-cc $(LINUX_OUT)/.config
    $(LINUX_MAKE) menuconfig

linux-savedefconfig: check-cc $(LINUX_OUT)/.config
    $(call kconfig-savedefconfig,$(LINUX_MAKE),$(LINUX_OUT),$(LINUX_DEFCONFIG))

linux-clean: check-fw
    rm -rf $(LINUX_OUT) $(LINUX_INSTALL)

# ------------------------------------------------------------------------------
# Root filesystem (Buildroot)

ROOTFS_OUT := $(OUT)/buildroot
ROOTFS_MAKE = BR2_DL_DIR=$(DL_DIR) $(MAKE) -C $(BUILDROOT_SRC) O=$(ROOTFS_OUT) \
              BR2_EXTERNAL=$(BUILDROOT_EXTERNAL) $(BUILDROOT_MAKE_ARGS)

ifneq ($(FW),)
$(call inputs-stamp,$(ROOTFS_OUT),$(BUILDROOT_DEFCONFIG) $(BUILDROOT_EXTERNAL))
endif

$(ROOTFS_OUT)/.config: $(ROOTFS_OUT)/.sdk-inputs $(filter /%,$(BUILDROOT_DEFCONFIG))
    @echo ">>> Configuring $(ROOTFS_OUT)"
    $(if $(findstring /,$(BUILDROOT_DEFCONFIG)),$(ROOTFS_MAKE) defconfig BR2_DEFCONFIG=$(BUIL
DROOT_DEFCONFIG),$(ROOTFS_MAKE) $(BUILDROOT_DEFCONFIG))

.PHONY: rootfs rootfs-configure rootfs-menuconfig rootfs-savedefconfig rootfs-sdk
rootfs-make rootfs-clean
rootfs: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE)
    @mkdir -p $(IMAGES_DIR)
    cp $(ROOTFS_OUT)/images/rootfs.tar $(IMAGES_DIR)/

rootfs-configure: check-fw $(ROOTFS_OUT)/.config

rootfs-menuconfig: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) menuconfig

rootfs-savedefconfig: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) savedefconfig

# Relocatable cross toolchain + sysroot for application development
rootfs-sdk: check-fw $(ROOTFS_OUT)/.config
```

```
    $(ROOTFS_MAKE) sdk
    @mkdir -p $(IMAGES_DIR)
    cp $(ROOTFS_OUT)/images/*_sdk-buildroot.tar.gz $(IMAGES_DIR)/

# Run any Buildroot target: make rootfs-make BR=busybox-menuconfig
rootfs-make: check-fw $(ROOTFS_OUT)/.config
    $(ROOTFS_MAKE) $(BR)

rootfs-clean: check-fw
    rm -rf $(ROOTFS_OUT) $(IMAGES_DIR)/rootfs.tar

# ------------------------------------------------------------------------
# SD card image

IMAGE_FILE := $(IMAGES_DIR)/$(IMAGE_NAME).img
IMAGE_DTB_LAYOUT ?= tree

.PHONY: image all flash flash-boot clean info help
image: check-fw
    KERNEL=$(LINUX_INSTALL)/$(LINUX_IMAGE) \
    LINUX_DIR=$(LINUX_INSTALL) \
    DTB_LAYOUT=$(IMAGE_DTB_LAYOUT) \
    RAW="$(call sdkpath-pairs,$(IMAGE_RAW),@)" \
    BOOT_FILES="$(call sdkpath-pairs,$(IMAGE_BOOT_FILES),:)" \
    ROOTFS_TAR=$(IMAGES_DIR)/rootfs.tar \
    CMDLINE="$(LINUX_CMDLINE)" \
    BOOT_SIZE_MB=$(IMAGE_BOOT_SIZE_MB) \
    ROOTFS_FREE_MB=$(IMAGE_ROOTFS_FREE_MB) \
    DISK_ID=$(IMAGE_DISK_ID) \
    WORK_DIR=$(OUT)/image-work \
    HOST_TOOLS=$(ROOTFS_OUT)/host/bin \
    $(SDK_DIR)/scripts/mkimage.sh $(IMAGE_FILE)

all: uboot linux rootfs $(BOOTFW_DEPS)
    $(MAKE) FW=$(FW) image

# make flash DEV=/dev/sdX
flash: check-fw
    $(SDK_DIR)/scripts/flash.sh $(IMAGE_FILE) $(DEV) 0

# Rewrite the bootloader and the BOOT partition (kernel, dtbs, extlinux),
# keeping the MBR and the rootfs: make flash-boot DEV=/dev/sdX.
# The card must hold an image of the same firmware config.
flash-boot: check-fw
    @test -f $(IMAGES_DIR)/boot-area.sector || { echo "Run 'make image' first"; exit 1; }
    $(SDK_DIR)/scripts/flash.sh $(IMAGES_DIR)/boot-area.bin $(DEV) $$(cat $(IMAGES_DIR)/boot-
area.sector)

clean: check-fw
    rm -rf $(OUT)

gitdesc = [$$(git -C $(1) describe --always --dirty 2>/dev/null)]

info: check-fw
    @echo "Firmware:  $(FW) ($(FW_CONFIG))"
    @echo "Output:    $(OUT)"
    @echo "Compiler:  $(CROSS_COMPILE)gcc"
    @$(if $(TFA_PLAT),echo "TF-A:      $(TFA_SRC) $(call gitdesc,$(TFA_SRC)) PLAT=$
(TFA_PLAT)",true)
    @echo "U-Boot:    $(UBOOT_SRC) $(call gitdesc,$(UBOOT_SRC)) $(UBOOT_DEFCONFIG) $(notdir $
(UBOOT_FRAGMENTS))"
    @$(if $(UBOOT_BL31_FILE),echo "BL31:      $(UBOOT_BL31_FILE)",true)
    @$(if $(strip $(BOOTFW_FILES)),echo "Boot FW:   $(BOOTFW_NAME): $(BOOTFW_FILES)",true)
    @echo "Linux:     $(LINUX_SRC) $(call gitdesc,$(LINUX_SRC)) ARCH=$(LINUX_ARCH) $(LINUX_DE
FCONFIG) $(notdir $(LINUX_FRAGMENTS))"
    @echo "Buildroot: $(BUILDROOT_SRC) $(call gitdesc,$(BUILDROOT_SRC)) $
(BUILDROOT_DEFCONFIG)"
    @echo "Image:     $(IMAGE_FILE)"

help:
    @echo "Firmware config:"
    @echo "  <name>_defconfig       select configs/<name>_defconfig (or pass FW=<name>)"
```

```
    @echo "  info                   show selected config and component versions"
    @echo
    @echo "Components (each is built separately, out-of-tree):"
    @echo "  tfa                    BL31 from src/tf-a (boards with TFA_PLAT)"
    @echo "  uboot                  U-Boot (+ tfa if UBOOT_BL31=tf-a)"
    @echo "  bootfw                 fetch boot firmware blobs (boards with BOOTFW_FILES)"
    @echo "  linux                  kernel image, dtbs, modules"
    @echo "  rootfs                 Buildroot rootfs.tar"
    @echo "  rootfs-sdk             Buildroot cross toolchain + sysroot tarball"
    @echo "  rootfs-make BR=<tgt>   run a Buildroot target (e.g. BR=busybox-menuconfig)"
    @echo "  <comp>-configure / -menuconfig / -savedefconfig / -clean"
    @echo
    @echo "Image:"
    @echo "  image                  glue built components into images/<name>.img"
    @echo "  all (default)          all components + image"
    @echo "  flash DEV=/dev/sdX     write the image to an SD card"
    @echo "  flash-boot DEV=...     rewrite bootloader + BOOT partition, keep rootfs"
    @echo "  clean                  remove output/<fw>"
```

scripts/mkimage.sh

scripts/mkimage.sh

```
#!/bin/bash
# Glue separately built components into an SD card image.
#
#   sector 0        MBR (disk id DISK_ID)
#   RAW             raw bootloader blobs ("file@sector ..."), below 16 MiB
#   16M             p1 FAT32 "BOOT" (bootable): kernel, dtbs, extlinux.conf,
#                   BOOT_FILES ("src[:name] ...", e.g. MLO, RPi firmware)
#   16M+BOOT        p2 ext4 "rootfs": Buildroot rootfs.tar + kernel modules
#
# DTB_LAYOUT picks where the dtbs land in BOOT, matching how U-Boot builds
# ${fdtfile} for extlinux "fdtdir /": "tree" keeps the vendor dirs
# (rockchip/foo.dtb), "flat" puts them in the root, "both" does both.
#
# Usage: mkimage.sh <output.img>, inputs come from the environment (see Makefile).
set -euo pipefail

OUT_IMG=$1
: "${KERNEL:?}" "${LINUX_DIR:?}" "${ROOTFS_TAR:?}" "${CMDLINE:?}"
: "${BOOT_SIZE_MB:?}" "${ROOTFS_FREE_MB:?}" "${DISK_ID:?}" "${WORK_DIR:?}"
RAW=${RAW:-}
BOOT_FILES=${BOOT_FILES:-}
DTB_LAYOUT=${DTB_LAYOUT:-tree}

P1_START=32768  # 16 MiB, in 512-byte sectors

die() { echo "mkimage: $*" >&2; exit 1; }

# System tools first, Buildroot host tools as a fallback
export PATH="$PATH:/usr/sbin:/sbin${HOST_TOOLS:+:$HOST_TOOLS}"
need() { command -v "$1" >/dev/null || die "'$1' not found (package: $2)"; }
need sfdisk fdisk
need mkfs.vfat dosfstools
need mcopy mtools
need mke2fs e2fsprogs
need fakeroot fakeroot

for f in "$KERNEL" "$ROOTFS_TAR"; do
    [ -e "$f" ] || die "missing $f: build the component first"
done

# --- raw blobs: sorted by sector, must not overlap each other or p1 ---
raw_sorted=$(for r in $RAW; do echo "${r##*@} ${r%@*}"; done | sort -n)
first_sector=$P1_START
prev_end=1
while read -r sector file; do
    [ -n "$sector" ] || continue
    [ -f "$file" ] || die "missing $file: build the component first"
```

```
    [ "$sector" -ge "$prev_end" ] || die "$(basename "$file") at sector $sector overlaps the
previous blob"
    prev_end=$(( sector + ($(stat -c %s "$file") + 511) / 512 ))
    [ "$prev_end" -le $P1_START ] || die "$(basename "$file") at sector $sector overlaps the
first partition"
    if [ "$sector" -lt "$first_sector" ]; then first_sector=$sector; fi
done <<< "$raw_sorted"

rm -rf "$WORK_DIR"
mkdir -p "$WORK_DIR/boot/extlinux"

# --- boot partition ---
BOOT_DIR=$WORK_DIR/boot
cp "$KERNEL" "$BOOT_DIR/"
if [ -d "$LINUX_DIR/dtbs" ]; then
    case $DTB_LAYOUT in
    tree|both) cp -r "$LINUX_DIR/dtbs/." "$BOOT_DIR/" ;;&
    flat|both) find "$LINUX_DIR/dtbs" -name '*.dtb' -exec cp {} "$BOOT_DIR/" \; ;;
    tree) ;;
    *) die "unknown DTB_LAYOUT '$DTB_LAYOUT' (tree, flat or both)" ;;
    esac
fi

for f in $BOOT_FILES; do
    src=${f%%:*}
    dst=${f#*:}; [ "$dst" != "$f" ] || dst=$(basename "$src")
    [ -f "$src" ] || die "missing $src: build the component first"
    cp "$src" "$BOOT_DIR/$dst"
done

disk_id=$(printf '%08x' $(( DISK_ID )))
# U-Boot sets ${fdtfile} from the board model; fdtdir prepends the path.
cat > "$BOOT_DIR/extlinux/extlinux.conf" <<EOT
default linux

label linux
    kernel /$(basename "$KERNEL")
    fdtdir /
    append root=PARTUUID=$disk_id-02 $CMDLINE
EOT

boot_kb=$(du -sk "$BOOT_DIR" | cut -f1)
[ "$boot_kb" -lt $(( BOOT_SIZE_MB * 1024 * 95 / 100 )) ] || \
    die "boot files ($((boot_kb / 1024)) MiB) do not fit into IMAGE_BOOT_SIZE_MB=$BOOT_SIZE_M
B"

BOOT_IMG=$WORK_DIR/boot.vfat
mkfs.vfat -F 32 -n BOOT -i "$disk_id" -C "$BOOT_IMG" $(( BOOT_SIZE_MB * 1024 )) >/dev/null
mcopy -i "$BOOT_IMG" -s -p -m "$BOOT_DIR"/* ::/

# --- rootfs partition ---
ROOT_DIR=$WORK_DIR/rootfs
ROOT_IMG=$WORK_DIR/rootfs.ext4
fakeroot -- bash -euo pipefail -c '
    root=$1 tar=$2 mods=$3 free_mb=$4 img=$5
    mkdir -p "$root"
    tar -xpf "$tar" -C "$root"
    if [ -d "$mods/lib/modules" ]; then
        mkdir -p "$root/lib/modules"
        cp -r "$mods/lib/modules/." "$root/lib/modules/"
        chown -R 0:0 "$root/lib/modules"
    fi
    # BOOT partition on /boot, by label: the mmcblk number differs per board
    mkdir -p "$root/boot"
    sed -i "/[[:space:]]\/boot[[:space:]]/d" "$root/etc/fstab"
    printf "LABEL=BOOT\t/boot\tvfat\tdefaults,noatime\t0\t0\n" >> "$root/etc/fstab"
    size_mb=$(( $(du -sm "$root" | cut -f1) * 11 / 10 + free_mb ))
    mke2fs -q -F -t ext4 -L rootfs -d "$root" -E root_owner=0:0 "$img" "${size_mb}M"
' _ "$ROOT_DIR" "$ROOTFS_TAR" "$LINUX_DIR/modules" "$ROOTFS_FREE_MB" "$ROOT_IMG"
rm -rf "$ROOT_DIR"

# --- disk ---
```

```
boot_sectors=$(( BOOT_SIZE_MB * 2048 ))
p2_start=$(( P1_START + boot_sectors ))
root_sectors=$(( $(stat -c %s "$ROOT_IMG") / 512 ))
total_sectors=$(( p2_start + root_sectors + 2048 ))

TMP_IMG=$OUT_IMG.tmp
rm -f "$TMP_IMG"
truncate -s $(( total_sectors * 512 )) "$TMP_IMG"
sfdisk -q "$TMP_IMG" <<EOT
label: dos
label-id: $DISK_ID
start=$P1_START, size=$boot_sectors, type=c, bootable
start=$p2_start, size=$root_sectors, type=83
EOT

put() { dd if="$1" of="$TMP_IMG" bs=1M seek=$(( $2 * 512 )) oflag=seek_bytes conv=notrunc sta
tus=none; }
while read -r sector file; do
    if [ -n "$sector" ]; then put "$file" "$sector"; fi
done <<< "$raw_sorted"
put "$BOOT_IMG" "$P1_START"
put "$ROOT_IMG" "$p2_start"
mv "$TMP_IMG" "$OUT_IMG"

# Bootloader blobs + BOOT partition for "make flash-boot"
out_dir=$(dirname "$OUT_IMG")
dd if="$OUT_IMG" of="$out_dir/boot-area.bin" bs=1M skip=$(( first_sector * 512 )) \
    count=$(( (p2_start - first_sector) * 512 )) iflag=skip_bytes,count_bytes status=none
echo "$first_sector" > "$out_dir/boot-area.sector"

echo "Image: $OUT_IMG ($(( total_sectors / 2048 )) MiB)"
sfdisk -l "$OUT_IMG" | sed -n '/^Device/,$p'
echo "root=PARTUUID=$disk_id-02"
```

configs/rpi4_defconfig

configs/rpi4_defconfig

```
# Firmware: Raspberry Pi 4 Model B, Raspberry Pi 400, CM4 (BCM2711, Cortex-A72).
# The EEPROM bootloader starts the GPU firmware (start4.elf), which reads
# config.txt, loads <model>.dtb from the BOOT root and starts mainline U-Boot
# (64-bit). U-Boot picks broadcom/<model>.dtb and boots extlinux.conf.
#
# See configs/oga_defconfig for the meaning of the variables.

BOARD_DIR     := board/rpi4
CROSS_COMPILE := aarch64-linux-gnu-

# --- U-Boot ---
UBOOT_SRC       := src/u-boot
UBOOT_DEFCONFIG := rpi_4_defconfig
UBOOT_IMAGES    := u-boot.bin

# --- Raspberry Pi GPU firmware ---
BOOTFW_NAME  := rpi-firmware-1.20260915
BOOTFW_URL   := https://github.com/raspberrypi/firmware/raw/1.20260915/boot
BOOTFW_FILES := start4.elf fixup4.dat LICENCE.broadcom
BOOTFW_HASH  := board/common/rpi-firmware.sha256

# --- Linux ---
LINUX_SRC       := src/linux
LINUX_ARCH      := arm64
LINUX_IMAGE     := Image
LINUX_DEFCONFIG := defconfig
LINUX_FRAGMENTS := $(BOARD_DIR)/linux/rpi4.config
LINUX_DTBS      := broadcom/bcm2711-rpi-4-b.dtb \
                   broadcom/bcm2711-rpi-400.dtb \
                   broadcom/bcm2711-rpi-cm4-io.dtb
LINUX_CMDLINE   := rootwait rw console=tty1 console=ttyS1,115200n8
```

```
# --- Root filesystem (Buildroot) ---
BUILDROOT_SRC       := src/buildroot
BUILDROOT_EXTERNAL  := board/common/buildroot
BUILDROOT_DEFCONFIG := $(BOARD_DIR)/buildroot/defconfig

# --- SD card image ---
# Layout: [MBR][p1 FAT32 BOOT @16M: firmware, config.txt, u-boot.bin,
#          Image, dtbs][p2 ext4 rootfs]
IMAGE_NAME := rpi4
IMAGE_BOOT_FILES := $(addprefix $(IMAGES_DIR)/,$(BOOTFW_FILES) u-boot.bin) \
                    $(BOARD_DIR)/boot/config.txt
# U-Boot wants broadcom/<model>.dtb, the firmware wants <model>.dtb in the root
IMAGE_DTB_LAYOUT := both
IMAGE_BOOT_SIZE_MB   := 128
IMAGE_ROOTFS_FREE_MB := 256
IMAGE_DISK_ID        := 0x52504934
```

board/rpi4/buildroot/defconfig

board/rpi4/buildroot/defconfig

```
BR2_aarch64=y
BR2_cortex_a72=y
BR2_TOOLCHAIN_BUILDROOT_GLIBC=y
BR2_TOOLCHAIN_BUILDROOT_CXX=y
BR2_PACKAGE_HOST_GDB=y
BR2_TARGET_GENERIC_HOSTNAME="raspberrypi"
BR2_TARGET_GENERIC_ISSUE="Welcome to Raspberry Pi 4 / 400"
BR2_TARGET_GENERIC_GETTY_PORT="ttyS1"
BR2_TARGET_GENERIC_GETTY_BAUDRATE_115200=y
BR2_SYSTEM_DHCP="eth0"
BR2_PACKAGE_ALSA_UTILS=y
BR2_PACKAGE_EVTEST=y
BR2_ROOTFS_DEVICE_CREATION_DYNAMIC_MDEV=y
BR2_ROOTFS_OVERLAY="$(BR2_EXTERNAL_SDK_PATH)/rootfs-overlay"
BR2_ROOTFS_POST_BUILD_SCRIPT="$(BR2_EXTERNAL_SDK_PATH)/post-build.sh"
BR2_ROOTFS_POST_SCRIPT_ARGS="tty1"
BR2_PACKAGE_BUSYBOX_CONFIG_FRAGMENT_FILES="$(BR2_EXTERNAL_SDK_PATH)/busybox.fragment"
BR2_PACKAGE_GDB=y
BR2_PACKAGE_KMOD=y
BR2_PACKAGE_KMOD_TOOLS=y
BR2_PACKAGE_DROPBEAR=y
BR2_TARGET_ROOTFS_TAR=y
BR2_PACKAGE_HOST_MTOOLS=y
```